

spintent

SPANISH PARSE INTENTS

V0.99 2026-10-01^{*}

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	8
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	8
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	10
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	17
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	34
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	35
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	36
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	39
			12	Índice de la Implementación	163

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con MathCAT[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de *The LaTeX Tagged PDF Project* y escucharlo con NVDA/MathCAT para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/MathCAT.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de MathML. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

^{*}Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-10-01.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/lualatex/spintent/
spintent.lua  ⇒ TDS:tex/lualatex/spintent/
spintent.pdf  ⇒ TDS:doc/lualatex/spintent/
README.md    ⇒ TDS:doc/lualatex/spintent/
spintent.dtx ⇒ TDS:source/lualatex/spintent/
spintent.ins  ⇒ TDS:source/lualatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{comando}{key-uno = valor, key-dos = valor, key-tres = valor, ...}`

El comando `\setspintent` establece las llaves (*keys*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*keys*) definidas en el *argumento opcional* [*key = val*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*key = val*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*key = val*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{<número>}` `\spnum{<número,número>}` `\spnum{<número.número>}` `\spnum{<número;número>}`
`\spnum[key = val]{<número,número;número>}` `\spnum[key = val]{<número,número;número e exponente>}`
`\spnum[key = val]{<número.número;número>}` `\spnum[key = val]{<número.número;número e exponente>}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<número>}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una *coma* ',' o con un *punto* '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un *punto y coma* ';'.

Si no se proporciona el *argumento opcional* [*key = val*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {<número>} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La *coma* ',' o el *punto* '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {<argumento>}. El {<argumento>} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {true | false}` default: true

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\', entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '10 00 00' en la salida PDF.

`read-sign = {terse | clear}` default: terse

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en `terse`, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {coma | punto}` default: coma

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ';' de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ',' en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá '.' en la

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

salida PDF y NVDA/MathCAT verbalizará «punto». Por ejemplo $\$ \backslash \text{spnum}\{3;4\}$ imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma cuatro periódico» y $\$ \backslash \text{spnum}[\text{read-period-sep=punto}]\{3;4\}$ imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «tres punto cuatro periódico»

- Esta llave es necesaria SOLO cuando se utilizan «decimales periódicos puros», es decir, cuando no se ha incluido en el $\{ \langle \text{argumento} \rangle \}$ una coma ‘,’ o un punto ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = { \langle times \rangle | \langle cdot \rangle }`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «notación científica» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «por»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por». Por ejemplo $\$ \backslash \text{spnum}\{3,14\text{e}3\}$ imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo» y $\$ \backslash \text{spnum}[\text{notation-sym=times}]\{3,14\text{e}3\}$ imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo».

- Decir «notación científica» aquí es un poco exagerado, en realidad es «multiplicación por potencias de 10», no se verifica si la entrada cumple o no con la condición de estar escrita en «notación científica».

3.2 El comando `\spunit`

<code>\spunit</code>	<code>\spunit\{ \langle unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>
	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \langle unidad \rangle \}</code>		<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \langle unidad / unidad \rangle \}</code>
	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \langle unidad \rangle \langle unidad \rangle \}</code>		<code>\spunit[\langle key = val \rangle]\{ \langle número e exponente unidad \rangle \}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle unidad \rangle \}$, el cual puede contener una «parte numérica» opcional la cual se procesa internamente por el comando `\spnum` al inicio del $\{ \langle argumento \rangle \}$ seguido de una $\{ \langle unidad \rangle \}$ o un $\{ \langle alias \rangle \}$ valido descritos en §3.2.2 o una $\{ \langle unidad \rangle \}$ creada por el comando `\spnewunit` (§3.2.3) o un $\{ \langle alias \rangle \}$ creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \backslash \text{spunit}\{23 \text{ m}\}$ imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros».

La multiplicación entre las $\{ \langle unidades \rangle \}$ se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^ $\{ \langle exponente \rangle \}$ ’. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m} * \text{s}^{\{2\}}\}$ se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros segundos al cuadrado».

Se pueden usar *fracciones lineales* con las $\{ \langle unidades \rangle \}$ utilizando una «única barra» ‘/’ para la separación entre las $\{ \langle unidades \rangle \}$ del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo: $\$ \backslash \text{spunit}\{23 \text{ m} / \text{s}^{\{2\}}\}$ imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros por segundos al cuadrado».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El $\{ \langle argumento \rangle \}$ puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\{#1\}}` o `\mathbf{\{#1\}}` por ejemplo, NO puede contener más de una «única barra» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para `\spunit`

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de $\{ \langle unidades \rangle \}$. Si se establece en `true`, se imprimirá ‘·’ entre las $\{ \langle unidades \rangle \}$ en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las $\{ \langle unidades \rangle \}$ en la salida PDF.

`read-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de $\{ \langle unidades \rangle \}$. Si se establece en `true`, NVDA/MathCAT verbalizará «por» entre las $\{ \langle unidades \rangle \}$; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las $\{ \langle unidades \rangle \}$.

`read-slash = { \langle por \rangle | \langle partido-por \rangle }`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «barra» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «partido por».

3.2.2 Argumentos para `\spunit`

Resumen de $\{ \langle unidades \rangle \}$ y $\{ \langle alias \rangle \}$ soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\nueva unidad}\{verbalización NVDA}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\langle \textit{nueva unidad} \rangle$ que establece lo que se imprimirá en la salida PDF y $\langle \textit{verbalización NVDA} \rangle$ que establece como NVDA/MathCAT verbalizará la $\langle \textit{nueva unidad} \rangle$.

La `{\nueva unidad}` no debe estar registrada como `{\nunidad}` o `{\nalias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\langle \text{nueva unidad} \rangle$ es «global» y se almacena en el módulo `spintent.lua` como una $\langle \text{unidad} \rangle$ valida para pasar como $\langle \text{argumento} \rangle$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\langle \text{nueva unidad} \rangle$ ‘px’ y cuando se utilice `$\spunit{100 px}$` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{⟨nuevo alias⟩}{⟨expresión de unidades⟩}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nuevo alias} \rangle\}$ que establece es el nuevo $\{\langle \text{alias} \rangle\}$ de $\{\langle \text{argumento} \rangle\}$ para `\spunit` y $\{\langle \text{expresión de unidades} \rangle\}$ que puede contener multiplicación o división de $\{\langle \text{unidades} \rangle\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{\nuevo alias}` es «global» y se almacena en el módulo `spintent`. Lua como una `{\alias}` valido para pasar como `{\argumento}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^{\textstyle 3}}/s}` creará el `{\alias}` ‘caudal’ y cuando se utilice `\$ \spunit{50 caudal} \$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<u>\spmoney</u>	\spmoney{ <i>{cifra}</i> }	\spmoney[<i>{key = val}</i>]{ <i>{cifra}</i> }
	\spmoney{ <i>{cifra símbolo moneda}</i> }	\spmoney[<i>{key = val}</i>]{ <i>{cifra símbolo moneda}</i> }
	\spmoney{ <i>{cifra alias moneda}</i> }	\spmoney[<i>{key = val}</i>]{ <i>{cifra alias moneda}</i> }

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \textit{cifra} \rangle \}$ que contiene la «cifra numérica» positiva o negativa la cual se procesa internamente por el comando `\spnum` acompañada de «símbolo moneda» o «alias moneda» opcionales descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmoney{500}` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase «*símbolo moneda*» o «*alias moneda*» en el argumento *obligatorio* donde tienen prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dolares}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dólares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá 'CLP500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el «*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 clp} \$` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo moneda*» como '\$', '€', '£' o un «*alias moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT. Los «*alias moneda*» se pueden pasar con o sin tilde, por ejemplo dólares o dólares.

- La posición del «*símbolo moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando (o del *modo texto* si se establece `money-math=false`), se debe tener presente en caso de querer usar un «*símbolo moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, con la única excepción de `\$`. La coma `'`, o el punto `.` como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la `(meta-key)` *cifras-sep* pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: *peso*

Establece el *símbolo moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo moneda*» o el «*alias moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: *false*

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares} \$` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

- Si la «*cifra numérica*» no tiene parte decimal, la llave `sub-units` no tiene efecto. Si la moneda en curso no admite *subunidades* se emite una advertencia y la llave tampoco tiene efecto.

`money-math = {(true | false)}` default: *true*

Establece si el *símbolo moneda* se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir el *símbolo moneda* en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir el *símbolo moneda* en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los «*símbolos moneda*» y «*alias moneda*» soportados en el `{(argumento)}` de `\spmoney`.

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	« <i>peso / pesos</i> »
dolar	dolares, dólar, dólares	dólar	\$	« <i>dólar / dólares</i> »
€	euro, euros	euro	€	« <i>euro / euros</i> »
£	libra, libras	libra	£	« <i>libra / libras</i> »
¥	yen, yenes	yen	¥	« <i>yen / yenes</i> »
yuan	yuanes	yuan	¥	« <i>yuan / yuanes</i> »
₩	won, wons	won	₩	« <i>won / wons</i> »
₪	shekel, shekels, sequel, sequeles	séquel	₪	« <i>séquel / séqueles</i> »
₹	rupia, rupias	rupia	₹	« <i>rupia / rupias</i> »
₡	rublo, rublos	rublo	₡	« <i>rublo / rublos</i> »
₺	lira, liras	lira	₺	« <i>lira / liras</i> »
₴	grivna, grivnas	grivna	₴	« <i>grivna / grivnas</i> »
¢	centavo, centavos	centavo	¢	« <i>centavo / centavos</i> »
Divisas en formato ISO (Mayúsculas)				
CLP	—	peso chileno	CLP	« <i>peso chileno / pesos chilenos</i> »
MXN	—	peso mexicano	MXN	« <i>peso mexicano / pesos mexicanos</i> »
USD	—	dólar	USD	« <i>dólar estadounidense / dólares estadounidenses</i> »
EUR	—	euro	EUR	« <i>euro / euros</i> »
GBP	—	libra	GBP	« <i>libra / libras</i> »

Continúa en la siguiente página...

Cuadro 2 – Continuación de la página anterior

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	—	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4 El comando \spshort

```
\spshort {<abreviaturas | acrónimos | números ordinales>}
\spshort [key = val] {<abreviaturas | acrónimos | números ordinales>}
\spshort [read-arg={<verbalización NVDA>}] {<comandos LATEX>}
```

El comando `\spshort` trabaja solo en modo texto y recibe un argumento obligatorio del tipo `{<abreviaturas>}`, `{<acrónimos>}` o `{<números ordinales>}` descritos en §3.4.2.

Si no se proporciona el argumento opcional `[key = val]` y se ejecuta `\spshort{dr.a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{<números ordinales>}` se puede ejecutar `\spshort{1.a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1.o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear un nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del argumento obligatorio que puede contener `{<comandos LATEX>}` en modo texto los cuales se imprimirán en salida PDF.

- Para la escritura de `{<números ordinales>}` se debe tener cuidado y no confundir la «letra o voladita» ‘o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del modo texto está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para `shorthands` y el comando `\sptext` provistos por el paquete `babel` en español compatible con `tagged` PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja solo en modo texto. Internamente utiliza la llave `E` (expansion text) proveída por el paquete `tagpdf`[29].

3.4.1 Llaves para \spshort

```
read-arg = {<verbalización NVDA>} default: no usado
Establece la verbalización del argumento {<comandos LATEX>} de la nueva {<abreviatura>}, {<acrónimo>}
o {<número ordinal>} para NVDA. El valor de esta llave siempre debe pasarse entre '{ }'. Por ejemplo:
\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}} creará una nueva {<abreviatura>}
que imprimirá ‘8.avo’ en la salida PDF y NVDA verbalizará «octavo».

small-caps = {<true | false>} default: false
Establece si se usarán versalitas para imprimir {<acrónimos>} y para la «letra volada» en {<abreviaturas>} y
{<números ordinales>} respetando las reglas de la RAE para el uso de estas. Si se establece en true, las abreviaturas
con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se
establece en false, no se aplicarán versalitas. Por ejemplo: \spshort[small-caps=true]{dr.a} imprimirá
‘Dr.A’ en la salida PDF y NVDA verbalizará «doctora».
```

3.4.2 Argumentos para \spshort

El cuadro 3 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 4 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	—	ONG	«organización no gubernamental»
ongs	—	ONGs	«organizaciones no gubernamentales»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 5 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 5: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>} \spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el argumento obligatorio `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘XXI’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El argumento obligatorio `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

`print-era = {⟨ac⟩ | ⟨dc⟩ | none}`

default: *none*

Establece si se *imprime* o no la abreviatura después de imprimir el argumento `{⟨siglo⟩}`. Si se establece en *ac*, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xxi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en *dc*, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xxi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en *none*, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

`\sptime` `\sptime{⟨horas⟩ : ⟨minutos⟩}` `\sptime{⟨horas⟩ : ⟨minutos⟩ : ⟨segundos⟩}`

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨horas⟩ : ⟨minutos⟩}` o `{⟨horas⟩ : ⟨minutos⟩ : ⟨segundos⟩}`. Por ejemplo: `$\sptime{23:45}$` imprimirá en «modo texto» la salida PDF ‘23:45’ y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

`\spdate` `\spdate{⟨DD/MM/YYYY⟩}` `\spdate{⟨YYYY/MM/DD⟩}` `\spdate{⟨YYYY-MM-DD⟩}` `\spdate{⟨DD-MM-YYYY⟩}`

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨DD/MM/YYYY⟩}`, `{⟨DD-MM-YYYY⟩}` o `{⟨YYYY-MM-DD⟩}`. Por ejemplo: `$\spdate{1981-01-02}$` y `$\spdate{02/01/1981}$` imprimirán en «modo texto» la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento `{⟨YYYY/MM/DD⟩}`, internamente se convertirá en `{⟨DD/MM/YYYY⟩}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee cuatro comandos `\spnewread`, `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spnewread`

`\spnewread` `\spnewread{⟨símbolo⟩}{⟨verbalización NVDA⟩}`

El comando `\spnewread` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{⟨símbolo⟩}` y `{⟨verbalización NVDA⟩}` los cuales se almacenan de forma interna para ser utilizados como como `{⟨argumento⟩}` en `\spread`. Solo se establece como NVDA/MathCAT verbalizará el `{⟨símbolo⟩}` al utilizarlo con `\spread` sin modificar la impresión de este.

El argumento `{⟨símbolo⟩}` no debe estar registrado. Si ya está registrado se devolverá un “error” y se detendrá la compilación del documento. La declaración es «global» y se almacena en el módulo `spintent.lua`. Por ejemplo: `\spnewread{\cong}{es-congruente-con}` registrará ‘`\cong`’ y cuando se utilice `$\spread{\cong}$` se imprimirá ‘ $\cong$ ’ en la salida PDF y NVDA/MathCAT verbalizará «es congruente con».

El comando `\spread`

`\spread` `\spread{⟨símbolo⟩}` `\spread[⟨key = val⟩]{⟨símbolo⟩}`
`\spread[⟨read-arg={⟨verbalización NVDA⟩}⟩]{⟨comandos  $\TeX$ ⟩}`

El comando `\spread` trabaja en *modo matemático* y recibe el *argumento obligatorio* de la forma `{⟨símbolo⟩}` previamente registrado por el comando `\spnewread` o `{⟨comandos  $\TeX$ ⟩}` para ser usado junto a la llave `read-arg`.

Si el argumento es `{⟨símbolo⟩}`, `$\spread{\cong}$` imprimirá ‘ $\cong$ ’ en la salida PDF y el argumento `{⟨verbalización NVDA⟩}` establecido al registrar `{⟨símbolo⟩}` con `\spnewread` se verbalizará en NVDA/MathCAT.

Si el argumento es `{⟨comandos  $\TeX$ ⟩}` y no se establece la llave `read-arg` se devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spread`

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Establece la *verbalización* del argumento $\{\langle\text{comandos L}\text{T}\text{E}\text{X}\rangle\}$. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y sobrescribe la verbalización del $\{\langle\text{símbolo}\rangle\}$ si está registrado previamente. Por ejemplo: `\spread[read-arg={es-paralela-a}]{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela a».

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del argumento $\{\langle\text{símbolo}\rangle\}$ o $\{\langle\text{comandos L}\text{T}\text{E}\text{X}\rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- Al utilizar *silent* no es necesario que el argumento $\{\langle\text{símbolo}\rangle\}$ esté registrado o usar la llave *read-arg* con $\{\langle\text{comandos L}\text{T}\text{E}\text{X}\rangle\}$. Dependiendo de que contenga $\{\langle\text{argumento}\rangle\}$ puede que *silent* no sea suficiente, en ese caso utilice *read-arg={_}*.

El comando `\spusep`

`\spusep` `\spusep{\{[ ( ) ] \}}` `\spusep[key = val]{\{[ ( ) ] \}}`

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\{[()] \}\}$, el cual debe ser «solo uno» de los paréntesis soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’ o ‘\’.

Si no se proporciona el *argumento opcional* $[key = val]$, `\spusep{}` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «abrir paréntesis» acorde a su propia heurística.

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaT_EX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

`size = {⟨none | big | Big | bigg | Bigg⟩}`

default: *none*

Establece el *tamaño del paréntesis* pasado como $\{\langle\text{argumento}\rangle\}$ que se imprimirá en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *none*, se imprime el $\{\langle\text{argumento}\rangle\}$ en el tamaño por defecto del modo matemático en curso; si se establece en *big*, se usa `\bigl/\bigr` antes del $\{\langle\text{argumento}\rangle\}$; si se establece en *Big*, se usa `\Bigl/\Bigr` antes del $\{\langle\text{argumento}\rangle\}$; si se establece en *bigg*, se usa `\biggl/\biggr` antes del $\{\langle\text{argumento}\rangle\}$; y si se establece en *Bigg*, se usa `\Biggl/\Biggr` antes del $\{\langle\text{argumento}\rangle\}$.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* en NVDA/MathCAT la verbalización del $\{\langle\text{argumento}\rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spdsym`

`\spdsym` `\spdsym[key = val]`

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* $[key = val]$ mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[key = val]$, `\spdsym` imprimirá ‘:’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el $\{\langle\text{valor}\rangle\}$ establecido por la llave *read-sym*. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`suffix = {⟨sufijo⟩}`

default: *no usado*

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el $\{\langle\text{valor}\rangle\}$ establecido por la llave *read-sym*. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`set-sym = {⟨old-style | two-dots | slash⟩}`

default: *two-dots*

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en *old-style*, imprimirá ‘÷’; si se establece en *two-dots*, imprimirá ‘:’ y si se establece *slash*, imprimirá ‘/’.

`read-sym = {⟨dividido | dividido-por | dividido-entre⟩}`

default: *dividido*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en *dividido*, NVDA/MathCAT verbalizará «dividido»; si se establece en *dividido-por*, NVDA/MathCAT verbalizará «dividido por» y si se establece en *dividido-entre*, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

 $\spmsym$ $\spmsym$ $\spmsym[\langle key = val \rangle]$

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* $[\langle key = val \rangle]$ mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \spmsym \$$ imprimirá ‘ $\cdot$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*por*».

Llaves para `\spmsym`

$prefix = \{\langle prefijo \rangle\}$ default: *no usado*

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT «*antes*» de *verbalizar* el $\{\langle valor \rangle\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

$suffix = \{\langle sufijo \rangle\}$ default: *no usado*

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT «*después*» de *verbalizar* el $\{\langle valor \rangle\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

$set-sym = \{\langle cross \mid dot \mid asterisk \rangle\}$ default: *dot*

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá ‘ $\times$ ’; si se establece en `dot`, imprimirá ‘ $\cdot$ ’ y si se establece en `asterisk`, imprimirá ‘ $*$ ’.

$read-sym = \{\langle por \mid multiplicado-por \mid veces \rangle\}$ default: *por*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «*multiplicado por*» y si se establece en `veces`, NVDA/MathCAT verbalizará «*veces*».

$not-print = \{\langle true \mid false \rangle\}$ default: *false*

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

 $\spdiv$ $\spdiv\{\langle dividendo \rangle\}\{\langle divisor \rangle\}$ $\spdiv[\langle key = val \rangle]\{\langle dividendo \rangle\}\{\langle divisor \rangle\}$

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* $\{\langle dividendo \rangle\}$ y $\{\langle divisor \rangle\}$.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \spdiv \{20\}\{26\} \$$ imprimirá ‘ $20 : 26$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*veinte dividido veinte y seis*»

Llaves para `\spdiv`

El comando `\spdiv` posee las *(meta-keys)* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *(meta-keys)* `set-sym` y `read-sym` pasadas al comando `\spdsym` (pág. 12).

$wrap-parens = \{\langle true \mid false \rangle\}$ default: *false*

Establece si se *imprimen* paréntesis redondos ‘ $(...)$ ’ alrededor de los *argumentos* $\{\langle dividendo \rangle\}$ y $\{\langle divisor \rangle\}$ pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

$read-parens = \{\langle true \mid false \rangle\}$ default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘ $(...)$ ’ alrededor de los *argumentos* $\{\langle dividendo \rangle\}$ y $\{\langle divisor \rangle\}$ pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

 $\spmul$ $\spmul\{\langle factor \rangle\}\{\langle factor \rangle\}$ $\spmul[\langle key = val \rangle]\{\langle factor \rangle\}\{\langle factor \rangle\}$

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* $\{\langle factor \rangle\}$ y $\{\langle factor \rangle\}$.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \spmul \{20\}\{26\} \$$ imprimirá ‘ $20 \cdot 26$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*veinte por veinte y seis*».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* $\{\langle factor \rangle\}$ y $\{\langle factor \rangle\}$ pasados a `\spmul`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = $\{\langle true \mid false \rangle\}$ default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* $\{\langle factor \rangle\}$ y $\{\langle factor \rangle\}$ pasados a `\spmul`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

`\spnD` `\spnD` `\spnD(\langle número \rangle)` `\spnD[\langle key = val \rangle](\langle número \rangle)` `\spnD(\langle letra \rangle)` `\spnD[\langle key = val \rangle](\langle letra \rangle)`

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle número \rangle)$ que se procesa internamente por `\spnum` o $(\langle letra \rangle)$ que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$ ni el *argumento opcional delimitado por paréntesis*, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de ene».

Si se usa el argumento $(\langle número \rangle)$, `\spnD(6)` imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Si se usa el argumento $(\langle letra \rangle)$, `\spnD(m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de eme».

Llaves para `\spnD`

`concept` = $\{\langle true \mid false \rangle\}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «divisores de» y el $(\langle argumento \rangle)$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $(\langle argumento \rangle)$.

`prefix` = $\{\langle prefijo \rangle\}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «divisores de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *imprime* el resultado de calcular los *divisores* del argumento $(\langle número \rangle)$. Si se establece en *true*, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis son uno, dos, tres y seis»; si se establece en *false*, no se imprimirá ni verbalizará el resultado de calcular los *divisores*.

`result-num` = $\{\langle número \text{ natural} \rangle\}$ default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en *2*, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo *5* valores y luego se añadirá ‘...’ en la salida PDF.

`silent` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» y del $(\langle argumento \rangle)$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnM`

`\spnM` `\spnM` `\spnM(\langle número \rangle)` `\spnM[\langle key = val \rangle](\langle número \rangle)` `\spnM(\langle letra \rangle)` `\spnM[\langle key = val \rangle](\langle letra \rangle)`

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle número \rangle)$ que se procesa internamente por `\spnum` o $(\langle letra \rangle)$ que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$ ni el *argumento opcional delimitado por paréntesis*, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de ene».

Si se usa el argumento $(\langle número \rangle)$, `\spnM(6)` imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Si se usa el argumento $(\langle letra \rangle)$, `\spnM(m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de eme».

Llaves para \spnM

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*múltiplos de*» al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT verbalizará «*múltiplos de*» y el (*argumento*); si se establece en **false**, NVDA/MathCAT solo verbalizará el (*argumento*).

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*múltiplos de*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {⟨true | false⟩}` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del argumento (*número*). Si se establece en **true**, `\spnM[result=true](6)` imprimirá 'M(6) = {6, 12, 18, 24, 30, ...}' en la salida PDF y NVDA/MathCAT verbalizará «*múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos*»; si se establece en **false**, no se imprimirá ni verbalizará el resultado de calcular los *múltiplos*.

`result-num = {⟨número natural⟩}` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá '...' en la salida PDF.

`silent = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*multiplos de*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \spgcd

`\spgcd` `\spgcd` `\spgcd(⟨número, número, ...⟩)` `\spgcd(⟨letra, letra, ...⟩)` `\spgcd[key = val](⟨número, número, ...⟩)`

El comando `\spgcd` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spgcd` imprimirá 'mcd' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor*».

Si se usa el argumento (*número, número, ...*), `\spgcd(2, 3, 6)` imprimirá 'mcd(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), `\spgcd(m, n)` imprimirá 'mcd(m, n)' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre eme y ene*».

- Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para \spgcd

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT verbalizará «*máximo común divisor ...*» o «*m c d ...*» y el (*argumento*); si se establece en **false**, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {⟨terse | clear⟩}` default: clear

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en **terse**, NVDA/MathCAT verbalizará «*m c d*»; si se establece en **clear**, NVDA/MathCAT verbalizará «*máximo común divisor*».

`upper-case = {⟨true | false⟩}` default: false

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en **true**, se imprimirá 'MCD' en la salida PDF; si se establece en **false**, se imprimirá 'mcd' en la salida PDF.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {⟨true | false⟩}` default: false

Establece si se *imprime* el resultado de calcular el «*máximo común divisor*» del argumento (*número, número, ...*). Si se establece en **true**: `\spgcd[result=true](2, 3, 6)` imprimirá 'mcd(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis es igual a uno*»; si se establece en **false** no se imprimirá ni verbalizará el resultado el resultado de calcular el «*máximo común divisor*».

`silent = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*máximo común divisor ...*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \spmcn

\spmcn **\spmcn** **\spmcn**(*número, número, ...*) **\spmcn**(*letra, letra, ...*) **\spmcn**[*key = val*](*número, número, ...*)

El comando **\spmcn** trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por **\spnum** o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, **\spmcn** imprimirá 'mcm' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

Si se usa el argumento (*número, número, ...*), **\spmcn(2, 3, 6)** imprimirá 'mcm(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), **\spmcn(m, n)** imprimirá 'mcm(*m, n*)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre eme y ene*».

Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para \spmcn

concept = {*true* | *false*} default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave **read-cmd** al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT verbalizará «*mínimo común múltiplo ...*» o «*m c m ...*» y el (*argumento*); si se establece en **false**, NVDA/MathCAT solo verbalizará el (*argumento*).

read-cmd = {*terse* | *clear*} default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en **terse**, NVDA/MathCAT verbalizará «*m c d*»; si se establece en **clear**, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

upper-case = {*true* | *false*} default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en **true**, se imprimirá 'MCM' en la salida PDF; si se establece en **false**, se imprimirá 'mcm' en la salida PDF.

prefix = {*prefijo*} default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave **read-cmd**. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

result = {*true* | *false*} default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del argumento (*número, número, ...*). Si se establece en **true**: **\spmcn[result=true](2, 3, 6)** imprimirá 'mcm(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a uno*»; si se establece en **false** no se imprimirá ni verbalizará el resultado de calcular el «*mínimo común múltiplo*».

silent = {*true* | *false*} default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*mínimo común múltiplo ...*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \spnZ

\spnZ **\spnZ**{*número entero*} **\spnZ**[*key = val*]{*número entero*}

El comando **\spnZ** trabaja en *modo matemático* y recibe el *argumento* obligatorio {*número entero*} el cual se procesa internamente bajo el comando **\spnum**. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para \spnZ

El comando **\spnZ** posee las *meta-keys* **cifras-sep** y **read-sign** pasadas al comando **\spnum** (§3.1.1).

wrap-parens = {*true* | *false*} default: *false*

Establece si se *imprimen* paréntesis redondos '(...)' alrededor del argumento {*número entero*} pasado a **\spnZ**. Si se establece en **true**, se imprimirán paréntesis redondos alrededor del {*número entero*}; si se establece en **false**, no se imprimirán los paréntesis.

read-parens = {*true* | *false*} default: *false*

Establece si NVDA/MathCAT verbalizará los paréntesis redondos '(...)' alrededor del *argumento* {*número entero*} pasado a **\spnZ**. Si se establece en **true**, NVDA/MathCAT verbalizará la lectura del paréntesis '(...)'; si se establece en **false**, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` $\text{\spmQ}\langle\text{entero}\rangle\langle\text{numerador}\rangle\langle\text{denominador}\rangle$ $\text{\spmQ}[\text{key} = \text{val}]\langle\text{entero}\rangle\langle\text{numerador}\rangle\langle\text{denominador}\rangle$

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: $\langle\text{número entero}\rangle$, $\langle\text{numerador}\rangle$ y $\langle\text{denominador}\rangle$, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* $[\text{key} = \text{val}]$, $\text{\spmQ}\{2\}\{5\}\{7\}$ imprimirá $2\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de `spintent`. Por ejemplo, la entrada $\text{\$3}\text{\frac{5}{7}}\text{\$}$ con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word` = $\{\langle\text{entero} \mid \text{enteros} \mid \text{y}\rangle\}$ default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en *entero*, $\text{\$}\text{\spmQ}[\text{join-word}=\text{entero}]\{1\}\{5\}\{7\}\text{\$}$ imprimirá $1\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*un entero cinco séptimos*»; si se establece en *enteros*, $\text{\$}\text{\spmQ}[\text{join-word}=\text{enteros}]\{2\}\{5\}\{7\}\text{\$}$ imprimirá $2\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*dos enteros cinco séptimos*»; si se establece en *y*, $\text{\$}\text{\spmQ}[\text{join-word}=\text{y}]\{2\}\{5\}\{7\}\text{\$}$ imprimirá $2\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

`use-spnZ` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *true*

Establece si el $\langle\text{numerador}\rangle$ y el $\langle\text{denominador}\rangle$ que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en *true*, $\langle\text{numerador}\rangle$ y $\langle\text{denominador}\rangle$ se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en *false*, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\text{\$}\text{\spmQ}[\text{print-dfrac}=\text{true}]\{2\}\{5\}\{7\}\text{\$}$ imprimirá la fracción en tamaño *display* $2\frac{5}{7}$ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en *true*, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría

El paquete `spintent` provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como $\langle\text{argumento}\rangle$ base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben $\langle\text{argumento}\rangle$ único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘’ y los subíndices siempre deben escribirse usando llaves ‘A_{...}’ y nunca sin estas ‘A_n’ que funciona solo por coincidencia. .

El comando `\spcoord`

`\spcoord` $\text{\spcoord}\langle\text{abscisa}, \text{ordenada}\rangle$ $\text{\spcoord}[\text{key} = \text{val}]\langle\text{abscisa}, \text{ordenada}\rangle$
 $\text{\spcoord}\langle\text{abscisa}; \text{ordenada}\rangle$ $\text{\spcoord}[\text{key} = \text{val}]\langle\text{abscisa}; \text{ordenada}\rangle$

El comando `\spcoord` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\langle\text{abscisa}, \text{ordenada}\rangle$ o $\langle\text{abscisa}; \text{ordenada}\rangle$ el cual se procesa internamente bajo el comando `\spnum` cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* $[\text{key} = \text{val}]$, $\text{\$}\text{\spcoord}\{-3, 4\}\text{\$}$ imprimirá ‘(-3, 4)’ en la salida PDF y NVDA/MathCAT verbalizará «*menos tres coma cuatro*»; $\text{\$}\text{\spcoord}\{x_{\{1\}}, y_{\{1\}}\}\text{\$}$ imprimirá ‘(x₁, y₁)’ y verbalizará «*exis subíndice uno y subíndice uno*».

- El comando `\spcoord` solo acepta *un separador* ‘,’ o ‘;’ para la abscisa y la ordenada cuando son valores numéricos no fraccionarios.

Llaves para `\spcoord`

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoint`

<code>\spPoint</code>	<code>\spPoint{⟨punto⟩}</code>	<code>\spPoint[⟨key = val⟩]{⟨punto⟩}</code>
	<code>\spPoint{⟨punto⟩}(⟨abscisa, ordenada⟩)</code>	<code>\spPoint[⟨key = val⟩]{⟨punto⟩}(⟨abscisa, ordenada⟩)</code>
	<code>\spPoint{⟨punto⟩}(⟨abscisa; ordenada⟩)</code>	<code>\spPoint[⟨key = val⟩]{⟨punto⟩}(⟨abscisa; ordenada⟩)</code>

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto⟩}` el cual es una *única letra mayúscula* con prima `'` o subíndice `'_{...}'` y el *argumento opcional delimitado por paréntesis* `(⟨abscisa, ordenada⟩)` el cual se procesa internamente bajo el comando `\spcoord`.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spPoint{A}` imprimirá *A* en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá *A₁* y verbalizará «punto A sub uno».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spPoint{A}(-3,4)` imprimirá *A(-3,4)* en la salida PDF y NVDA/MathCAT verbalizará «punto A menos tres coma cuatro»; `\spPoint{A_{1}}(-3,4)` imprimirá *A₁(-3,4)* y verbalizará «punto A sub uno menos tres coma cuatro».

Llaves para `\spPoint`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}` sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «punto A»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «punto». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el `{⟨argumento⟩}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{⟨letra⟩}</code>	<code>\spvec{⟨punto, punto⟩}</code>
	<code>\spvec{⟨letra⟩}(⟨abscisa, ordenada⟩)</code>	<code>\spvec{⟨punto, punto⟩}(⟨abscisa, ordenada⟩)</code>
	<code>\spvec[⟨key = val⟩]{⟨letra⟩}(⟨abscisa, ordenada⟩)</code>	<code>\spvec[⟨key = val⟩]{⟨punto, punto⟩}(⟨abscisa, ordenada⟩)</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` (minúscula) o `{⟨punto, punto⟩}` junto al *argumento opcional delimitado por paréntesis* `(⟨abscisa, ordenada⟩)` el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma `{⟨punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (minúscula) con subíndice `'_{...}'` opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spvec{a}` imprimirá *$\vec{a}$* en la salida PDF y NVDA/MathCAT verbalizará «vector a»; `\spvec{AB}` imprimirá *$\overrightarrow{AB}$* y verbalizará «vector A B».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spvec{a}(-3,4)` imprimirá *$\vec{a} = (-3,4)$* en la salida PDF y NVDA/MathCAT verbalizará «vector a igual a menos tres coma cuatro»; `\spvec{AB}(-3,4)` imprimirá *$\overrightarrow{AB} = (-3,4)$* y verbalizará «vector A B igual a menos tres coma cuatro».

Llaves para `\spvec`

<code>read-cmd</code>	<code>= {⟨school mathcat⟩}</code>	default: <i>school</i>
Establece la <i>forma</i> en la que NVDA/MathCAT verbalizará el argumento <code>{⟨punto, punto⟩}</code> . Si se establece en <i>school</i> , NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: <code>\$\spvec{AB}\$</code> se verbaliza «vector A B»; si se establece en <i>mathcat</i> , NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.		
<code>prefix</code>	<code>= {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT <i>antes</i> de verbalizar el concepto «vector». El valor de esta llave <i>siempre</i> debe pasarse entre llaves <code>{ }</code> .		
<code>autofit</code>	<code>= {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>utiliza</i> <code>\vphantom{b}</code> al ejecutar el comando cuando se utiliza el argumento <code>{⟨letra⟩}</code> . Si se establece en <i>true</i> , se aplicará <code>\vphantom{b}</code> al inicio del argumento <code>{⟨letra⟩}</code> ; si se establece en <i>false</i> , no se aplicará <code>\vphantom{b}</code> .		
<code>concept</code>	<code>= {⟨true false⟩}</code>	default: <i>true</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en <i>true</i> , NVDA/MathCAT verbalizará el concepto y el <code>{⟨argumento⟩}</code> ; si se establece en <i>false</i> , NVDA/MathCAT solo verbalizará el <code>{⟨argumento⟩}</code> acorde al valor establecido por la llave <code>read-cmd</code> .		
<code>silent</code>	<code>= {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT del concepto «vector» y del <code>{⟨argumento⟩}</code> pasado al comando sin modificar la impresión de este. Si se establece en <i>true</i> , NVDA/MathCAT NO verbalizará la salida del comando; si se establece en <i>false</i> , NVDA/MathCAT sí verbalizará la salida del comando.		

El comando `\sprecta`

<code>\sprecta</code>	<code>\sprecta{⟨punto, punto⟩}</code>	<code>\sprecta{⟨letra⟩}</code>
	<code>\sprecta[key = val]{⟨punto, punto⟩}</code>	<code>\sprecta[key = val]{⟨letra⟩}</code>
El comando <code>\sprecta</code> trabaja en <i>modo matemático</i> y recibe un <i>argumento obligatorio</i> de la forma <code>{⟨letra⟩}</code> o <code>{⟨punto, punto⟩}</code> con prima(s) <code>'</code> o subíndice(s) <code>_{...}</code> opcionales. Si el <i>argumento</i> es de la forma <code>{⟨punto, punto⟩}</code> se aplica <code>\overleftrightharpoonrightarrow</code> en la impresión; si el <i>argumento</i> es de la forma <code>{⟨letra⟩}</code> (mayúscula o minúscula), se imprimirá solo la <code>{⟨letra⟩}</code> sin aplicar <code>\overleftrightharpoonrightarrow</code> .		
Si no se proporciona el <i>argumento opcional</i> <code>[key = val]</code> , <code>\$\sprecta{AB}\$</code> imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; <code>\$\sprecta{m}\$</code> imprimirá <i>m</i> en la salida PDF y NVDA/MathCAT verbalizará «recta m».		

Llaves para `\sprecta`

<code>read-cmd</code>	<code>= {⟨school mathcat⟩}</code>	default: <i>school</i>
Establece la <i>forma</i> en la que NVDA/MathCAT verbalizará el argumento <code>{⟨punto, punto⟩}</code> y el argumento <code>{⟨letra⟩}</code> cuando está en mayúscula. Si se establece en <i>school</i> , NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: <code>\$\sprecta{AB}\$</code> se verbaliza «recta A B»; si se establece en <i>mathcat</i> , NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.		
<code>prefix</code>	<code>= {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT <i>antes</i> de verbalizar el concepto «recta». El valor de esta llave <i>siempre</i> debe pasarse entre llaves <code>{ }</code> .		
<code>concept</code>	<code>= {⟨true false⟩}</code>	default: <i>true</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT del concepto «recta» al ejecutar el comando sin modificar la impresión de este. Si se establece en <i>true</i> , NVDA/MathCAT verbalizará el concepto y el <code>{⟨argumento⟩}</code> ; si se establece en <i>false</i> , NVDA/MathCAT solo verbalizará el <code>{⟨argumento⟩}</code> acorde al valor establecido por la llave <code>read-cmd</code> .		
<code>silent</code>	<code>= {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT del concepto «recta» y del <code>{⟨argumento⟩}</code> pasado al comando sin modificar la impresión de este. Si se establece en <i>true</i> , NVDA/MathCAT NO verbalizará la salida del comando; si se establece en <i>false</i> , NVDA/MathCAT sí verbalizará la salida del comando.		

El comando `\sprayo`

<code>\sprayo</code>	<code>\sprayo{⟨punto, punto⟩}</code>	
	<code>\sprayo[key = val]{⟨punto, punto⟩}</code>	
El comando <code>\sprayo</code> trabaja en <i>modo matemático</i> y recibe el <i>argumento obligatorio</i> <code>{⟨punto, punto⟩}</code> con prima(s) <code>'</code> o subíndice(s) <code>_{...}</code> opcionales. Representa un <i>rayo</i> o <i>semirrecta</i> con origen en el primer punto y dirección hacia el segundo, aplicando <code>\overrightarrow</code> en la impresión.		
Si no se proporciona el <i>argumento opcional</i> <code>[key = val]</code> , <code>\$\sprayo{AB}\$</code> imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».		

Llaves para `\sprayo`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\sprayo{AB}$` se verbaliza «rayo A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<rayo | semirrecta>` } default: `rayo`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$\sprayo[read-sty=rayo]{AB}$`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `$\sprayo[read-sty=semirrecta]{AB}$`, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ' $\overline{AB}$ ' en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside` `\spside{<punto, punto>}`
`\spside[<key = val>]{<punto, punto>}`

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) ' _{...}' opcionales. Representa un *lado, segmento o trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `$\spside{AB}$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\spside{AB}$` se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$\spside[read-sty=lado]{AB}$`, NVDA/MathCAT verbalizará «lado A B»; si se utiliza `$\spside[read-sty=segmento]{AB}$`, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza `$\spside[read-sty=trazo]{AB}$`, NVDA/MathCAT verbalizará «trazo A B».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$\spside[read-arg={hipotenusa}]{AB}$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent = {\langle true | false \rangle}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del `\langle argumento \rangle` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` `\spmside{\langle punto, punto \rangle}`
`\spmside[\langle key = val \rangle]{\langle punto, punto \rangle}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `\langle punto, punto \rangle` con prima(s) ‘*’* o subíndice(s) ‘*_{\dots}*’ opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ‘ $\overline{AB}$ ’; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spmside{AB} \$` imprimirá ‘*AB*’ en la salida PDF y NVDA/MathCAT verbalizará «*medida del lado A B*».

Llaves para `\spmside`

`read-cmd = {\langle school | mathcat \rangle}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `\langle punto, punto \rangle`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\$ \spmside{AB} \$` se verbaliza «*medida del lado A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {\langle lado | segmento | trazo \rangle}`

default: *lado*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «*medida del*» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\$ \spmside[\langle read-sty=lado \rangle]{AB} \$`, NVDA/MathCAT verbalizará «*medida del lado A B*»; si se utiliza `\$ \spmside[\langle read-sty=segmento \rangle]{AB} \$`, NVDA/MathCAT verbalizará «*medida del segmento A B*» y si se utiliza `\$ \spmside[\langle read-sty=trazo \rangle]{AB} \$`, NVDA/MathCAT verbalizará «*medida del trazo A B*».

`prefix = {\langle prefijo \rangle}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*medida del*» seguido por el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’.

`concept = {\langle true | false \rangle}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*» y del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «*medida del*», el concepto y el `\langle argumento \rangle`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `\langle argumento \rangle` acorde al valor establecido por la llave `read-cmd`.

`read-arg = {\langle verbalización NVDA \rangle}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’ y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `\$ \spmside[\langle concept=false, read-arg={medida de la cuerda} \rangle]{AB} \$` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*medida de la cuerda A B*».

`print-m = {\langle true | false \rangle}`

default: *false*

Establece si se *imprime* la notación visual ‘*m*’ antes aplicar `\overline` al `\langle argumento \rangle`. Si se establece en *true*, `\$ \spmside[\langle print-m=true \rangle]{AB} \$` imprimirá ‘*m* $\overline{AB}$ ’ en la salida PDF, si se establece en *false* imprimirá ‘*AB*’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

`silent = {\langle true | false \rangle}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*», del «*concepto*» establecido por la llave `read-sty` y del `\langle argumento \rangle` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<code>\spangle</code>	<code>\spangle{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
	<code>\spangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}` admiten prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa»; `\spangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo uno» y `\spangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

- Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{AOB}` se verbaliza «ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`recto = {⟨true | false⟩}` default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A»; `\spangle[recto=true]{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en `false`, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto «ángulo» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

<code>\spmangle</code>	<code>\spmangle{⟨punto letra griega número⟩}</code>	<code>\spmangle{⟨punto, punto, punto⟩}</code>
	<code>\spmangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spmangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}` admiten prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto>` } o { `<punto, punto, punto>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`recto` = { `<true | false>` }

default: *false*

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá '⊥A' en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{AOB}` imprimirá '⊥AOB' en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en *false*, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`print-m` = { `<true | false>` }

default: *false*

Establece *si se imprime* la notación visual 'm' antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en *true*, `\spmangle[print-m=true]{AOB}` imprimirá 'm∠AOB' en la salida PDF, si se establece en *false* imprimirá '∠AOB' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» seguido por el concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «medida del ángulo» y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del concepto «ángulo» y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

```
\spang \spang{<grados>}
\spang \spang{<grados : minutos>}
\spang \spang{<grados : minutos : segundos>}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<grados>` }, { `<grados : minutos>` } o { `<grados : minutos : segundos>` }. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá '45°30'15"' en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

```
\sparc \sparc{<punto, punto>} \sparc{<punto, punto, punto>}
\sparc \sparc[<key = val>]{<punto, punto>} \sparc[<key = val>]{<punto, punto, punto>}
```

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma { `<punto, punto>` } o { `<punto, punto, punto>` } con prima(s) ' ' o subíndice(s) '_{...}' opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sparc{AB}` imprimirá ' $\widehat{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` } o { `<punto, punto, punto>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{AB}` se verbaliza «arco A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «arco» y el {⟨argumento⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨argumento⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

`\spmarc {⟨punto, punto⟩}`

`\spmarc {⟨punto, punto, punto⟩}`

`\spmarc [⟨key = val⟩] {⟨punto, punto⟩}`

`\spmarc [⟨key = val⟩] {⟨punto, punto, punto⟩}`

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma {⟨punto, punto⟩} o {⟨punto, punto, punto⟩} con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales colocando ‘*m*’ antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `\$ \spmarc{AB} \$` imprimirá ‘*m* $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* ‘ $\widehat{AB}$ ’; `\spmarc` representa una *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {⟨punto, punto⟩} o {⟨punto, punto, punto⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\$ \spmarc{AB} \$` se verbaliza «medida del arco A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «medida del arco» y el {⟨argumento⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨argumento⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

`\spPoly {⟨punto, punto, punto, ...⟩}`

`\spPoly [⟨key = val⟩] {⟨punto, punto, punto, ...⟩}`

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨punto, punto, punto, ...⟩}, una lista de al menos tres {⟨puntos⟩} con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el concepto «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `\$ \spPoly{ABC} \$` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\$ \spPoly{ABCD} \$` imprimirá ‘ $\square ABCD$ ’ y verbalizará «cuadrado A B C D»; `\$ \spPoly{ABCDE} \$` imprimirá ‘ $ABCDE$ ’ y verbalizará «A B C D E».

Llaves para `\spPoly`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto, punto, ...>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{ABC}` se verbaliza «*triángulo A B C*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el { `<argumento>` } o los conceptos «*triángulo*» y «*cuadrado*» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves { `< >` }.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «*triángulo*» y «*cuadrado*» acorde al número de puntos en el { `<argumento>` } al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*triángulo*» o «*cuadrado*» y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`print-sym` = { `<true | false>` }

default: *true*

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión sin modificar la verbalización de este. Si se establece en *true*, `\spPoly[print-sym=true]{ABC}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly[print-sym=true]{ABCD}` imprimirá ' $\square ABCD$ ' en la salida PDF y NVDA/MathCAT verbalizará «*cuadrado A B C*»; si se establece en *false*, solo se imprimirá el { `<argumento>` } en la salida PDF y la verbalización será la misma.

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

<code>\spAfig</code>	<code>\spAfig</code>	<code>\spAfig(<punto, punto, punto, ...>)</code>
<code>\spAfig[<key = val>]</code>	<code>\spAfig[<key = val>](<punto, punto, punto, ...>)</code>	
<code>\spAfig(<1 2 3 ... n>)</code>	<code>\spAfig(F P B L T)</code>	
<code>\spAfig[<key = val>](<1 2 3 ... n>)</code>	<code>\spAfig[<key = val>](F P B L T)</code>	

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma { `<1 | 2 | 3 | ... | n>` }, { `<punto, punto, punto, ...>` } o { `<(F | P | B | L | T)>` } aplicando `\mathcal{A}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al { `<argumento>` } y la llave `concept`.

Si se proporciona el *argumento* { `<1 | 2 | 3 | ... | n>` }, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área uno*»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área enes*».

Si se proporciona el *argumento* { `<punto, punto, punto, ...>` }, `\spAfig(ABC)` imprimirá ' $\mathcal{A}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área del triángulo A B C*»; `\spAfig(ABCD)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área del cuadrado A B C D*»;

Si se proporciona el *argumento* { `<(F | P | B | L | T)>` }, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área de la figura*»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área de la figura uno*».

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el *argumento opcional delimitado por paréntesis*, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área*».

- El *argumento* { `<punto, punto, punto, ...>` } acepta prima(s) '`'`' o subíndice(s) '`_{...}`'. El *argumento* { `<(F | P | B | L | T)>` } solo acepta subíndices '`_{...}`' y tiene una impresión y verbalización fija para cada letra: F «*figura*», P «*polígono*», B «*basal*», L «*lateral*» y T «*total*».

Llaves para `\spAfig`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(ABC)` se verbaliza «*área del triángulo A B C*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*área del*» y el concepto «*nombre de la figura*» acorde al { `<argumento>` } pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves { `< >` }.

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del» y el concepto «nombre de la figura» acorde al (`⟨argumento⟩`) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «área del», el concepto «nombre de la figura» y el (`⟨argumento⟩`); si se establece en `false`, NVDA/MathCAT solo verbalizará el (`⟨argumento⟩`) acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨nombre⟩}` default: no usado

Sobrescribe el término «área del» y el concepto «nombre de la figura» cuando se utiliza el argumento (`⟨punto, punto, punto, ...⟩`) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `$\spAfig[read-arg={trapezio}](ABCD)$` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapecio A B C D».

Esta llave no se puede utilizar con los argumentos (`⟨F | P | B | L | T⟩`) y (`⟨1 | 2 | 3 | ... | n⟩`).

`concept* = {⟨valor prohibido⟩}` default: no usado

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al (`⟨argumento⟩`) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «área» y el (`⟨argumento⟩`) acorde al valor establecido por la llave `read-cmd`.

`read-sub = {⟨true | false⟩}` default: false

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos (`⟨1 | 2 | 3 | ... | n⟩`) y (`⟨F | P | B | L | T⟩`) para NVDA/MathCAT. Si se utiliza (`⟨1 | 2 | 3 | ... | n⟩`) y se establece en `true`, `$\spAfig[read-sub=true](1)$` imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «área sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área uno».

Si se utiliza (`⟨F | P | B | L | T⟩`) y se establece en `true`, `$\spAfig[read-sub=true](F_{1})$` imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área de la figura uno».

`print-sym = {⟨auto | triangle | square | none⟩}` default: auto

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos (`⟨punto, punto, punto, ...⟩`) y (`⟨1 | 2 | 3 | ... | n⟩`) pasado al comando sin modificar la verbalización de este. Si se utiliza (`⟨punto, punto, punto, ...⟩`) se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: `$\spAfig[print-sym=auto](ABC)$` y `$\spAfig[print-sym=none](ABC)$` imprimirán ' $\mathcal{A}_{\triangle ABC}$ ' y ' $\mathcal{A}_{ABC}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «área del triángulo A B C».

Si se utiliza (`⟨1 | 2 | 3 | ... | n⟩`) los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: `$\spAfig[print-sym=triangle](1)$` imprimirá ' $\mathcal{A}_{\triangle_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del triángulo uno»; `$\spAfig[print-sym=square](1)$` imprimirá ' $\mathcal{A}_{\square_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del cuadrado uno».

Esta llave no se puede utilizar con el argumento (`⟨F | P | B | L | T⟩`).

`silent = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del», el concepto «nombre de la figura» y el (`⟨argumento⟩`) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(⟨punto, punto, punto, ...⟩)</code>
<code>\spPfig[key = val]</code>	<code>\spPfig[key = val]</code>	<code>\spPfig[key = val](⟨punto, punto, punto, ...⟩)</code>
<code>\spPfig(⟨1 2 3 ... n⟩)</code>	<code>\spPfig(⟨F P B L T⟩)</code>	<code>\spPfig(⟨F P B L T⟩)</code>
<code>\spPfig[key = val](⟨1 2 3 ... n⟩)</code>	<code>\spPfig[key = val](⟨F P B L T⟩)</code>	

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`⟨1 | 2 | 3 | ... | n⟩`), (`⟨punto, punto, punto, ...⟩`) o (`⟨F | P | B | L | T⟩`) aplicando `\mathcal{P}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al (`⟨argumento⟩`) y la llave `concept`.

Si se proporciona el *argumento* (`⟨1 | 2 | 3 | ... | n⟩`), `$\spPfig(1)$` imprimirá ' $\mathcal{P}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro uno»; `$\spPfig(n)$` imprimirá ' $\mathcal{P}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro ene».

Si se proporciona el *argumento* (`⟨punto, punto, punto, ...⟩`), `$\spPfig(ABC)$` imprimirá ' $\mathcal{P}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo A B C»; `$\spPfig(ABCD)$` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado A B C D»;

Si se proporciona el *argumento* (`⟨F | P | B | L | T⟩`), `$\spPfig(F)$` imprimirá ' $\mathcal{P}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura»; `$\spPfig(F_{1})$` imprimirá ' $\mathcal{P}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura uno».

Si no se proporciona el *argumento opcional* [*(key = val)*] ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spPfig} \$$ imprimirá ‘P’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro».

- El *argumento* (*(punto, punto, punto, ...)*) acepta prima(s) ‘_’ o subíndice(s) ‘_{...}’. El *argumento* (*(F|P|B|L|T)*) solo acepta subíndices ‘_{...}’ y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para $\backslash \text{spPfig}$

$\text{read-cmd} = \{ \langle \text{school} \mid \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spPfig} (\text{ABC}) \$$ se verbaliza «perímetro del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

$\text{prefix} = \{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro del» y el concepto «nombre de la figura» acorde al (*(argumento)*) pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

$\text{concept} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del» y el concepto «nombre de la figura» acorde al (*(argumento)*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «perímetro del», el concepto «nombre de la figura» y el (*(argumento)*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*(argumento)*) acorde al valor establecido por la llave *read-cmd*.

$\text{read-arg} = \{ \langle \text{nombre} \rangle \}$ default: *no usado*

Sobrescribe el término «perímetro del» y el concepto «nombre de la figura» cuando se utiliza el argumento (*(punto, punto, punto, ...)*) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave *concept*. Por ejemplo: $\$ \backslash \text{spPfig} [\text{read-arg} = \{ \text{trapezio} \}] (\text{ABCD}) \$$ imprimirá ‘ $\mathcal{P}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos (*(F|P|B|L|T)*) y (*(1|2|3|...|n)*).

$\text{concept}^* = \langle \text{valor prohibido} \rangle$ default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al (*(argumento)*) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «perímetro» y el (*(argumento)*) acorde al valor establecido por la llave *read-cmd*.

$\text{read-sub} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos (*(1|2|3|...|n)*) y (*(F|P|B|L|T)*) para NVDA/MathCAT. Si se utiliza (*(1|2|3|...|n)*) y se establece en *true*, $\$ \backslash \text{spPfig} [\text{read-sub} = \text{true}] (1) \$$ imprimirá ‘ $\mathcal{P}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «perímetro uno».

Si se utiliza (*(F|P|B|L|T)*) y se establece en *true*, $\$ \backslash \text{spPfig} [\text{read-sub} = \text{true}] (F_{\{1\}}) \$$ imprimirá ‘ $\mathcal{P}_{F_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «perímetro de la figura uno».

$\text{print-sym} = \{ \langle \text{auto} \mid \text{triangle} \mid \text{square} \mid \text{none} \rangle \}$ default: *auto*

Establece si se *usará* $\backslash \text{triangle}$ o $\backslash \text{mdlgwhtsquare}$ en la impresión para los argumentos (*(punto, punto, punto, ...)*) y (*(1|2|3|...|n)*) pasado al comando sin modificar la verbalización de este. Si se utiliza (*(punto, punto, punto, ...)*) se infiere el *símbolo* acorde al número de puntos; si establece en *auto* se imprime y si se establece en *none* se omite: $\$ \backslash \text{spPfig} [\text{print-sym} = \text{auto}] (\text{ABC}) \$$ y $\$ \backslash \text{spPfig} [\text{print-sym} = \text{none}] (\text{ABC}) \$$ imprimirán ‘ $\mathcal{P}_{\triangle ABC}$ ’ y ‘ $\mathcal{P}_{ABC}$ ’ en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «perímetro del triángulo A B C».

Si se utiliza (*(1|2|3|...|n)*) los valores *auto* y *none* quedan sin efecto; si se establece en *triangle* o *square* anida el argumento bajo el *símbolo*: $\$ \backslash \text{spPfig} [\text{print-sym} = \text{triangle}] (1) \$$ imprimirá ‘ $\mathcal{P}_{\triangle 1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo uno»; $\$ \backslash \text{spPfig} [\text{print-sym} = \text{square}] (1) \$$ imprimirá ‘ $\mathcal{P}_{\square 1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado uno».

- Esta llave no se puede utilizar con el argumento (*(F|P|B|L|T)*).

$\text{silent} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del», el concepto «nombre de la figura» y el (*(argumento)*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprad` y `\spdiam`

<code>\sprad</code>	<code>\sprad</code>	<code>\sprad[⟨key = val⟩]</code>
<code>\spdiam</code>	<code>\spdiam</code>	<code>\spdiam[⟨key = val⟩]</code>

Los comandos `\sprad` y `\spdiam` trabajan en *modo matemático* y reciben el *argumento opcional* [`⟨key = val⟩`] mediante el cual se establece la «*forma*» en la que se imprimirán en la salida PDF las letras ‘*r*’ y ‘*d*’ y como se verbalizarán en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`], `\sprad` imprimirá ‘*r*’ en la salida PDF y NVDA/MathCAT verbalizará «*radio*»; `\spdiam` imprimirá ‘*d*’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

Llaves para `\sprad` y `\spdiam`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «*radio*» o «*diámetro*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\sprad` y `\spdiam` se verbalizan como «*radio*» y «*diámetro*» en NVDA/MathCAT; si se establece en *false*, se verbalizarán las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ cuando `concept=false` y `upper-case=true`. Si se establece en *school*, NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ sin el prefijo «mayúscula»: si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`upper-case = {⟨true | false⟩}` default: *false*

Establece si se *imprime* usando mayúsculas. Si se establece en *true*, `\sprad[upper-case=true]` y `\spdiam[upper-case=true]` imprimirán ‘*R*’ y ‘*D*’ en la salida PDF; si se establece en *false* se imprimirá usando minúsculas.

🔊 La verbalización al establecer esta llave es controlada por la llave `read-cmd`.

`print-sym = {⟨true | false⟩}` default: *false*

Establece si se *usará* `\diameter` antes de la letra ‘*d*’ o ‘*D*’ en la impresión del comando `\spdiam`. Si se establece en *true*, `\spdiam[print-sym=true]` imprimirá ‘ $\varnothing d$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

🔊 La llave `print-sym` no tiene ningún efecto en `\sprad`.

`sub-index = {⟨subíndice⟩}` default: *no usado*

Establece un *subíndice* sobre las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\sprad[sub-index={1}]` imprimirá ‘ r_1 ’ en la salida PDF y NVDA/MathCAT verbalizará «*radio uno*».

`read-sub = {⟨true | false⟩}` default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\sprad[sub-index={1}, read-sub=true]` se verbalizará «*radio sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*radio uno*».

`read-arg = {⟨verbalización NVDA⟩}` default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\sprad[read-arg={medida exacta}]` imprimirá ‘*r*’ en la salida PDF y NVDA/MathCAT verbalizará «*radio medida exacta*»; `\sprad[concept=false, read-arg={medida exacta}]` imprimirá ‘*r*’ y NVDA/MathCAT verbalizará «*medida exacta*».

🔊 Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’ al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spcirc`

<code>\spcirc</code>	<code>\spcirc</code>	<code>\spcirc[⟨key = val⟩]</code>
<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro⟩)</code>
<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro, radio⟩)</code>

El comando `\spcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(⟨centro⟩)$ o $(⟨centro, radio⟩)$, donde $⟨radio⟩$ es opcional y debe pasarse junto al argumento $(⟨centro⟩)$ separado por una coma ‘,’ obligatoria.

La forma en la que se procesa el $⟨radio⟩$ junto a su impresión y verbalización se establece de manera interna: Si es un $⟨número⟩$ se usa `\spnum`; si es una $⟨medida⟩$ se usa `\spunit` y si es un segmento $⟨punto punto⟩$ se usa `\spside` o `\spmside` acorde al valor establecido por la llave `radio-sty`. En cualquier otro caso no se procesa de manera interna.

Si no se proporciona el *argumento opcional* $[⟨key = val⟩]$ ni el *argumento opcional delimitado por paréntesis*, `\spcirc` imprimirá ‘ $\odot$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia».

Si se proporciona el *argumento* $(⟨centro⟩)$, `\spcirc(0)` imprimirá ‘ $\odot(O)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O».

Si se proporciona el *argumento* $(⟨centro, radio⟩)$, `\spcirc(0,3)` imprimirá ‘ $\odot(O,3)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio tres».

- El argumento $(⟨centro⟩)$ y el argumento $⟨radio⟩$ cuando tiene la forma $⟨punto punto⟩$ aceptan prima ‘ $\prime$ ’ o subíndice ‘ $_{\dots}$ ’.

Llaves para `\spcirc`

El comando `\spcirc` posee las *meta-keys* `spside` pasada al comando `\spside` (v. pág. 20) y `spmside` pasada al comando `\spmside` (v. pág. 21).

`read-cmd = {⟨school | mathcat⟩}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $(⟨centro⟩)$. Si se establece en `school`, NVDA/MathCAT verbaliza el argumento $(⟨centro⟩)$ sin el prefijo «mayúscula»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

- Esta llave tiene efecto sobre el argumento $⟨radio⟩$ cuando tiene la forma $⟨punto punto⟩$.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`print-sym = {⟨usual | odot⟩}` default: odot

Establece si se usará `\bigodot` o la letra ‘C’ en la impresión del comando sin modificar la verbalización de este. Si se establece en `usual`, `\spcirc[print-sym=usual](0)` imprimirá ‘C(O)’ en la salida PDF; si se establece en `odot`, `\spcirc[print-sym=odot](0)` imprimirá ‘ $\odot(O)$ ’ en la salida PDF.

`read-sty = {⟨circun | circle⟩}` default: circun

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se establece en `circun`, `\spcirc[read-sty=circun](0)` se verbalizará «circunferencia de centro en O» en NVDA/MathCAT; si se establece en `circle`, `\spcirc[read-sty=circle](0)` se verbalizará «círculo de centro en O» en NVDA/MathCAT.

`diameter = {⟨true | false⟩}` default: false

Establece *como* se verbaliza el argumento $⟨radio⟩$ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spcirc[diameter=true](0,3)` se verbalizará «circunferencia de centro en O y diámetro tres» en NVDA/MathCAT; si se establece en `false`, `\spcirc[diameter=false](0,3)` se verbalizará «circunferencia de centro en O y radio tres» en NVDA/MathCAT.

`radio-sty = {⟨side | mside⟩}` default: mside

Establece el *comando* que procesa internamente el argumento $⟨radio⟩$ cuando tiene la forma $⟨punto punto⟩$. Si se establece en `side` se usará `\spside{⟨punto punto⟩}`, `\spcirc[radio-sty=side](O, AB)` imprimirá ‘ $\odot(O, AB)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio lado A B»; si se establece en `mside` se usará `\spmside{⟨punto punto⟩}`, `\spcirc[radio-sty=mside](O, AB)` imprimirá ‘ $\odot(O, AB)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio medida del lado A B».

- Las *meta-keys* `spside` y `spmside` permiten modificar la impresión y verbalización del argumento $⟨radio⟩$ cuando tiene la forma $⟨punto punto⟩$.

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará

el «concepto» y el argumento ($\langle centro \rangle$) o ($\langle centro, radio \rangle$); si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento ($\langle centro \rangle$) o ($\langle centro, radio \rangle$) acorde al valor establecido por la llave `read-cmd`.

`sub-index = { \langle subíndice \rangle }` default: no usado

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\$ \spAcirc[sub-index={1},radio-sty=mside](O,AB) \$` imprimirá $\odot_1(O,AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia uno de centro en O y radio medida del lado A B».

`read-sub = { \langle true | false \rangle }` default: false

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\$ \spAcirc[sub-index={1},read-sub=true](O) \$` se verbalizará «circunferencia sub uno de centro en O» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «circunferencia uno de centro en O».

`read-arg = { \langle verbalización NVDA \rangle }` default: no usado

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\$ \spAcirc[read-arg={disco}](O) \$` imprimirá $\odot(O)$ en la salida PDF y NVDA/MathCAT verbalizará «disco de centro en O».

`silent = { \langle true | false \rangle }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty`, del argumento ($\langle centro \rangle$) o ($\langle centro, radio \rangle$) pasados al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAcirc`

<code>\spAcirc</code>	<code>\spAcirc</code>	<code>\spAcirc[\langle key = val \rangle]</code>
<code>\spAcirc(\langle centro \rangle)</code>	<code>\spAcirc[\langle key = val \rangle](\langle centro \rangle)</code>	
<code>\spAcirc(\langle 1 2 3 ... n \rangle)</code>	<code>\spAcirc[\langle key = val \rangle](\langle 1 2 3 ... n \rangle)</code>	

El comando `\spAcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma ($\langle centro \rangle$) o ($\langle 1 | 2 | 3 | ... | n \rangle$), aplicando `\mathcal{A}` a la impresión.

Si no se proporciona el *argumento opcional* [`\langle key = val \rangle`] ni el *argumento opcional delimitado por paréntesis*, `\$ \spAcirc \$` imprimirá 'A' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia».

Si se proporciona el *argumento* ($\langle centro \rangle$), `\$ \spAcirc(O) \$` imprimirá $\mathcal{A}_{\odot(O)}$ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia con centro en O».

Si se proporciona el *argumento* ($\langle 1 | 2 | 3 | ... | n \rangle$), `\$ \spAcirc(1) \$` imprimirá $\mathcal{A}_{\odot_1}$ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno».

- El *argumento* ($\langle centro \rangle$) acepta prima '' o subíndice '`_{\dots}`' y no acepta un $\langle radio \rangle$. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spAcirc`

`read-cmd = { \langle school | mathcat \rangle }` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento ($\langle centro \rangle$). Si se establece en `school`, NVDA/MathCAT verbaliza el argumento ($\langle centro \rangle$) sin el prefijo «mayúscula»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { \langle prefijo \rangle }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty = { \langle circun | circle \rangle }` default: circun

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «área de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en `circun`, `\$ \spAcirc[read-sty=circun](O) \$` se verbalizará «área de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en `circle`, `\$ \spAcirc[read-sty=circle](O) \$` se verbalizará «área del círculo con centro en O» en NVDA/MathCAT.

`concept = { \langle true | false \rangle }` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término, el concepto y el argumento ($\langle centro \rangle$) o ($\langle 1 | 2 | 3 | ... | n \rangle$); si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento ($\langle centro \rangle$) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`\langle centro \rangle`) o con el argumento (`\langle 1 | 2 | 3 | \dots | n \rangle`) es equivalente a usar `silent=true`.

`read-arg = { \langle verbalización NVDA \rangle }`

default: no usado

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spAcirc[read-arg={esfera}](0)` se verbalizará «área de la esfera con centro en O» en NVDA/MathCAT; `\spAcirc[concept=false,read-arg={esfera}](0)` se verbalizará «esfera con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`\langle 1 | 2 | 3 | \dots | n \rangle`) es equivalente a usar `silent=true`.

`print-sym = { \langle usual | odot \rangle }`

default: odot

Establece si se usará `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en `usual`, `\spAcirc[print-sym=usual](0)` imprimirá ' $\mathcal{A}_{C(O)}$ ' en la salida PDF; si se establece en `odot`, `\spAcirc[print-sym=odot](0)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF.

`sub-index = { \langle subíndice \rangle }`

default: no usado

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`\langle centro \rangle`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAcirc[sub-index={1}](0)` imprimirá ' $\mathcal{A}_{\odot_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno con centro en O».

`read-sub = { \langle true | false \rangle }`

default: false

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`\langle 1 | 2 | 3 | \dots | n \rangle`) al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spAcirc[read-sub=true](1)` se verbalizará «área de la circunferencia sub uno» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «área de la circunferencia uno».

`silent = { \langle true | false \rangle }`

default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...», del concepto establecido por la llave `read-sty` y del argumento (`\langle centro \rangle`) o (`\langle 1 | 2 | 3 | \dots | n \rangle`) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPcirc`

`\spPcirc`

`\spPcirc(\langle centro \rangle)`

`\spPcirc(\langle 1 | 2 | 3 | \dots | n \rangle)`

`\spPcirc[\langle key = val \rangle]`

`\spPcirc[\langle key = val \rangle](\langle centro \rangle)`

`\spPcirc[\langle key = val \rangle](\langle 1 | 2 | 3 | \dots | n \rangle)`

El comando `\spPcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`\langle centro \rangle`) o (`\langle 1 | 2 | 3 | \dots | n \rangle`), aplicando `\mathcal{P}` a la impresión.

Si no se proporciona el *argumento opcional* [`\langle key = val \rangle`] ni el *argumento opcional delimitado por paréntesis*, `\spPcirc` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia».

Si se proporciona el *argumento* (`\langle centro \rangle`), `\spPcirc(0)` imprimirá ' $\mathcal{P}_{\odot(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia con centro en O».

Si se proporciona el *argumento* (`\langle 1 | 2 | 3 | \dots | n \rangle`), `\spPcirc(1)` imprimirá ' $\mathcal{P}_{\odot_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

- El *argumento* (`\langle centro \rangle`) acepta prima '' o subíndice '_{...}' y no acepta un *radio*. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spPcirc`

`read-cmd = { \langle school | mathcat \rangle }`

default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (`\langle centro \rangle`). Si se establece en `school`, NVDA/MathCAT verbaliza el argumento (`\langle centro \rangle`) sin el prefijo «mayúscula»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { \langle prefijo \rangle }`

default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro de ...» seguido por el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty = { \langle circun | circle \rangle }`

default: circun

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «perímetro de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en `circun`, `\spPcirc[read-sty=circun](0)` se verbalizará «perímetro de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en `circle`, `\spPcirc[read-sty=circle](0)` se verbalizará «perímetro del círculo con centro en O» en NVDA/MathCAT.

`concept = { \langle true | false \rangle }`

default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro de ...*» y del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`(\centro)`) o (`(1 | 2 | 3 | ... | n)`); si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento (`(\centro)`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`(\centro)`) o con el argumento (`(1 | 2 | 3 | ... | n)`) es equivalente a usar `silent=true`.

`read-arg = {(\verbalización NVDA)}` default: no usado

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spPcirc[read-sty=circle,read-arg={disco}](O)` se verbalizará «*perímetro del disco con centro en O*» en NVDA/MathCAT; `\spPcirc[concept=false,read-arg={disco}]` se verbalizará «*disco con centro en O*» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`(1 | 2 | 3 | ... | n)`) es equivalente a usar `silent=true`.

`print-sym = {(\usual | odot)}` default: odot

Establece si se usará `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en `usual`, `\spPcirc[print-sym=usual](O)` imprimirá ' $\mathcal{P}_{C(O)}$ ' en la salida PDF; si se establece en `odot`, `\spPcirc[print-sym=odot](O)` imprimirá ' $\mathcal{P}_{O(O)}$ ' en la salida PDF.

`sub-index = {(\subíndice)}` default: no usado

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`(\centro)`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spPcirc[sub-index={1}](O)` imprimirá ' $\mathcal{P}_{O_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*perímetro de la circunferencia uno con centro en O*».

`read-sub = {(\true | false)}` default: false

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`(1 | 2 | 3 | ... | n)`) al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spPcirc[read-sub=true](1)` se verbalizará «*perímetro de la circunferencia sub uno*» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «*perímetro de la circunferencia uno*».

`silent = {(\true | false)}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro de ...*», del concepto establecido por la llave `read-sty` y del argumento (`(\centro)`) o (`(1 | 2 | 3 | ... | n)`) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT si verbalizará la salida del comando.

El comando `\spVol`

`\spVol` `\spVol` `\spVol[\key = val]`

El comando `\spVol` trabaja en *modo matemático* y recibe el *argumento opcional* `[\key = val]` mediante el cual se establece la «*forma*» en la que se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[\key = val]`, `\spVol` imprimirá 'V' en la salida PDF y NVDA/MathCAT verbalizará «*volumen*».

Llaves para `\spVol`

`concept = {(\true | false)}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*volumen*» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spVol` se verbaliza como «*volumen*» en NVDA/MathCAT; si se establece en `false`, se verbalizará la letra 'V' acorde al valor establecido por la llave `read-cmd`.

`read-cmd = {(\school | mathcat)}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará la letra 'V' cuando `concept=false`. Si se establece en `school`, NVDA/MathCAT verbalizará únicamente la letra 'V'; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas sobre 'V'.

`sub-index = {(\subíndice)}` default: no usado

Establece un *subíndice* sobre 'V'. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spVol[sub-index={1}]` imprimirá ' $\mathcal{V}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «*volumen uno*».

`read-sub = {(\true | false)}` default: false

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spVol[sub-index={1},read-sub=true]` se verbalizará «*volumen sub uno*» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «*volumen uno*».

`read-arg = {(\verbalización NVDA)}` default: no usado

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spVol[read-arg={medida exacta}]\$` imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen medida exacta*»; `\spVol[concept=false,read-arg={medida exacta}]\$` imprimirá ‘ $\mathcal{V}$ ’ y NVDA/MathCAT verbalizará «*medida exacta*».

🔊 Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = \{ \langle true | false \rangle \}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de ‘ $\mathcal{V}$ ’ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-like) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-10-01 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAcirc	30
\spAfig	25
\spPcirc	31
\spPfig	26
\spPoint	18
\spPoly	24
\spVol	32
\spangle	22
\spang	23
\sparc	23
\spcirc	29
\spcoord	17
\spdate	11
\spdiam	28
\spdiv	13
\spdsym	12
\spmQ	17
\spmangle	22
\spmarc	24
\spmcd	15
\spmcm	16
\spmoney	6
\spmside	21
\spmsym	13
\spmul	13
\spnD	14
\spnM	14
\spnZ	16
\spnewread	11
\spnewunit	6
\spnum	3
\sprad	28
\sprayo	19
\spread	11
\sprecta	19
\spshort	8
\spside	20
\spsiglo	10
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
\spvec	18
K	
Keys for \spAcirc provide by spintent :	
concept	30
prefix	30
print-sym	31
read-arg	31
read-cmd	30
read-sty	30
read-sub	31
silent	31
sub-index	31
Keys for \spAfig provide by spintent :	
concept*	26
concept	26
prefix	25
print-sym	26
read-arg	26
read-cmd	25
read-sub	26
silent	26
Keys for \spangle provide by spintent :	
concept	22
prefix	22
read-cmd	22
recto	22
silent	22
Keys for \sparc provide by spintent :	
concept	24
prefix	23
read-cmd	23
silent	24
Keys for \spcirc provide by spintent :	
concept	29
diameter	29
prefix	29
print-sym	29
radio-sty	29
read-arg	30
read-cmd	29
read-sty	29
read-sub	30
silent	30
spmside	29
spside	29
sub-index	30
Keys for \spcoord provide by spintent :	
silent	18
Keys for \spdiam provide by spintent :	
concept	28
print-sym	28
read-arg	28
read-cmd	28
read-sub	28
silent	28
sub-index	28
upper-case	28
Keys for \spdiv provide by spintent :	
cifras-sep	13
notation-sym	13
read-parens	13
read-period-sep	13
read-sign	13
read-sym	13
set-sym	13
wrap-parens	13
Keys for \spdsym provide by spintent :	
prefix	12
read-sym	12
set-sym	12
suffix	12

Keys for \spmangle provide by spintent :		silent	14
concept	23	Keys for \spnM provide by spintent :	
prefix	23	concept	15
print-m	23	prefix	15
read-cmd	23	result-num	15
recto	23	result	15
silent	23	silent	15
Keys for \spmarc provide by spintent :		Keys for \spnum provide by spintent :	
concept	24	cifras-sep	3
prefix	24	notation-sym	4
read-cmd	24	read-period-sep	3
silent	24	read-sign	3
Keys for \spmcd provide by spintent :		Keys for \spnZ provide by spintent :	
concept	15	cifras-sep	16
prefix	15	read-parens	16
read-cmd	15	read-sign	16
result	15	wrap-parens	16
silent	15	Keys for \spPcirc provide by spintent :	
upper-case	15	concept	31
Keys for \spmcm provide by spintent :		prefix	31
concept	16	print-sym	32
prefix	16	read-arg	32
read-cmd	16	read-cmd	31
result	16	read-sty	31
silent	16	read-sub	32
upper-case	16	silent	32
Keys for \spmoney provide by spintent :		sub-index	32
cifras-sep	7	Keys for \spPfig provide by spintent :	
currency	7	concept*	27
money-math	7	concept	27
sub-units	7	prefix	27
Keys for \spmQ provide by spintent :		print-sym	27
join-word	17	read-arg	27
print-dfrac	17	read-cmd	27
read-parens	17	read-sub	27
use-spN	17	silent	27
wrap-parens	17	Keys for \spPoint provide by spintent :	
Keys for \spmside provide by spintent :		concept	18
concept	21	prefix	18
prefix	21	read-cmd	18
print-m	21	silent	18
read-arg	21	Keys for \spPoly provide by spintent :	
read-cmd	21	concept	25
read-sty	21	prefix	25
silent	21	print-sym	25
Keys for \spmsym provide by spintent :		read-cmd	25
not-print	13	silent	25
prefix	13	Keys for \sprad provide by spintent :	
read-sym	13	concept	28
set-sym	13	read-arg	28
suffix	13	read-cmd	28
Keys for \spmul provide by spintent :		read-sub	28
cifras-sep	14	silent	28
notation-sym	14	sub-index	28
read-parens	14	upper-case	28
read-period-sep	14	Keys for \sprayo provide by spintent :	
read-sign	14	concept	20
read-sym	14	prefix	20
set-sym	14	read-cmd	20
wrap-parens	14	read-sty	20
Keys for \spnD provide by spintent :		silent	20
concept	14	Keys for \spread provide by spintent :	
prefix	14	read-arg	12
result-num	14	silent	12
result	14		

Keys for \sprecta provide by **spintent**:

concept

19

prefix

19

read-cmd

19

silent

19

Keys for \spshort provide by **spintent**:

read-arg

8

small-caps

8

Keys for \spside provide by **spintent**:

concept

20

prefix

20

read-arg

20

read-cmd

20

read-sty

20

silent

21

Keys for \spsiglo provide by **spintent**:

print-era

11

Keys for \spunit provide by **spintent**:

cifras-sep

4

notation-sym

4

print-cdot

4

read-cdot

4

read-period-sep

4

read-slash

4

Keys for \spusep provide by **spintent**:

silent

12

size

12

Keys for \spvec provide by **spintent**:

autofit

19

concept

19

prefix

19

read-cmd

19

silent

19

Keys for \spVol provide by **spintent**:

concept

32

read-arg

32

read-cmd

32

read-sub

32

silent

33

sub-index

32

M

Lua module provide by **spintent**:

spintent.lua

6–11

P

Packages :

babel-spanish

2

babel

2, 8

documentmetadata-support

1

enumext

1

expl3

3

fontspec

2

fourier-otf

2

hyperref

2

l3keys

3

latex-lab-mathintent

1

libertinus-otf

2

luamml

1

scontents

1

sourcecodepro

2

tagpdf

1, 8, 10

unicode-math

1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-10-01} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_LTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34     \IfPackageLoadedF { unicode-math }
35     {
36       \IfPackageLoadedF { lua-unicode-math }
37       {
38         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
39         \RequirePackage{unicode-math}

```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

```

\str_if_eq:nVTF \str_if_eq:nVF
\str_uppercase:V \str_lowercase:V
\seq_set_from_clist:Ne
\cs_generate_variant:Nn \str_if_eq:nnTF { V }
\cs_generate_variant:Nn \str_if_eq:nnF { V }
\cs_generate_variant:Nn \str_uppercase:n { V }
\cs_generate_variant:Nn \str_lowercase:n { V }
\cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal`, `\rightanglesqr` and `\attribute`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`), the string variable `\l__spintent_luafun_right_angle_sqr:nn` used by the function `\__spintent_luafun_right_angle_sqr:nn` to copy `\rightanglesqr` alongside the string variable `\l__spintent_luaset_scope_attr_str` used by `\attribute` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str
69 \str_new:N \l__spintent_luaset_scope_attr_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str`, `\l__spintent_luaset_sym_fam_ok_str`, and `\l__spintent_luaset_scope_attr_str`.)

11.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

70 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for `tagged PDF` — `math-AF` mode does not want either ghost present.

```

71 \bool_if:NTF \l__spintent_debug_ghost_bool
72 { \__spintent_luafun_set_debug_ghost:n { true } }
73 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

74 \int_new:N \l__spintent_scope_attr_int
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_luaset_scope_attr_str }

```

(End of definition for `\__spintent_scope_attr_int`.)

`\__spintent_luafun_spunit_lookup_alias:n` Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_spmoney_lookup_data:n` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean_split_and_set:n` (used by `\spnum`).

```

76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:n` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

`mathcal-fallback`
`right-angle-sqr-fallback`

First, we'll define two error messages in case our fallback fails.

```

80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=(#2)`. A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0" "0" "2063
127 }
```

🔗 Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

11.9 Normalize argument and affixes

We normalize the `{⟨argument⟩}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     \__spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn` The function `\__spintent_sanitize_affix:Nn` checks if the `{⟨argument⟩}` passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘`␣`’ to hyphens ‘`-`’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

136 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Ne #1 {#2}
142     \tl_trim_spaces:N #1
143     \tl_replace_all:Nnn #1 { ~ } { - }
144     \__spintent_collapse_hyphens:N #1
145     \tl_put_left:Nn #1 { \q_mark }
146     \tl_put_right:Nn #1 { \q_stop }
147     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
148     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
149     \tl_remove_all:Nn #1 { \q_mark }
150     \tl_remove_all:Nn #1 { \q_stop }
151   }
152 }
153 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure `⟨keys⟩` “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\l__spintent_setspintent_input_arg_cmd_tl` The variable `\l__spintent_setspintent_input_arg_cmd_tl` will store the command or command name passed to the “first” `{⟨argument⟩}` ‘#1’ of the command `\setspintent`.

```
154 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the `{⟨argument⟩}` ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

155 \msg_new:nnnn { spintent } { unknown-command-setup }
156 {
157   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
158 }
159 {
160   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
161   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
162 }
```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

__spintent_setspintent_set_keys:nn
__spintent_setspintent_set_keys:en

```

The function `__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in ‘#1’ and set the `⟨keys⟩` passed as `{⟨argument⟩}` in ‘#2’ to the correct path; otherwise, it will return an error.

```

163 \cs_new_protected:Nn __spintent_setspintent_set_keys:nn
164 {
165   \cs_if_exist:cTF { #1 }
166   {
167     \keys_set:nn { spintent / #1 } { #2 }
168   }
169   {
170     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
171   }
172 }
173 \cs_generate_variant:Nn __spintent_setspintent_set_keys:nn { en }

```

(End of definition for `__spintent_setspintent_set_keys:nn`.)

```

__spintent_setspintent_parse_args:nn
__spintent_setspintent_parse_args:Vn

```

The function `__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `__spintent_setspintent_set_keys:nn`.

```

174 \cs_new_protected:Nn __spintent_setspintent_parse_args:nn
175 {
176   \tl_if_single:nTF { #1 }
177   {
178     \token_if_cs:NTF #1
179     {
180       __spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
181     }
182     {
183       __spintent_setspintent_set_keys:nn { #1 } { #2 }
184     }
185   }
186   {
187     __spintent_setspintent_set_keys:nn { #1 } { #2 }
188   }
189 }
190 \cs_generate_variant:Nn __spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `__spintent_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```

191 \NewDocumentCommand \setspintent { m m }
192 {
193   \tl_if_blank:nTF { #1 }
194   {
195     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
196   }
197   {
198     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
199     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
200     __spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
201   }
202 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```

\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown

```

We define the variable `\l__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by `⟨keys⟩` that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the `⟨key⟩` `unknown` in commands that use `[⟨key = val⟩]`.

```

203 \str_new:N \l__spintent_command_name_str
204 \msg_new:nnn { spintent } { unknown-choice }
205 {
206   The~value~'~#3'~for~'~#1'~key~is~invalid~use~('~#2').
207 }
208 \msg_new:nnnn { spintent } { cmd-key-unknown }
209 {
210   The~key~'~#1'~is~unknown~by~command~

```

```

211     \c_backslash_str \l__spintent_command_name_str \c_space_tl
212     and~is~being~ignored~\msg_line_context:.
213 }
214 {
215     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
216     does~not~have~a~key~called~'#1'.\\
217     Check~that~you~have~spelled~the~key~name~correctly.
218 }
219 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
220 {
221     The~key~'#1=#2'~is~unknown~by~command~
222     \c_backslash_str \l__spintent_command_name_str \c_space_tl
223     and~is~being~ignored~\msg_line_context:.
224 }
225 {
226     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
227     does~not~have~a~key~called~'#1'.\\
228     Check~that~you~have~spelled~the~key~name~correctly.
229 }

```

(End of definition for `\l__spintent_command_name_str` and others.)

```

\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n

```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the `<key>` unknown.

```

230 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
231 {
232     \tl_if_blank:nTF {#2}
233     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
234     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
235 }
236 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
237 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
238 {
239     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
240 }

```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

11.12 Definition of core base variables

```

\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V

```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{<argument>}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or `stores` information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```

\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str

```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

241 \tl_new:N \l__spintent_luaset_sign_tl
242 \tl_new:N \l__spintent_luaset_part_int_tl
243 \tl_new:N \l__spintent_luaset_dec_sep_tl
244 \tl_new:N \l__spintent_luaset_part_dec_tl
245 \tl_new:N \l__spintent_luaset_part_period_tl
246 \str_new:N \l__spintent_luaset_has_integer_str
247 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

```

\l__spintent_luaset_only_part_int_str
\l__spintent_luaset_only_part_dec_str
\l__spintent_luaset_only_part_period_str
\l__spintent_luaset_format_part_int_str
\l__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\l__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\l__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the “*integer part*” and `\l__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```

248 \str_new:N \l__spintent_luaset_only_part_int_str
249 \str_new:N \l__spintent_luaset_only_part_dec_str
250 \str_new:N \l__spintent_luaset_only_part_period_str
251 \str_new:N \l__spintent_luaset_format_part_int_str
252 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

```

\l__spintent_luaset_millions_str
\l__spintent_luaset_decimal_period_str
\l__spintent_luaset_not_decimal_period_str
\l__spintent_luaset_not_decimal_not_period_str
\l__spintent_luaset_dec_is_all_zeros_str
\l__spintent_luaset_has_exponent_str
\l__spintent_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

253 \str_new:N \l__spintent_luaset_millions_str
254 \str_new:N \l__spintent_luaset_decimal_period_str
255 \str_new:N \l__spintent_luaset_not_decimal_period_str
256 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
257 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
258 \str_new:N \l__spintent_luaset_has_exponent_str
259 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

```

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

260 \str_new:N \l__spintent_luaset_has_units_str
261 \str_new:N \l__spintent_luaset_status_str
262 \tl_new:N \l__spintent_luaset_arg_above_tl
263 \tl_new:N \l__spintent_luaset_arg_below_tl
264 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

265 \msg_new:nnnn { spintent } { need-math-mode }
266 {
267   Math~mode~required~for~'#1'~\msg_line_context:.
268 }
269 {
270   Use~inside~$...$~or~in~math~environment.
271 }
272 \msg_new:nnnn { spintent } { need-text-mode }
273 {
274   Text~mode~required~for~'#1'~\msg_line_context:.
275 }
276 {
277   Use~outside~of~$...$~or~math~environment.
278 }

```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the *intent functions* passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

The variable `\l__spintenc_spnum_read_terse_bool` handled by the key `read-sign`, controls the *intent* passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “*positivo*” or “*negativo*” after reading the number; if set to `terse`, it will say “*más*” or “*menos*” before reading the number.

The variable `\l__spintenc_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the *intent* to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘,’; if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\l__spintenc_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘·’ when set to `cdot`.

The variable `\l__spintenc_spnum_intent_read_sign_str` will *store* the spoken translation used in the *intent* function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintenc_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the *intent* function passed to `\MathMLintent` for reading the decimal separation as a “*coma*” or a “*punto*” according to the value found in the variable `\l__spintenc_luaset_dec_sep_tl`.

```

279 \bool_new:N \l__spintenc_spnum_read_terse_bool
280 \str_new:N \l__spintenc_spnum_read_period_sep_str
281 \tl_new:N \l__spintenc_spnum_notation_sym_tl
282 \str_new:N \l__spintenc_spnum_intent_read_sign_str
283 \str_new:N \l__spintenc_spnum_intent_read_dec_sep_str

```

(End of definition for `\l__spintenc_spnum_read_terse_bool` and others.)

```

cifras-sep      Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.
read-sign
read-period-sep
notation-sym
284 \keys_define:nn { spintenc/spnum }
285 {
286   cifras-sep          .bool_set:N = \l__spintenc_spnum_cifras_sep_bool,
287   cifras-sep          .initial:n = true,
288   cifras-sep          .value_required:n = true,
289   read-sign           .choices:nn =
290     { terse , clear }
291     {
292       \int_case:nn { \l_keys_choice_int }
293       {
294         {1} { \bool_set_true:N \l__spintenc_spnum_read_terse_bool }
295         {2} { \bool_set_false:N \l__spintenc_spnum_read_terse_bool }
296       }
297     },
298   read-sign           .initial:n = terse,
299   read-sign           .value_required:n = true,
300   read-sign / unknown .code:n =
301     \msg_error:nneee { spintenc } { unknown-choice }
302     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
303   read-period-sep     .choices:nn =
304     { coma , punto }
305     {
306       \int_case:nn { \l_keys_choice_int }
307       {
308         {1} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { coma } }
309         {2} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { punto } }
310       }
311     },
312   read-period-sep     .initial:n = coma,
313   read-period-sep     .value_required:n = true,
314   read-period-sep / unknown .code:n =
315     \msg_error:nneee { spintenc } { unknown-choice }
316     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
317   notation-sym        .choices:nn =
318     { times , cdot }

```

```

319     {
320       \int_case:nn { \l_keys_choice_int }
321       {
322         {1} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \times } }
323         {2} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \cdot } }
324       }
325     },
326     notation-sym .initial:n = cdot,
327     notation-sym .value_required:n = true,
328     notation-sym / unknown .code:n =
329     \msg_error:nneee { spintent } { unknown-choice }
330     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
331     unknown .code:n =
332     {
333       \str_set:Nn \__spintent_command_name_str { snum }
334       \__spintent_unknown_keys_cmd:n {#1}
335     },
336   }

```

(End of definition for *cifras-sep* and others.)

11.14.2 Internal functions for \snum

```

\__spintent_snum_render_part:n
\__spintent_snum_render_int:
\__spintent_snum_render_dec:

```

The function `\__spintent_snum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_snum_render_int:` and `\__spintent_snum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintenc`. It evaluates whether the key *cifras-sep* is active or not.

```

337 \cs_new_protected:Nn \__spintent_snum_render_part:n
338 {
339   \MathMLintenc { \str_use:c { l__spintenc_luaset_only_part_#1_str } :number }
340   {
341     \bool_if:NTF \__spintenc_snum_cifras_sep_bool
342     { \str_use:c { l__spintenc_luaset_format_part_#1_str } }
343     { \tl_use:c { l__spintenc_luaset_part_#1_tl } }
344   }
345 }
346 \cs_new_protected:Nn \__spintenc_snum_render_int:
347 {
348   \__spintenc_snum_render_part:n { int }
349 }
350 \cs_new_protected:Nn \__spintenc_snum_render_dec:
351 {
352   \__spintenc_snum_render_part:n { dec }
353 }

```

(End of definition for `\__spintenc_snum_render_part:n`, `\__spintenc_snum_render_int:`, and `\__spintenc_snum_render_dec:`.)

```

\__spintenc_snum_render_only_period:
\__spintenc_snum_print_period_bar:
\__spintenc_snum_render_period_sep:

```

The function `\__spintenc_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintenc` which will be used by `\MathMLarg` in the function `\__spintenc_snum_print_period_bar:`.

```

354 \cs_new_protected:Nn \__spintenc_snum_render_only_period:
355 {
356   \MathMLintenc { \l__spintenc_luaset_only_part_period_str :number }
357   {
358     \l__spintenc_luaset_part_period_tl
359   }
360 }

```

The function `\__spintenc_snum_print_period_bar:` inserts the function `\__spintenc_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintenc`, generating the correct spoken for the “infinite periodic decimal part”.

```

361 \cs_new_protected:Nn \__spintenc_snum_print_period_bar:
362 {
363   \MathMLintenc { _periódico:postfix($dp) }
364   {
365     {
366       \overline
367       {
368         \MathMLarg { dp } { \__spintenc_snum_render_only_period: }
369       }
370     }
371   }

```

```
372 }
```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```
373 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
374 {
375   \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
376   {
377     \MathMLintent { _coma:prefix($pp) }
378     {
379       \mathord{,}
380       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
381     }
382   }
383   {
384     \MathMLintent { _punto:prefix($pp) }
385     {
386       \mathord{.}
387       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
388     }
389   }
390 }
```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

```
\__spintent_snum_render_decimal_sep:
```

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```
391 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
392 {
393   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
394   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
395   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
396   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
397   {
398     \mathord { \l__spintent_luaset_dec_sep_tl }
399     \MathMLarg { dp }
400     {
401       \__spintent_inner_sep:
402       \__spintent_snum_render_dec:
403     }
404   }
405   % period part
406   \str_if_eq:VnTF \l__spintent_luaset_decimal_period_str { true }
407   {
408     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
409     {
410       \__spintent_inner_sep:
411       \__spintent_snum_print_period_bar:
412     }
413     {
414       \MathMLintent { _con:prefix($pnum) }
415       {
416         \__spintent_inner_sep:
417         \MathMLarg { pnum }
418         {
419           \__spintent_snum_print_period_bar:
420         }
421       }
422     }
423   }
424 }
```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

```
\__spintent_snum_print_part_dec:
\__spintent_snum_print_part_period:
```

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”).

It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintenc_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

425 \cs_new_protected:Nn \__spintenc_snum_print_part_dec:
426 {
427   \tl_if_empty:NF \l__spintenc_luaset_part_dec_tl
428   {
429     \__spintenc_snum_render_decimal_sep:
430   }
431 }
432 \cs_new_protected:Nn \__spintenc_snum_print_part_period:
433 {
434   \tl_if_empty:NF \l__spintenc_luaset_part_period_tl
435   {
436     \str_if_eq:VnTF \l__spintenc_luaset_not_decimal_period_str { true }
437     { \__spintenc_snum_render_period_sep: }
438     {
439       \tl_if_empty:NT \l__spintenc_luaset_part_dec_tl
440       {
441         \__spintenc_snum_render_period_sep:
442       }
443     }
444   }
445 }

```

(End of definition for `\__spintenc_snum_print_part_dec:` and `\__spintenc_snum_print_part_period:`.)

```

\__spintenc_snum_print_integer:
\__spintenc_snum_print_num_start:
\__spintenc_snum_print_number_core:

```

The function `\__spintenc_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintenc_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“más” or “menos”) or as a postfix (“positivo” or “negativo”), and wraps the intent around the rest of the number.

The function `\__spintenc_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

 Note for `\__spintenc_invisible_sep:`.

```

446 \cs_new_protected:Nn \__spintenc_snum_print_integer:
447 {
448   \str_if_eq:VnTF \l__spintenc_luaset_not_decimal_not_period_str { true }
449   { \__spintenc_snum_render_int: }
450   {
451     \MathMLintent { \l__spintenc_luaset_only_part_int_str :number:prefix($next) }
452     {
453       \bool_if:NTF \l__spintenc_snum_cifras_sep_bool
454       { \l__spintenc_luaset_format_part_int_str }
455       { \l__spintenc_luaset_part_int_tl }
456       \MathMLarg { next }
457       {
458         \tl_if_empty:NTF \l__spintenc_luaset_part_dec_tl
459         { \__spintenc_snum_print_part_period: }
460         { \__spintenc_snum_print_part_dec: }
461       }
462     }
463   }
464 }
465 \cs_new_protected:Nn \__spintenc_snum_print_num_start:
466 {
467   \tl_if_empty:NTF \l__spintenc_luaset_sign_tl
468   {
469     \__spintenc_invisible_sep: % need
470     \__spintenc_snum_print_integer:
471   }
472   {
473     \bool_if:NTF \l__spintenc_snum_read_terse_bool
474     {
475       \str_if_eq:VnTF \l__spintenc_luaset_sign_tl { + }

```

```

476         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _más:prefix($n)} }
477         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _menos:prefix($n)} }
478     }
479     {
480         \str_if_eq:VnTF \__spinttent_luaset_sign_tl { + }
481         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _positivo:postfix($n) } }
482         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _negativo:postfix($n) } }
483     }
484     \__spinttent_invisible_sep: % need
485     \MathMLintent { \__spinttent_snum_intent_read_sign_str }
486     {
487         \__spinttent_luaset_sign_tl
488         \MathMLarg { n } { \__spinttent_snum_print_integer: }
489     }
490 }
491 }
492 \cs_new_protected:Nn \__spinttent_snum_print_number_core:
493 {
494     \__spinttent_snum_print_num_start:
495     \str_if_eq:VnTF \__spinttent_luaset_has_exponent_str { true }
496     {
497         \MathMLintent { _por } { \__spinttent_snum_notation_sym_tl }
498         10^{\__spinttent_luaset_exponent_tl }
499     }
500 }

```

(End of definition for `\__spinttent_snum_print_integer:`, `\__spinttent_snum_print_num_start:`, and `\__spinttent_snum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

501 \msg_new:nnnn { spinttent } { spnum-has-units }
502 {
503     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(''#1'').
504 }
505 {
506     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
507 }
508 \msg_new:nnnn { spinttent } { spnum-no-digits }
509 {
510     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
511 }
512 {
513     You~provided~''#1',~which~contains~no~digits.
514 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[\langle key = val \rangle]$ argument in ‘#1’ and a mandatory $\langle rational number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spinttent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spinttent_snum_print_number_core:` to render the final number, and finally closes the group.

```

515 \NewDocumentCommand \spnum { O{} m }
516 {
517     \mode_if_math:TF
518     {
519         \group_begin:
520         \keys_set:nn { spinttent/spnum } { #1 }
521         \__spinttent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
522         \str_if_eq:VnTF \__spinttent_luaset_has_units_str { true }
523         { \msg_error:nnn { spinttent } { spnum-has-units } { #2 } }
524         {
525             \tl_if_empty:NTF \__spinttent_luaset_part_int_tl
526             { \msg_error:nnn { spinttent } { spnum-no-digits } { #2 } }
527             { \__spinttent_snum_print_number_core: }

```

```

528         }
529     \group_end:
530 }
531 { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
532 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

```

\__spintent_spunit_luaset_canonical_str
\__spintent_spunit_luaset_lookup_status_str
\__spintent_spunit_luaset_read_str
\__spintent_spunit_luaset_is_compact_str
\__spintent_spunit_luaset_compact_exp_str
\__spintent_spunit_luaset_is_sexagesimal_str
\__spintent_spunit_luaset_status_str

```

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

533 \str_new:N \__spintent_spunit_luaset_canonical_str
534 \str_new:N \__spintent_spunit_luaset_lookup_status_str
535 \str_new:N \__spintent_spunit_luaset_read_str
536 \str_new:N \__spintent_spunit_luaset_is_compact_str
537 \str_new:N \__spintent_spunit_luaset_compact_exp_str
538 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
539 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

```

\__spintent_spunit_exponent_tl
\__spintent_spunit_unit_name_tl
\__spintent_spunit_read_slash_str
\__spintent_spunit_exp_seq
\__spintent_spunit_mult_seq
\__spintent_spunit_is_mult_bool
\__spintent_spunit_read_cdot_bool

```

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

540 \tl_new:N \__spintent_spunit_exponent_tl
541 \tl_new:N \__spintent_spunit_unit_name_tl
542 \str_new:N \__spintent_spunit_read_slash_str
543 \seq_new:N \__spintent_spunit_exp_seq
544 \seq_new:N \__spintent_spunit_mult_seq
545 \bool_new:N \__spintent_spunit_is_mult_bool
546 \bool_new:N \__spintent_spunit_read_cdot_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

```

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

```

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle argument \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

547 \msg_new:nnnn { spintent } { spunit-invalid-unit }
548 {
549     The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
550 }
551 {
552     Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
553 }
554 \msg_new:nnnn { spintent } { spunit-no-units }
555 {
556     The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
557 }
558 {
559     Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
560 }
561 \msg_new:nnnn { spintent } { inline-numeric-denominator }
562 {
563     Illegal~numeric~coefficient~in~inline~denominator~'#1'.
564 }
565 {
566     The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
567     does~not~allow~numbers~after~the~slash~(/).
568 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

569 \keys_define:nn { spintent/spunit }
570 {
571   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
572   print-cdot      .initial:n   = false,
573   print-cdot      .value_required:n = true,
574   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
575   read-cdot       .initial:n   = false,
576   read-cdot       .value_required:n = true,
577   read-slash      .choices:nn =
578     { por , partido-por }
579   {
580     \int_case:nn { \l_keys_choice_int }
581     {
582       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
583       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
584     }
585   },
586   read-slash      .initial:n   = por,
587   read-slash      .value_required:n = true,
588   read-slash / unknown .code:n =
589     \msg_error:nneee { spintent } { unknown-choice }
590     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
591   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
592   cifras-sep      .value_required:n = true,
593   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
594   read-period-sep .value_required:n = true,
595   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
596   notation-sym    .value_required:n = true,
597   unknown         .code:n =
598     {
599       \str_set:Nn \l__spintent_command_name_str { spunit }
600       \__spintent_unknown_keys_cmd:n {#1}
601     },
602 }

```

(End of definition for `read-cdot` and others.)

11.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘*ℓ*’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

603 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
604 {
605   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
606   {
607     { ℓ } { \ell }
608   }
609   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
610 }

```

(End of definition for `\__spintent_spunit_format_canonical:`.)

`\__spintent_spunit_process_base_exp:n` The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘*#1*’. It first looks up the unit in the dictionary; if not found, flags it as invalid. Otherwise, unless this is the first unit in a product (`\__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

611 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
612 {
613   \tl_clear:N \l__spintent_spunit_exponent_tl
614   \__spintent_luafun_spunit_lookup_alias:n { #1 }
615   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
616   {
617     \str_set:Nn \l__spintent_luaset_status_str { invalid }
618   }
619   {
620     \bool_if:NF \l__spintent_spunit_is_mult_bool

```

```

621     {
622         \bool_if:NTF \l__spinttent_spunit_print_cdot_bool
623         {
624             \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
625             { \MathMLintent { por } { \cdot } }
626             { \MathMLintent { :silent } { \cdot } }
627         }
628         {
629             \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
630             { \MathMLintent { por } { \, } }
631             { \, }
632         }
633     }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spinttent_spunit_luaset_is_compact_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

634     \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_compact_str { true }
635     {
636         {
637             \MathMLintent { \l__spinttent_spunit_luaset_read_str }
638             {
639                 \l__spinttent_spunit_format_canonical:
640             }
641         }
642         ^ { \l__spinttent_spunit_luaset_compact_exp_str }
643     }
644     {
645         \seq_set_split:Nnn \l__spinttent_spunit_exp_seq { ^ } { #1 }
646         \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_unit_name_tl
647         \seq_if_empty:NF \l__spinttent_spunit_exp_seq
648         {
649             \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_exponent_tl
650         }
651         \MathMLintent { \l__spinttent_spunit_luaset_read_str }
652         {
653             \l__spinttent_spunit_format_canonical:
654         }
655         \tl_if_empty:NF \l__spinttent_spunit_exponent_tl
656         {
657             ^ { \l__spinttent_spunit_exponent_tl }
658         }
659     }
660     \bool_set_false:N \l__spinttent_spunit_is_mult_bool
661 }
662 }

```

(End of definition for `\l__spinttent_spunit_process_base_exp:n`)

`\l__spinttent_spunit_process_mult:n`
`\l__spinttent_spunit_process_mult:V`

The function `\l__spinttent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the `{(argument)}` at each “star symbol” and applies `\l__spinttent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

663 \cs_new_protected:Nn \l__spinttent_spunit_process_mult:n
664 {
665     \bool_set_true:N \l__spinttent_spunit_is_mult_bool
666     \seq_set_split:Nnn \l__spinttent_spunit_mult_seq { * } { #1 }
667     \seq_map_function:NN \l__spinttent_spunit_mult_seq \l__spinttent_spunit_process_base_exp:n
668 }
669 \cs_generate_variant:Nn \l__spinttent_spunit_process_mult:n { V }

```

(End of definition for `\l__spinttent_spunit_process_mult:n`)

`\l__spinttent_print_spunit_hierarchy:`

The function `\l__spinttent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

670 \cs_new_protected:Nn \l__spinttent_print_spunit_hierarchy:
671 {
672     \tl_if_empty:NF \l__spinttent_luaset_arg_above_tl
673     {
674         \l__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_above_tl
675         \tl_if_empty:NF \l__spinttent_luaset_arg_below_tl
676         {
677             \MathMLintent { \l__spinttent_spunit_read_slash_str } { / }

```

```

678         \__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_below_tl
679     }
680 }
681 }

```

(End of definition for `\__spinttent_print_spunit_hierarchy:`.)

`\__spinttent_spunit_safe_exec:n` The function `\__spinttent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “*without any space in between*”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

682 \cs_new_protected:Nn \__spinttent_spunit_safe_exec:n
683 {
684     \str_if_eq:VnTF \l__spinttent_luaset_has_units_str { false }
685     { \msg_error:nnn { spinttent } { spunit-no-units } { #1 } }
686     {
687         \str_if_eq:VnTF \l__spinttent_luaset_status_str { valid }
688         {
689             \str_if_eq:VnTF
690             \l__spinttent_luaset_denom_has_number_str { true }
691             {
692                 \msg_error:nnn { spinttent }
693                 { inline-numeric-denominator } { #1 }
694             }
695             {
696                 \__spinttent_luafun_spunit_lookup_alias:V \l__spinttent_luaset_arg_above_tl
697                 \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_sexagesimal_str { true }
698                 {
699                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
700                     {
701                         \__spinttent_spnum_print_number_core:
702                     }
703                     \MathMLintent { \l__spinttent_spunit_luaset_read_str }
704                     {
705                         \mathord
706                         {
707                             \str_case:Vn \l__spinttent_spunit_luaset_canonical_str
708                             {
709                                 { " } { \prime \! \prime }
710                                 { ' } { \prime
711                                 { ° } { °
712                             }
713                         }
714                     }
715                 }
716                 {
717                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
718                     {
719                         \__spinttent_spnum_print_number_core: \,
720                     }
721                     \__spinttent_print_spunit_hierarchy:
722                 }
723             }
724         }
725         { \msg_error:nnn { spinttent } { spunit-invalid-unit } { #1 } }
726     }
727 }

```

(End of definition for `\__spinttent_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional `[⟨key = val⟩]` argument in ‘#1’ and a mandatory `{⟨unit expression⟩}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{⟨argument⟩}` to Lua module for parsing via `\__spinttent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spinttent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

728 \NewDocumentCommand \spunit { 0{ } m }

```

```

729 {
730   \mode_if_math:TF
731   {
732     \group_begin:
733     \keys_set:nn { spintent/spunit } { #1 }
734     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
735     \__spintent_spunit_safe_exec:n { #2 }
736     \group_end:
737   }
738   { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
739 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintent_spunit_luaset_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

740 \msg_new:nnnn { spintent } { spnewunit-duplicate }
741 {
742   The~unit~symbol~or~alias~'#1'~is~already~defined!
743 }
744 {
745   You~cannot~redefine~with~\token_to_str:N \spnewunit.
746 }
747 \msg_new:nnnn { spintent } { spunitalias-duplicate }
748 {
749   The~alias~'#1'~is~already~defined~or~reserved!
750 }
751 {
752   You~cannot~redefine~an~existing~unit~or~another~alias.
753 }
754 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
755 {
756   The~expression~'#2'~for~alias~'#1'~is~invalid.
757 }
758 {
759   It~must~contain~only~defined~units,~exponents~and~one~'/'~.
760 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

761 \NewDocumentCommand \spnewunit { m m }
762 {
763   \mode_if_math:TF
764   {
765     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
766   }
767   {
768     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
769     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
770     {
771       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
772     }
773   }
774 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘`#1`’ and the target expression ‘`#2`’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

775 \NewDocumentCommand \spunitalias { m m }
776 {
777   \mode_if_math:TF
778   {
779     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
780   }
781   {
782     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
783     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
784     {
785       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
786     }
787     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
788     {
789       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
790     }
791   }
792 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

793 \str_new:N \l__spintent_spmoney_luaset_symbol_str
794 \str_new:N \l__spintent_spmoney_luaset_position_str
795 \str_new:N \l__spintent_spmoney_luaset_sing_str
796 \str_new:N \l__spintent_spmoney_luaset_plur_str
797 \str_new:N \l__spintent_spmoney_luaset_conde_str
798 \str_new:N \l__spintent_spmoney_luaset_subsing_str
799 \str_new:N \l__spintent_spmoney_luaset_subplur_str
800 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
801 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
802 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

```

\l__spintent_spmoney_sub_units_bool
\l__spintent_spmoney_intent_money_str
\l__spintent_spmoney_intent_sign_str
\l__spintent_spmoney_intent_sub_unit_str
\l__spintent_spmoney_res_main_voice_str
\l__spintent_spmoney_subnum_int
\l__spintent_spmoney_extracted_curr_str
\l__spintent_spmoney_extracted_num_tl

803 \bool_new:N \l__spintent_spmoney_sub_units_bool
804 \str_new:N \l__spintent_spmoney_intent_money_str
805 \str_new:N \l__spintent_spmoney_intent_sign_str
806 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
807 \str_new:N \l__spintent_spmoney_res_main_voice_str
808 \int_new:N \l__spintent_spmoney_subnum_int
809 \str_new:N \l__spintent_spmoney_extracted_curr_str
810 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

currency-no-subunits
invalid-currency

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

811 \msg_new:nnnn { spintent } { currency-no-subunits }
812 {
813     The~currency~'#1'~does~not~support~subunits.~
814     The~option~'sub-units=true'~is~ignored.
815 }
816 {
817     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
818     do~not~have~fractional~subunits~registered~in~Lua~module.~
819 }
820 \msg_new:nnnn { spintent } { invalid-currency }
821 {
822     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
823 }
824 {
825     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
826     the~Lua~module.
827 }
```

(End of definition for currency-no-subunits and invalid-currency.)

11.17.2 Auxiliary functions for \spmoney

__spintenc_spmoney_set_currency:n
__spintenc_spmoney_normalize_key:n
__spintenc_spmoney_normalize_key:V

The function `\__spintenc_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintenc_spmoney_normalize_key:n`, which interfaces with the Lua module `spintenc.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintenc_spmoney_luaset_currency_arg_str` for subsequent processing.

```

828 \cs_new_protected:Nn \__spintenc_spmoney_set_currency:n
829 {
830     \__spintenc_spmoney_normalize_key:n {#1}
831     \str_if_eq:VnTF \__spintenc_spmoney_luaset_status_str { notfound }
832     {
833         \msg_error:nnn { spintent } { invalid-currency } {#1}
834     }
835     { \str_set:Nn \__spintenc_spmoney_luaset_currency_arg_str {#1} }
836 }
837 \cs_new_protected:Nn \__spintenc_spmoney_normalize_key:n
838 {
839     \__spintenc_luafun_spmoney_normalize_key:n { \exp_not:n { #1 } }
840 }
841 \cs_generate_variant:Nn \__spintenc_spmoney_normalize_key:n { V }
```

(End of definition for __spintenc_spmoney_set_currency:n and __spintenc_spmoney_normalize_key:n.)

11.17.3 Definition of keys for \spmoney

currency
sub-units
money-math
cifras-sep

Now we add the keys `currency`, `sub-units`, `money-math` and `cifras-sep`.

```

842 \keys_define:nn { spintenc / spmoney }
843 {
844     currency      .code:n      = { \__spintenc_spmoney_set_currency:n {#1} },
845     currency      .initial:n    = { pesos },
846     currency      .value_required:n = true,
847     sub-units     .bool_set:N    = \__spintenc_spmoney_sub_units_bool,
848     sub-units     .value_required:n = true,
849     money-math    .bool_set:N    = \__spintenc_spmoney_money_math_bool,
850     money-math    .initial:n    = { true },
851     money-math    .value_required:n = true,
852     cifras-sep    .meta:nn      = { spintenc/spnum } { cifras-sep = {#1} },
853     cifras-sep    .value_required:n = true,
854     unknown       .code:n      =
855     {
856         \str_set:Nn \__spintenc_command_name_str { spmoney }
857         \__spintenc_unknown_keys_cmd:n {#1}
858     },
859 }
```

(End of definition for currency and others.)

11.17.4 Internal functions for \spmoney

__spintrent_spmoney_validate_money:

The function `\__spintrent_spmoney_validate_money`: verifies if the parsed input contains an embedded currency identifier by checking `\l__spintrent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintrent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintrent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintrent.lua` via `\__spintrent_luafun_spmoney_lookup_data:V`.

```

860 \cs_new_protected:Nn \__spintrent_spmoney_validate_money:
861 {
862   \str_if_eq:VnT \l__spintrent_luaset_has_units_str { true }
863   {
864     \__spintrent_spmoney_normalize_key:V \l__spintrent_luaset_arg_above_tl
865     \str_if_eq:VnTF \l__spintrent_spmoney_luaset_status_str { notfound }
866     { \msg_error:nne { spintrent } { invalid-currency } { \l__spintrent_luaset_arg_above_tl } }
867     {
868       \str_set:NV
869         \l__spintrent_spmoney_luaset_currency_arg_str
870         \l__spintrent_luaset_arg_above_tl
871     }
872   }
873   \str_set:Ne \l__spintrent_spmoney_luaset_status_str { found }
874   \__spintrent_luafun_spmoney_lookup_data:V \l__spintrent_spmoney_luaset_currency_arg_str
875 }

```

(End of definition for __spintrent_spmoney_validate_money:.)

__spintrent_spmoney_print_symbol:

The function `\__spintrent_spmoney_print_symbol`: renders the “*currency symbol*”. If `\l__spintrent_spmoney_luaset_print_iso_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”, directly in math mode when `\l__spintrent_spmoney_money_math_bool` is `true`, or wrapped inside `\text` when it is `false`. For dollar signs ‘\$’, the symbol is typeset through the control sequence `\$` in both cases.

```

876 \cs_new_protected:Nn \__spintrent_spmoney_print_symbol:
877 {
878   \str_if_eq:VnTF \l__spintrent_spmoney_luaset_print_iso_str { true }
879   { \mathrm { \l__spintrent_spmoney_luaset_currency_arg_str } }
880   {
881     \str_if_eq:VnTF \l__spintrent_spmoney_luaset_symbol_str { $ }
882     {
883       \bool_if:NTF \l__spintrent_spmoney_money_math_bool
884       { \$ } { \text { \$ } }
885     }
886     {
887       \bool_if:NTF \l__spintrent_spmoney_money_math_bool
888       { \l__spintrent_spmoney_luaset_symbol_str }
889       { \text { \l__spintrent_spmoney_luaset_symbol_str } }
890     }
891   }
892 }

```

(End of definition for __spintrent_spmoney_print_symbol:.)

__spintrent_spmoney_render_layout:

The function `\__spintrent_spmoney_render_layout`: determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintrent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

893 \cs_new_protected:Nn \__spintrent_spmoney_render_layout:
894 {
895   \str_if_eq:VnTF \l__spintrent_spmoney_luaset_position_str { pre }
896   {
897     \__spintrent_spmoney_print_symbol:
898     \MathMLarg { n } { \__spintrent_spmoney_render_number_node: }
899   }
900   {
901     \MathMLarg { n } { \__spintrent_spmoney_render_number_node: }
902     \__spintrent_spmoney_print_symbol:
903   }
904 }

```

(End of definition for __spintrent_spmoney_render_layout:.)

__spintent_spmoney_render_number_node:
__spintent_spmoney_render_only_integer_node:

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

905 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
906 {
907     \__spintent_spmoney_render_int:
908     \__spintent_spmoney_print_part_dec:
909 }
910 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
911 {
912     \__spintent_spmoney_render_int:
913 }
```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

__spintent_spmoney_print_int:

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

914 \cs_new_protected:Nn \__spintent_spmoney_print_int:
915 {
916     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
917     {
918         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
919         { \__spintent_spmoney_set_and_print_sub_units: }
920         { \__spintent_spmoney_intent_int: }
921     }
922     {
923         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
924         {
925             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
926         }
927         {
928             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
929         }
930         \__spintent_invisible_sep:
931         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
932         {
933             \l__spintent_luaset_sign_tl
934             \MathMLarg { n }
935             {
936                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
937                 { \__spintent_spmoney_set_and_print_sub_units: }
938                 { \__spintent_spmoney_intent_int: }
939             }
940         }
941     }
942 }
```

(End of definition for `\__spintent_spmoney_print_int:`.)

__spintent_spmoney_intent_int:

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix MathML intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

943 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
944 {
945     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
946     {
947         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
948         {
949             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
950         }
951         {
952             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
953         }
954         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
955         {
956             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
957         }
958     }
```

```

959     {
960         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
961         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
962         {
963             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
964             {
965                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
966             }
967             {
968                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur
969             }
970         }
971     }
972     \__spintent_invisible_sep:
973     \MathMLintant { _ \l__spintent_spmoney_intent_money_str :postfix($n) }
974     {
975         \__spintent_spmoney_render_layout:
976     }
977 }

```

(End of definition for `\__spintent_spmoney_intent_int:`.)

`\__spintent_spmoney_check_sub_units:`
`\__spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag. The flag is also disabled, without a warning, when the amount has no decimal part, since there are no “*subunits*” to voice.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘.5’ to ‘.50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:.`

```

978 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
979 {
980     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
981     {
982         \msg_warning:nne { spintent } { currency-no-subunits }
983         { \l__spintent_spmoney_luaset_currency_arg_str }
984         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
985     }
986     \tl_if_empty:NT \l__spintent_luaset_dec_sep_tl
987     { \bool_set_false:N \l__spintent_spmoney_sub_units_bool }
988 }
989 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
990 {
991     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
992     {
993         \int_set:Nn \l__spintent_spmoney_subnum_int
994         { \l__spintent_luaset_only_part_dec_str * 10 }
995         \str_set:Ne \l__spintent_luaset_only_part_dec_str
996         { \l__spintent_luaset_only_part_dec_str 0 }
997     }
998     {
999         \int_set:Nn \l__spintent_spmoney_subnum_int
1000         { \l__spintent_luaset_only_part_dec_str }
1001     }
1002     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
1003     {
1004         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1005         { _ \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1006     }
1007     {
1008         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1009         { _ \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1010     }
1011     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1012     {
1013         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1014         \l__spintent_spmoney_luaset_sing_str
1015     }
1016     {
1017         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1018         \l__spintent_spmoney_luaset_plur_str

```

```

1019     }
1020     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1021     {
1022         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1023         \l__spintent_spmoney_luaset_conde_str
1024     }
1025     \__spintent_spmoney_build_tree:
1026 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`.)

```

\__spintent_spmoney_build_tree:
\__spintent_spmoney_print_sep_con:
\__spintent_spmoney_print_dec_sub_unit:

```

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘_con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```

1027 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1028 {
1029     \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1030     {
1031         \MathMLintent { _ \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1032         {
1033             \__spintent_spmoney_print_symbol:
1034             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1035         }
1036         \__spintent_spmoney_print_sep_con:
1037         \__spintent_spmoney_print_dec_sub_unit:
1038     }
1039     {
1040         \MathMLintent { _ \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1041         {
1042             \__spintent_inner_sep:
1043             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1044         }
1045         \__spintent_spmoney_print_sep_con:
1046         \__spintent_spmoney_print_dec_sub_unit:
1047         \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1048     }
1049 }
1050 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1051 {
1052     \MathMLintent { _con }
1053     {
1054         \mathord { \l__spintent_luaset_dec_sep_tl }
1055     }
1056 }
1057 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1058 {
1059     \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1060     {
1061         \__spintent_inner_sep:
1062         \MathMLarg { sub }
1063         {
1064             \__spintent_spmoney_render_dec:
1065         }
1066     }
1067 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

```
\__spintent_spmoney_parse_and_route:n
```

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\{(\textit{argument})\}$ passed in ‘#1’ using Lua module to extract the numeric value and any “*currency*” identifier. If a “*currency*” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “*currency*”. If “*subunit*” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1068 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1069 {
1070     \__spintent_luafun_spmoney_prepare_input:n { \exp_not:n { #1 } }

```

```

1071     \__spintent_luafun_clean_split_and_set:n { \exp_not:V \l__spintent_spmoney_extracted_num_tl }
1072     \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1073     {
1074         \str_set:Ne \l__spintent_luaset_has_units_str { true }
1075         \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1076     }
1077     \__spintent_spmoney_validate_money:
1078     \bool_if:NT \l__spintent_spmoney_sub_units_bool
1079     { \__spintent_spmoney_check_sub_units: }
1080     \__spintent_spmoney_print_int:
1081 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney` The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1082 \NewDocumentCommand \spmoney { 0{} m }
1083 {
1084     \mode_if_math:TF
1085     {
1086         \group_begin:
1087         \keys_set:nn { spintent / spmoney } { #1 }
1088         \__spintent_spmoney_parse_and_route:n { #2 }
1089         \group_end:
1090     }
1091     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1092 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1093 \str_new:N \l__spintent_spshort_luaset_status_str
1094 \str_new:N \l__spintent_spshort_luaset_layout_str
1095 \str_new:N \l__spintent_spshort_luaset_expand_str
1096 \str_new:N \l__spintent_spshort_luaset_output_str
1097 \str_new:N \l__spintent_spshort_luaset_base_str
1098 \str_new:N \l__spintent_spshort_luaset_suffix_str

```

(End of definition for `\l__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

`unknown-abbreviation` The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1099 \msg_new:nnnn { spintent } { unknown-abbreviation }
1100 { The~abbreviation~or~ordinal~'~#1~'~is~invalid~or~not~recognized. }
1101 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

```

__spintent_spshort_tag_span:nn
__spintent_spshort_tag_span:en

```

The internal function `__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1102 \cs_new_protected:Nn __spintent_spshort_tag_span:nn
1103 {
1104   \tag_mc_end_push:
1105   \tag_struct_begin:n { tag=Span, E={ #1 } }
1106   \tag_mc_begin:n { tag=Span }
1107   \tag_suspend:n { \spshort }
1108   #2
1109   \tag_resume:n { \spshort }
1110   \tag_mc_end:
1111   \tag_struct_end:
1112   \tag_mc_begin_pop:n {}
1113 }
1114 \cs_generate_variant:Nn __spintent_spshort_tag_span:nn { en }

```

(End of definition for `__spintent_spshort_tag_span:nn`.)

11.18.3 Definition of keys for \spshort

Now we add the keys `small-caps` and `read-arg`.

```

1115 \keys_define:nn { spintent / spshort }
1116 {
1117   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1118   small-caps .initial:n = false,
1119   small-caps .value_required:n = true,
1120   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1121   read-arg .initial:n = {},
1122   read-arg .value_required:n = true,
1123   unknown .code:n =
1124   {
1125     \str_set:Nn \__spintent_command_name_str { spshort }
1126     \__spintent_unknown_keys_cmd:n { #1 }
1127   },
1128 }

```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for \spshort

```
__spintent_spshort_print:nn
```

The internal function `__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `__spintent_spshort_render_bypass:n`.

```

1129 \cs_new_protected:Nn __spintent_spshort_print:nn
1130 {
1131   \str_clear:N \__spintent_spshort_read_arg_str
1132   \keys_set:nn { spintent / spshort } { #1 }
1133   \str_if_empty:NTF \__spintent_spshort_read_arg_str
1134   {
1135     __spintent_spshort_process_and_render:n { #2 }
1136   }
1137   {
1138     __spintent_spshort_render_bypass:n { #2 }
1139   }
1140 }

```

(End of definition for `__spintent_spshort_print:nn`.)

```
__spintent_spshort_process_and_render:n
```

The internal function `__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine `success`, `fallback` or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1141 \cs_new_protected:Nn __spintent_spshort_process_and_render:n
1142 {
1143   __spintent_luafun_spshort_process:n { #1 }

```

```

1144     \str_case:VnF \l__spintent_spshort_luaset_status_str
1145     {
1146         { success } { \l__spintent_spshort_render_success: }
1147         { fallback } { \l__spintent_spshort_render_fallback: }
1148         { error } { \l__spintent_spshort_render_error:n {#1} }
1149     }
1150     { \l__spintent_spshort_render_error:n {#1} }
1151 }

```

(End of definition for \l__spintent_spshort_process_and_render:n.)

```

\l__spintent_spshort_render_success:
\l__spintent_spshort_render_fallback:
\l__spintent_spshort_render_bypass:n
\l__spintent_spshort_render_error:n

```

The function `\l__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\l__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

Conversely, the function `\l__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the $\langle \textit{argument} \rangle$, bypassing the automated lookup Lua module. Finally, the function `\l__spintent_spshort_rende` triggers a structural exception while yielding the unformatted $\langle \textit{argument} \rangle$ to prevent fatal compilation.

```

1152 \cs_new_protected:Nn \l__spintent_spshort_render_success:
1153 {
1154     \l__spintent_spshort_tag_span:en
1155     { \l__spintent_spshort_luaset_expand_str }
1156     { \l__spintent_spshort_execute_layout: }
1157 }
1158 \cs_new_protected:Nn \l__spintent_spshort_render_fallback:
1159 {
1160     \l__spintent_spshort_tag_span:en
1161     { \l__spintent_spshort_luaset_expand_str }
1162     {
1163         \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1164         { \textsc{ \l__spintent_spshort_luaset_output_str } }
1165         { \l__spintent_spshort_luaset_output_str }
1166     }
1167 }
1168 \cs_new_protected:Nn \l__spintent_spshort_render_bypass:n
1169 {
1170     \l__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1171 }
1172 \cs_new_protected:Nn \l__spintent_spshort_render_error:n
1173 {
1174     \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1175 }

```

(End of definition for \l__spintent_spshort_render_success: and others.)

```
\l__spintent_spshort_execute_layout:
```

The internal function `\l__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional `SMALL CAPS` based on the active boolean variables.

```

1176 \cs_new_protected:Nn \l__spintent_spshort_execute_layout:
1177 {
1178     \str_case:VnF \l__spintent_spshort_luaset_layout_str
1179     {
1180         { linear_regular }
1181         {
1182             \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1183             { \textsc{ \l__spintent_spshort_luaset_output_str } }
1184             { \l__spintent_spshort_luaset_output_str }
1185         }
1186         { linear_caps }
1187         { \l__spintent_spshort_luaset_output_str }
1188         { linear_mixed }
1189         { \l__spintent_spshort_luaset_output_str }
1190         { superscript }
1191         {
1192             \l__spintent_spshort_luaset_base_str
1193             \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1194             {
1195                 \textsuperscript{
1196                     \textsc{
1197                         \str_lowercase:V \l__spintent_spshort_luaset_suffix_str

```

```

1198         }
1199     }
1200 }
1201 {
1202     \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1203 }
1204 }
1205 { small_caps_pure }
1206 {
1207     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1208     {
1209         \textsc{
1210             \str_lowercase:V \l__spintent_spshort_luaset_output_str
1211         }
1212     }
1213     { \l__spintent_spshort_luaset_output_str }
1214 }
1215 { acronym_long }
1216 { \l__spintent_spshort_luaset_output_str }
1217 }
1218 { \l__spintent_spshort_luaset_output_str }
1219 }

```

(End of definition for `\l__spintent_spshort_execute_layout:`.)

`\spshort` The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\l__spintent_spshort_print:nn` for processing and printing.

```

1220 \NewDocumentCommand \spshort { 0{} m }
1221 {
1222     \mode_if_math:TF
1223     {
1224         \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1225     }
1226     {
1227         \mode_leave_vertical:
1228         \group_begin:
1229             \l__spintent_spshort_print:nn {#1} {#2}
1230         \group_end:
1231     }
1232 }

```

(End of definition for `\spshort`. This function is documented on page 8.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as `{\argument}`, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

\l__spintent_spsiglo_luaset_error_str
\l__spintent_spsiglo_luaset_arabic_str
\l__spintent_spsiglo_luaset_roman_min_str
\l__spintent_spsiglo_print_era_str
1233 \str_new:N \l__spintent_spsiglo_luaset_error_str
1234 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1235 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1236 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

`invalid-century-format` This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the `{\argument}` to an integer within the valid range of `1` to `4000`, preventing incorrect speech synthesis.

```

1237 \msg_new:nnnn { spintent } { invalid-century-format }
1238 {

```

```

1239     Invalid~century~format~in~'#1'.
1240   }
1241   {
1242     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1243     Ensure~the~value~is~between~1~and~4000.
1244   }

```

(End of definition for *invalid-century-format*.)

11.19.2 Definition of keys for \spsiglo

`print-era` Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1245 \keys_define:nn { spintent / spsiglo }
1246   {
1247     print-era      .choices:nn =
1248     { none , ac , dc }
1249     {
1250       \int_case:nn { \l_keys_choice_int }
1251       {
1252         {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1253         {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1254         {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1255       }
1256     },
1257     print-era      .initial:n = none,
1258     print-era      .value_required:n = true,
1259     print-era / unknown .code:n =
1260     \msg_error:nneee { spintent } { unknown-choice }
1261     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1262     unknown        .code:n =
1263     {
1264       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1265       \__spintent_unknown_keys_cmd:n {#1}
1266     },
1267   }

```

(End of definition for *print-era*.)

11.19.3 Internal functions for \spsiglo

`\__spintent_spsiglo_render_simple:nn` The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1268 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1269   {
1270     \MathMLintent { #1:number } { \text { \textsc {#2} } }
1271   }
1272 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1273 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1274   {
1275     \str_if_eq:nnTF { #1 } { ac }
1276     {
1277       \MathMLintent { antes-de-cristo:postfix($z) }
1278       {
1279         \__spintent_inner_sep:
1280         \MathMLarg { z }
1281         {
1282           \MathMLintent { #2:number }
1283           {
1284             \text { \textsc {#3}~a.~C. }
1285           }
1286         }
1287       }
1288     }
1289     {
1290       \MathMLintent { después-de-cristo:postfix($z) }
1291       {
1292         \__spintent_inner_sep:
1293         \MathMLarg { z }
1294         {

```

```

1295         \MathMLintent { #2:number }
1296         {
1297             \text { \textsc {#3}~d.~C. }
1298         }
1299     }
1300 }
1301 }
1302 }
1303 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

```

\__spintent_spsiglo_tag_span:nn
\__spintent_spsiglo_tag_span:en

```

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1304 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1305 {
1306     \tag_mc_end_push:
1307     \tag_struct_begin:n { tag=Span, alt={ #1 } }
1308     \tag_mc_begin:n { tag=Span }
1309     \tag_suspend:n { \spsiglo }
1310     #2
1311     \tag_resume:n { \spsiglo }
1312     \tag_mc_end:
1313     \tag_struct_end:
1314     \tag_mc_begin_pop:n {}
1315 }
1316 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

```

\__spintent_spsiglo_print_math:n
\__spintent_spsiglo_print_text:n

```

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1317 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1318 {
1319     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1320     {
1321         \msg_error:nnnn { spintent } { invalid-century-format }
1322         { \spsiglo } { #1 }
1323     }
1324     {
1325         \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1326         {
1327             \__spintent_spsiglo_render_simple:VV
1328             \l__spintent_spsiglo_luaset_arabic_str
1329             \l__spintent_spsiglo_luaset_roman_min_str
1330         }
1331         {
1332             \__spintent_spsiglo_render_with_era:VVV
1333             \l__spintent_spsiglo_print_era_str
1334             \l__spintent_spsiglo_luaset_arabic_str
1335             \l__spintent_spsiglo_luaset_roman_min_str
1336         }
1337     }
1338 }
1339 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1340 {
1341     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1342     {
1343         \msg_error:nnnn { spintent } { invalid-century-format }
1344         { \spsiglo } { #1 }
1345     }
1346     {
1347         \str_case:Vn \l__spintent_spsiglo_print_era_str

```

```

1348     {
1349       { none }
1350     }
1351     \__spintent_spsiglo_tag_span:en
1352     { \__spintent_spsiglo_luaset_arabic_str }
1353     { \textsc { \__spintent_spsiglo_luaset_roman_min_str } }
1354   }
1355   { ac }
1356   {
1357     \__spintent_spsiglo_tag_span:en
1358     { \__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1359     { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1360   }
1361   { dc }
1362   {
1363     \__spintent_spsiglo_tag_span:en
1364     { \__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1365     { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1366   }
1367 }
1368 }
1369 }

```

(End of definition for __spintent_spsiglo_print_math:n and __spintent_spsiglo_print_text:n)

\spsiglo The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional *(keys)* passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1370 \NewDocumentCommand \spsiglo { 0{} m }
1371 {
1372   \group_begin:
1373   \keys_set:nn { spintent / spsiglo } { #1 }
1374   \__spintent_luafun_spsiglo_parse:n { #2 }
1375   \mode_if_math:TF
1376   { \__spintent_spsiglo_print_math:n { #2 } }
1377   {
1378     \mode_leave_vertical:
1379     \__spintent_spsiglo_print_text:n { #2 }
1380   }
1381   \group_end:
1382 }

```

(End of definition for \spsiglo. This function is documented on page 10.)

11.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1383 \str_new:N \__spintent_sptime_luaset_seconds_str
1384 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1385 \str_new:N \__spintent_sptime_luaset_error_str
1386 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1387 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

11.20.1 Definition of error messages

invalid-time-format

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1388 \msg_new:nnnn { spintent } { invalid-time-format }
1389 {
1390   Invalid~time~format~in~#1. \\
1391   The~argument~'~#2'~does~not~match~any~supported~time~format.
1392 }
1393 {

```

```

1394     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss.  \
1395     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1396   }

```

(End of definition for *invalid-time-format*.)

11.20.2 Internal functions for \sptime

__spintent_sptime_build_seconds:nn

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1397 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1398 {
1399   \MathMLintent { #1:prefix($z) }
1400   {
1401     \text { : }
1402     \MathMLarg { z }
1403     {
1404       \MathMLintent { _segundos:postfix($m) }
1405       {
1406         \__spintent_inner_sep:
1407         \MathMLarg { m }
1408         {
1409           \text { #2 }
1410         }
1411       }
1412     }
1413   }
1414 }

```

(End of definition for `\__spintent_sptime_build_seconds:nn`.)

__spintent_sptime_build_tree:

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:` with the appropriate prefix (‘con’ or ‘minutos-con’), or simply appends the ‘minutos’ intent tag if no seconds are present.

```

1415 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1416 {
1417   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1418   {
1419     \MathMLintent { :time }
1420     {
1421       \text { \l__spintent_sptime_luaset_base_str }
1422     }
1423     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1424     {
1425       \__spintent_sptime_build_seconds:nn { con }
1426       { \l__spintent_sptime_luaset_seconds_str }
1427     }
1428     {
1429       \__spintent_sptime_build_seconds:nn { _minutos-con }
1430       { \l__spintent_sptime_luaset_seconds_str }
1431     }
1432   }
1433   {
1434     \MathMLintent { :time }
1435     {
1436       \text { \l__spintent_sptime_luaset_base_str }
1437     }
1438     \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1439     {
1440       \MathMLintent { _minutos }
1441       {
1442         \c__spintent_invisible_sep_int
1443       }
1444     }
1445   }
1446 }

```

(End of definition for `\__spintent_sptime_build_tree:`.)

`\sptime` The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1447 \NewDocumentCommand \sptime { m }
1448 {
1449   \mode_if_math:TF
1450   {
1451     \group_begin:
1452     \__spintent_luafun_sptime_parse:n { #1 }
1453     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1454     {
1455       \msg_error:nnnn { spintent } { invalid-time-format }
1456       { \sptime } { #1 }
1457     }
1458     {
1459       \__spintent_sptime_build_tree:
1460     }
1461     \group_end:
1462   }
1463   { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1464 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1465 \str_new:N \l__spintent_spdate_luaset_output_str
1466 \str_new:N \l__spintent_spdate_luaset_error_str

```

(End of definition for `\l__spintent_spdate_luaset_output_str` and `\l__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

`invalid-date-format`

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1467 \msg_new:nnnn { spintent } { invalid-date-format }
1468 {
1469   Invalid~date~or~unsupported~format~in~#1. \\
1470   The~argument~'~#2'~does~not~match~a~supported~format.
1471 }
1472 {
1473   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\
1474   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1475 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\l__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML `intent` structure using the formatted date stored in `\l__spintent_spdate_luaset_output_str`.

```

1476 \cs_new_protected:Nn \__spintent_spdate_process:n
1477 {
1478   \__spintent_luafun_spdate_parse:n { #1 }
1479   \str_if_eq:VnTF \l__spintent_spdate_luaset_error_str { true }
1480   {
1481     \msg_error:nnnn { spintent } { invalid-date-format }
1482     { \spdate } { #1 }
1483   }
1484   {
1485     \MathMLintent { :date }
1486     {

```

```

1487         \text { \l__spintent_sdate_luaset_output_str }
1488     }
1489 }
1490 }

```

(End of definition for `\l__spintent_sdate_process:n`.)

`\sdate` The command `\sdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\l__spintent_sdate_process:n` to process the input.

```

1491 \NewDocumentCommand \sdate { m }
1492 {
1493     \mode_if_math:TF
1494     {
1495         \group_begin:
1496         \l__spintent_sdate_process:n { #1 }
1497         \group_end:
1498     }
1499     { \msg_error:nnn { spintent } { need-math-mode } { \sdate } }
1500 }

```

(End of definition for `\sdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *keys* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank *{argument}*, the second this is when an input is invalid.

```

1501 \msg_new:nnnn { spintent } { spusep-empty-arg }
1502 {
1503     Empty~argument~'#1'~for~\token_to_str:N \spusep.
1504 }
1505 {
1506     \token_to_str:N \spusep~requires~a~non-empty~argument.
1507 }
1508 \msg_new:nnn { spintent } { spusep-invalid-arg }
1509 {
1510     Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1511 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\l__spintent_spusep_arg_tl`
`\l__spintent_spusep_left_tl`
`\l__spintent_spusep_right_tl`

The variable `\l__spintent_spusep_arg_tl` stores the *{argument}*, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```

1512 \tl_new:N \l__spintent_spusep_arg_tl
1513 \tl_new:N \l__spintent_spusep_left_tl
1514 \tl_new:N \l__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the *keys* `size` and `silent`.

```

1515 \keys_define:nn { spintent / spusep }
1516 {
1517     size .choices:nn =
1518     { none , big , Big, bigg, Bigg }
1519     {
1520         \int_case:nn { \l_keys_choice_int }
1521         {
1522             { 1 }
1523             {
1524                 \tl_clear:N \l__spintent_spusep_left_tl
1525                 \tl_clear:N \l__spintent_spusep_right_tl
1526             }
1527         }
1528     }

```

```

1527         { 2 }
1528         {
1529             \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1530             \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1531         }
1532         { 3 }
1533         {
1534             \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1535             \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1536         }
1537         { 4 }
1538         {
1539             \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1540             \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1541         }
1542         { 5 }
1543         {
1544             \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1545             \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1546         }
1547     }
1548 },
1549 size .initial:n = none,
1550 size .value_required:n = true,
1551 size / unknown .code:n =
1552     \msg_error:nneee { spintent } { unknown-choice }
1553     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1554 silent .bool_set:N = \l__spintent_spusep_silent_bool,
1555 silent .initial:n = false,
1556 silent .value_required:n = true,
1557 unknown .code:n =
1558     {
1559         \str_set:Nn \l__spintent_command_name_str { spusep }
1560         \__spintent_unknown_keys_cmd:n {#1}
1561     },
1562 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1563 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1564 {
1565     \tl_put_right:cn { \l__spintent_spusep_ #1 _tl } { #2 }
1566     \bool_if:NTF \l__spintent_spusep_silent_bool
1567     {
1568         \MathMLint { :silent } { \tl_use:c { \l__spintent_spusep_ #1 _tl } }
1569     }
1570     {
1571         \tl_use:c { \l__spintent_spusep_ #1 _tl }
1572     }
1573 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\{\langle argument \rangle\}$ passed in ‘#1’ in the variable `\l__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1574 \cs_new_protected:Nn \__spintent_spusep_convert:n
1575 {
1576     \tl_set:Nn \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1577     \tl_if_empty:NT \l__spintent_spusep_arg_tl
1578     {
1579         \msg_error:nnn { spintent } { spusep-empty-arg } {#1}
1580     }
1581     \str_case:VnF \l__spintent_spusep_arg_tl
1582     {
1583         { ( }
1584         {

```

```

1585         \_spintent_spusep_set_parens:nn { left } { \lparen }
1586     }
1587     { ) }
1588     {
1589         \_spintent_spusep_set_parens:nn { right } { \rparen }
1590     }
1591     { [ ] }
1592     {
1593         \_spintent_spusep_set_parens:nn { left } { \lbrack }
1594     }
1595     { ] }
1596     {
1597         \_spintent_spusep_set_parens:nn { right } { \rbrack }
1598     }
1599     { \{ }
1600     {
1601         \_spintent_spusep_set_parens:nn { left } { \lbrace }
1602     }
1603     { \} }
1604     {
1605         \_spintent_spusep_set_parens:nn { right } { \rbrace }
1606     }
1607 }
1608 {
1609     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1610 }
1611 }

```

(End of definition for `\_spintent_spusep_convert:n`)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\_spintent_spusep_convert:n`.

```

1612 \NewDocumentCommand \spusep { O{} m }
1613 {
1614     \mode_if_math:TF
1615     {
1616         \group_begin:
1617         \keys_set:nn { spintent / spusep } {#1}
1618         \_spintent_spusep_convert:n {#2}
1619         \group_end:
1620     }
1621     { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1622 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

The “error” `spnewread-empty-args` is raised when `\spnewread` receives an empty or blank symbol or reading. The “error” `spnewread-duplicate` is raised when it registers a symbol already in the dictionary. The “error” `spread-unknown-sym` is raised when `\spread` is given a symbol that is neither registered nor accompanied by `read-arg`.

```

1623 \msg_new:nnnn { spintent } { spnewread-empty-args }
1624 {
1625     Empty~argument~in~\token_to_str:N \spnewread.
1626 }
1627 {
1628     \token_to_str:N \spnewread~requires~two~non-empty~arguments.
1629 }
1630 \msg_new:nnnn { spintent } { spnewread-duplicate }
1631 {
1632     The~symbol~'~#1'~is~already~defined!
1633 }
1634 {
1635     You~cannot~redefine~with~\token_to_str:N \spnewread.
1636 }
1637 \msg_new:nnnn { spintent } { spread-empty-sym }
1638 {
1639     Empty~symbol~argument~in~\token_to_str:N \spread.

```

```

1640 }
1641 {
1642   \token_to_str:N \spread~requires~a~non-empty~symbol.
1643 }
1644 \msg_new:nnnn { spintent } { spread-unknown-sym }
1645 {
1646   The~symbol~'#1'~is~not~registered~and~no~'read-arg'~was~given.
1647 }
1648 {
1649   Register~it~first~with~\token_to_str:N \spnewread,~or~pass~'read-arg'.
1650 }

```

(End of definition for `spnewread-empty-args`, `spnewread-duplicate`, and `spread-unknown-sym`.)

`\__spintent_spread_luaset_status_str` and `\__spintent_spread_luaset_reading_str` are set by `\__spintent_luafun_spread_new:nn/\__spintent_luafun_spread_lookup:n`. `\__spintent_spread_read_arg_tl` stores `read-arg`.

```

1651 \str_new:N \__spintent_spread_luaset_status_str
1652 \str_new:N \__spintent_spread_luaset_reading_str
1653 \tl_new:N \__spintent_spread_read_arg_tl

```

(End of definition for `\__spintent_spread_luaset_status_str`, `\__spintent_spread_luaset_reading_str`, and `\__spintent_spread_read_arg_tl`.)

`read-arg` We define the `<keys>` `read-arg` and `silent`.

```

silent
unknown
1654 \keys_define:nn { spintent / spread }
1655 {
1656   read-arg .code:n =
1657     { \__spintent_sanitize_affix:Nn \__spintent_spread_read_arg_tl {#1} },
1658   read-arg .initial:n = ,
1659   read-arg .value_required:n = true,
1660   silent .bool_set:N = \__spintent_spread_silent_bool,
1661   silent .initial:n = false,
1662   silent .value_required:n = true,
1663   unknown .code:n =
1664     {
1665       \str_set:Nn \__spintent_command_name_str { spread }
1666       \__spintent_unknown_keys_cmd:n {#1}
1667     },
1668 }

```

(End of definition for `read-arg`, `silent`, and `unknown`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` receives the optional `<keys>` in ‘#1’ and the `{<symbol>}` in ‘#2’. With `silent true`, the symbol is typeset with a `:silent` intent, nothing else evaluated. Otherwise, with `read-arg` given, it is used directly, overriding any registered reading for this call alone. With `read-arg` empty, calls `\__spintent_luafun_spread_lookup:n`, using the registered reading when found, reporting `spread-unknown-sym` otherwise.

```

1669 \cs_new_protected:Nn \__spintent_spread_exec:nn
1670 {
1671   \keys_set:nn { spintent / spread } { #1 }
1672   \tl_if_blank:nTF { #2 }
1673     { \msg_error:nn { spintent } { spread-empty-sym } }
1674     {
1675       \__spintent_invisible_sep:
1676       \bool_if:NTF \__spintent_spread_silent_bool
1677         {
1678           \MathMLintent { :silent } { #2 }
1679         }
1680         {
1681           \tl_if_empty:NTF \__spintent_spread_read_arg_tl
1682             {
1683               \__spintent_luafun_spread_lookup:n { \exp_not:n {#2} }
1684               \str_if_eq:VnTF \__spintent_spread_luaset_status_str { found }
1685               {
1686                 \MathMLintent { \__spintent_spread_luaset_reading_str } { #2 }
1687               }
1688               {
1689                 \msg_error:nne { spintent } { spread-unknown-sym } { \exp_not:n {#2} }
1690               }
1691             }

```

```

1692         {
1693             \MathMLintent { \l__spintent_spread_read_arg_tl } { #2 }
1694         }
1695     }
1696 }
1697 }

```

(End of definition for `\l__spintent_spread_exec:nn`.)

\spread The public command `\spread` enforces *math mode*, opens a local group, and delegates execution to `\l__spintent_spread_exec:nn`.

```

1698 \NewDocumentCommand \spread { O{} m }
1699 {
1700     \mode_if_math:TF
1701     {
1702         \group_begin:
1703         \l__spintent_spread_exec:nn { #1 } { #2 }
1704         \group_end:
1705     }
1706     { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1707 }

```

(End of definition for `\spread`. This function is documented on page 11.)

\spnewread The public command `\spnewread` registers a new symbol/reading pair. It passes the symbol ‘`#1`’ and its reading ‘`#2`’ to Lua via `\l__spintent_luafun_spread_new:nn`, reporting `spnewread-duplicate` when the symbol is already registered.

```

1708 \NewDocumentCommand \spnewread { m m }
1709 {
1710     \mode_if_math:TF
1711     {
1712         \msg_error:nnn { spintent } { need-text-mode } { \spnewread }
1713     }
1714     {
1715         \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1716         { \msg_error:nn { spintent } { spnewread-empty-args } }
1717         {
1718             \l__spintent_luafun_spread_new:nn { \exp_not:n { #1 } } { #2 }
1719             \str_if_eq:VnT \l__spintent_spread_luasets_status_str { duplicate }
1720             {
1721                 \msg_error:nnn { spintent } { spnewread-duplicate } { #1 }
1722             }
1723         }
1724     }
1725 }

```

(End of definition for `\spnewread`. This function is documented on page 11.)

11.23.2 The command `\spdsym`

`\l__spintent_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
1726 \tl_new:N \l__spintent_spdsym_read_sym_tl
1727 \tl_new:N \l__spintent_spdsym_symbol_tl
1728 \tl_new:N \l__spintent_spdsym_prefix_tl
1729 \tl_new:N \l__spintent_spdsym_suffix_tl

```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

prefix We define the *(keys)* `prefix`, `suffix`, `set-sym` and `read-sym` for `\spdsym`.

```

suffix
1730 \keys_define:nn { spintent / spdsym }
set-sym
1731 {
read-sym
1732     prefix .code:n =
1733     { \l__spintent_sanitize_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1734     prefix .initial:n = ,
1735     prefix .value_required:n = true,
1736     suffix .code:n =
1737     { \l__spintent_sanitize_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1738     suffix .initial:n = ,
1739     suffix .value_required:n = true,
1740     set-sym .choices:nn =
1741     { old-style , two-dots , slash }
1742     {

```

```

1743     \int_case:nn { \l_keys_choice_int }
1744     {
1745         {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1746         {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1747         {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1748     }
1749 },
1750 set-sym .initial:n = two-dots,
1751 set-sym .value_required:n = true,
1752 set-sym / unknown .code:n =
1753     \msg_error:nneee { spintent } { unknown-choice }
1754     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1755 read-sym .choices:nn =
1756     { dividido , dividido-por , dividido-entre }
1757     {
1758         \int_case:nn { \l_keys_choice_int }
1759         {
1760             {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1761             {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1762             {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1763         }
1764     },
1765 read-sym .initial:n = dividido,
1766 read-sym .value_required:n = true,
1767 read-sym / unknown .code:n =
1768     \msg_error:nneee { spintent } { unknown-choice }
1769     { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1770     { \exp_not:n {#1} },
1771 unknown .code:n =
1772     {
1773         \str_set:Nn \l__spintent_command_name_str { spdsym }
1774         \__spintent_unknown_keys_cmd:n {#1}
1775     },
1776 }

```

(End of definition for prefix and others.)

\spdsym Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1777 \NewDocumentCommand \spdsym { 0{} }
1778 {
1779     \mode_if_math:TF
1780     {
1781         \group_begin:
1782         \keys_set:nn { spintent / spdsym } { #1 }
1783         \MathMLintent
1784         {
1785             \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1786             { \l__spintent_spdsym_prefix_tl - }
1787             \l__spintent_spdsym_read_sym_tl
1788             \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1789             { - \l__spintent_spdsym_suffix_tl }
1790         }
1791         { \l__spintent_spdsym_symbol_tl }
1792         \group_end:
1793     }
1794     { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1795 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1796 \tl_new:N \l__spintent_spmsym_read_sym_tl
1797 \tl_new:N \l__spintent_spmsym_symbol_tl
1798 \tl_new:N \l__spintent_spmsym_prefix_tl
1799 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

```

prefix We define the  $\langle keys \rangle$  prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym
read-sym
not-print
unknown
1800 \keys_define:nn { spintent / spmsym }
1801 {
1802   prefix .code:n =
1803   { \__spintent_sanitizé_affix:Nn \__spintent_spmsym_prefix_tl {#1} },
1804   prefix .initial:n = ,
1805   prefix .value_required:n = true,
1806   suffix .code:n =
1807   { \__spintent_sanitizé_affix:Nn \__spintent_spmsym_suffix_tl {#1} },
1808   suffix .initial:n = ,
1809   suffix .value_required:n = true,
1810   set-sym .choices:nn =
1811   { cross , dot , asterisk }
1812   {
1813     \int_case:nn { \l_keys_choice_int }
1814     {
1815       {1} { \tl_set:Nn \__spintent_spmsym_symbol_tl { \times } }
1816       {2} { \tl_set:Nn \__spintent_spmsym_symbol_tl { \cdot } }
1817       {3} { \tl_set:Nn \__spintent_spmsym_symbol_tl { \ast } }
1818     }
1819   },
1820   set-sym .initial:n = dot,
1821   set-sym .value_required:n = true,
1822   set-sym / unknown .code:n =
1823   \msg_error:nneee { spintent } { unknown-choice }
1824   { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1825   read-sym .choices:nn =
1826   { por , multiplicado-por , veces }
1827   {
1828     \int_case:nn { \l_keys_choice_int }
1829     {
1830       {1} { \tl_set:Nn \__spintent_spmsym_read_sym_tl { por } }
1831       {2} { \tl_set:Nn \__spintent_spmsym_read_sym_tl { multiplicado-por } }
1832       {3} { \tl_set:Nn \__spintent_spmsym_read_sym_tl { veces } }
1833     }
1834   },
1835   read-sym .initial:n = por,
1836   read-sym .value_required:n = true,
1837   read-sym / unknown .code:n =
1838   \msg_error:nneee { spintent } { unknown-choice }
1839   { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1840   not-print .bool_set:N = \__spintent_spmsym_not_print_bool,
1841   not-print .initial:n = false,
1842   not-print .value_required:n = true,
1843   unknown .code:n =
1844   {
1845     \str_set:Nn \__spintent_command_name_str { spmsym }
1846     \__spintent_unknown_keys_cmd:n {#1}
1847   },
1848 }

```

(End of definition for prefix and others.)

$\backslash\text{spmsym}$ Defines the $\backslash\text{spmsym}$ operator. Similar to $\backslash\text{spdsym}$, it outputs the correct mathematical glyph based on the local $\langle keys \rangle$, safely attaching prefixes and suffixes if present.

```

1849 \NewDocumentCommand \spmsym { 0{} }
1850 {
1851   \mode_if_math:TF
1852   {
1853     \group_begin:
1854     \keys_set:nn { spintent / spmsym } { #1 }
1855     \MathMLintent
1856     {
1857       \tl_if_empty:NF \__spintent_spmsym_prefix_tl
1858       { \__spintent_spmsym_prefix_tl - }
1859       \__spintent_spmsym_read_sym_tl
1860       \tl_if_empty:NF \__spintent_spmsym_suffix_tl
1861       { - \__spintent_spmsym_suffix_tl }
1862     }
1863   }
1864   \bool_if:NTF \__spintent_spmsym_not_print_bool

```

```

1865         {
1866             \invisibletimes
1867         }
1868         { \l__spintent_spmsym_symbol_tl }
1869     }
1870     \group_end:
1871 }
1872 { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1873 }

```

(End of definition for `\spmsym`. This function is documented on page 13.)

11.24 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the $\langle\textit{dividend}\rangle$ and the $\langle\textit{divisor}\rangle$. Both $\langle\textit{arguments}\rangle$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\langle\textit{dividend}\rangle$ nor the $\langle\textit{divisor}\rangle$ is *blank*.

11.24.1 Definition of keys for `\spdiv`

We define the $\langle\textit{keys}\rangle$ `wrap-parens` and `read-parens` for `\spdiv` command. Number-related $\langle\textit{keys}\rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle\textit{keys}\rangle$ `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

cifras-sep
read-sign
read-period-sep
notation-sym
set-sym
read-sym
wrap-parens
read-parens
1874 \keys_define:nn { spintent / spdiv }
1875 {
1876     cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1877     read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1878     read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1879     notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1880     set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1881     read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1882     wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1883     wrap-parens     .initial:n  = false,
1884     wrap-parens     .value_required:n = true,
1885     read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1886     read-parens     .initial:n  = true,
1887     read-parens     .value_required:n = true,
1888     unknown         .code:n =
1889     {
1890         \str_set:Nn \l__spintent_command_name_str { spdiv }
1891         \__spintent_unknown_keys_cmd:n {#1}
1892     },
1893 }

```

(End of definition for `cifras-sep` and others.)

11.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\langle\textit{dividend}\rangle$ or the $\langle\textit{divisor}\rangle$ is missing or *blank*.

```

1894 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1895 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1896 {
1897     You~must~provide~both~a~dividend~and~a~divisor.\l
1898 }

```

(End of definition for `spdiv-empty-argument`.)

11.24.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1899 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1900 {
1901     \spnum {#1}
1902     \MathMLintent { \l__spintent_spdsym_read_sym_tl }
1903     {
1904         \l__spintent_spdsym_symbol_tl
1905     }
1906     \spnum {#2}
1907 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn`

The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1908 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1909 {
1910   \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1911   {
1912     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1913   }
1914   {
1915     \MathMLintent { :silent } { ( }
1916     \__spintent_spdiv_render_op:nn {#1} {#2}
1917     \MathMLintent { :silent } { ) }
1918   }
1919 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn`

The function `\__spintent_spdiv_process:nnn` sets `<keys>`, validates `{<arguments>}`, and decides on parenthesis wrapping.

```

1920 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1921 {
1922   \keys_set:nn { spintent / spdiv } { #1 }
1923   \tl_if_blank:NTF { #2 }
1924   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1925   {
1926     \tl_if_blank:NTF { #3 }
1927     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1928     {
1929       \bool_if:NTF \l__spintent_spdiv_wrap_parens_bool
1930       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1931       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1932     }
1933   }
1934 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv`

Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1935 \NewDocumentCommand \spdiv { O{} m m }
1936 {
1937   \mode_if_math:TF
1938   {
1939     \group_begin:
1940     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1941     \group_end:
1942   }
1943   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1944 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{<first factor>}` and the `{<second factor>}`. Both `{<arguments>}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{<first factor>}` nor the `{<second factor>}` is *blank*.

11.25.1 Definition of keys for `\spmul`

`cifras-sep`
`read-sign`
`read-period-sep`
`notation-sym`

We define the `<keys>` `wrap-parens` and `read-parens` for `\spmul` command. Number-related `<keys>` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `<keys>` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1945 \keys_define:nn { spintent / spmul }
1946 {
1947   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1948   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1949   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1950   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1951   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1952   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1953   wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,

```

```

1954     wrap-parens      .initial:n = false,
1955     wrap-parens      .value_required:n = true,
1956     read-parens      .bool_set:N = \l__spintent_spmul_read_parens_bool,
1957     read-parens      .initial:n = true,
1958     read-parens      .value_required:n = true,
1959     unknown          .code:n =
1960     {
1961         \str_set:Nn \l__spintent_command_name_str { spmul }
1962         \__spintent_unknown_keys_cmd:n {#1}
1963     },
1964 }

```

(End of definition for *cifras-sep* and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the $\langle\textit{first factor}\rangle$ or the $\langle\textit{second factor}\rangle$ is missing or *blank*.

```

1965 \msg_new:nnnn { spintent } { spmul-empty-argument }
1966 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1967 {
1968     You~must~provide~both~first~factor~and~second~factor.\\
1969 }

```

(End of definition for *spmul-empty-argument*.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1970 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1971 {
1972     \spnum {#1}
1973     \MathMLintent { \l__spintent_spmsym_read_sym_tl }
1974     {
1975         \l__spintent_spmsym_symbol_tl
1976     }
1977     \spnum {#2}
1978 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1979 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1980 {
1981     \bool_if:NTF \l__spintent_spmul_read_parens_bool
1982     {
1983         ( \__spintent_spmul_render_op:nn {#1} {#2} )
1984     }
1985     {
1986         \MathMLintent { :silent } { ( }
1987         \__spintent_spmul_render_op:nn {#1} {#2}
1988         \MathMLintent { :silent } { ) }
1989     }
1990 }

```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn` The function `\__spintent_spmul_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1991 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1992 {
1993     \keys_set:nn { spintent / spmul } { #1 }
1994     \tl_if_blank:NTF { #2 }
1995     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1996     {
1997         \tl_if_blank:NTF { #3 }
1998         { \msg_error:nnn { spintent } { spmul-empty-argument } }
1999         {
2000             \bool_if:NTF \l__spintent_spmul_wrap_parens_bool
2001             { \__spintent_spmul_render_parens:nn {#2} {#3} }
2002             { \__spintent_spmul_render_op:nn {#2} {#3} }
2003         }
2004     }
2005 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```
2006 \NewDocumentCommand \spmul { 0{} m m }
2007 {
2008   \mode_if_math:TF
2009   {
2010     \group_begin:
2011     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
2012     \group_end:
2013   }
2014   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
2015 }
```

(End of definition for `\spmul`. This function is documented on page 13.)

11.26 The commands `\spmcd` and `\spmcm`

The user-level commands `\spmcd` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for `\spmcd` and `\spmcm`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\__spintent_spmcm_spmcd_luaset_error_str
\__spintent_mc_luaset_types_str
\__spintent_mc_luaset_all_numeric_str
\__spintent_mc_args_seq
\__spintent_mc_raw_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl
```

```
2016 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
2017 \clist_new:N \__spintent_mc_luaset_types_clist
2018 \str_new:N \__spintent_mc_luaset_all_numeric_str
2019 \seq_new:N \__spintent_mc_args_seq
2020 \seq_new:N \__spintent_mc_raw_args_seq
2021 \seq_new:N \__spintent_mc_types_seq
2022 \int_new:N \__spintent_mc_idx_int
2023 \clist_map_inline:nn { mcd , mcm }
2024 {
2025   \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2026   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
2027   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2028 }
```

(End of definition for `\__spintent_spmcm_spmcd_luaset_error_str` and others.)

11.26.2 Definition of error messages

`mc-invalid-arguments`

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
2029 \msg_new:nnnn { spintent } { mc-invalid-arguments }
2030 {
2031   Invalid~arguments~provided~to~#1.
2032 }
2033 {
2034   The~arguments~must~be~a~comma-separated~list. \l
2035 }
```

(End of definition for `mc-invalid-arguments`.)

11.26.3 Keys for `\spmcd` and `\spmcm`

We define the shared *keys* `result`, `prefix`, `read-cmd`, `upper-case`, `concept` and `silent`.

```
result
prefix
read-cmd
upper-case
concept
silent
unknown
```

```
2036 \clist_map_inline:nn { mcd , mcm }
2037 {
2038   \keys_define:nn { spintent / sp #1 }
2039   {
2040     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2041     result      .initial:n = false,
2042     result      .value_required:n = true,
2043     prefix      .code:n =
2044       { \__spintent_sanitizate_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2045     prefix      .initial:n = ,
2046     prefix      .value_required:n = true,
2047     read-cmd    .choices:nn =
2048       { terse , clear }
2049     {
2050       \int_case:nn { \l_keys_choice_int }
2051       {
```

```

2052         {1} { \bool_set_true:c { l__spintent_sp #1 _read_terse_bool } }
2053         {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
2054     }
2055 },
2056 read-cmd .initial:n = clear,
2057 read-cmd .value_required:n = true,
2058 read-cmd / unknown .code:n =
2059     \msg_error:nneee { spintent } { unknown-choice }
2060     { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
2061 upper-case .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
2062 upper-case .initial:n = false,
2063 upper-case .value_required:n = true,
2064 concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
2065 concept .initial:n = true,
2066 concept .value_required:n = true,
2067 silent .bool_set:c = { l__spintent_sp #1 _silent_bool },
2068 silent .initial:n = false,
2069 silent .value_required:n = true,
2070 unknown .code:n =
2071     {
2072         \str_set:Nn \l__spintent_command_name_str { sp #1 }
2073         \__spintent_unknown_keys_cmd:n {##1}
2074     },
2075 }
2076 }

```

(End of definition for `resultt` and others.)

11.26.4 Internal functions for `\spmc` and `\spmcd`

`\__spintent_mc_process_args:nn`

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintent_mc_raw_args_seq` and `\l__spintent_mc_luaset_types_clist` into `\l__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintent_mc_args_seq`: each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2077 \cs_new_protected:Nn \__spintent_mc_process_args:nn
2078 {
2079     \__spintent_luafun_mc_classify_args:n {#2}
2080     \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#2}
2081     \seq_set_from_clist:NN \l__spintent_mc_types_seq \l__spintent_mc_luaset_types_clist
2082     \seq_clear:N \l__spintent_mc_args_seq
2083     \int_zero:N \l__spintent_mc_idx_int
2084     \seq_map_inline:Nn \l__spintent_mc_raw_args_seq
2085     {
2086         \int_incr:N \l__spintent_mc_idx_int
2087         \str_if_eq:eeTF
2088         { \seq_item:Nn \l__spintent_mc_types_seq { \l__spintent_mc_idx_int } } { number }
2089         {
2090             \seq_put_right:Nn \l__spintent_mc_args_seq { \spnum {##1} }
2091         }
2092         { \seq_put_right:Nn \l__spintent_mc_args_seq {##1} }
2093     }

```

Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2094     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2095     {
2096         \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
2097         {
2098             \MathMLintent { :silent } { ##1 }
2099         }
2100         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2101         \MathMLintent { :silent } { ( }
2102         \seq_use:Nnnn \l__spintent_mc_args_seq
2103         { \MathMLintent { :silent } { , } }
2104         { \MathMLintent { :silent } { , } }
2105         { \MathMLintent { :silent } { , } }
2106     }
2107     {
2108         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2109         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2110         {

```

```

2111         \MathMLintent { _entre } { ( }
2112     }
2113     {
2114         \MathMLintent { :silent } { ( }
2115     }
2116     \seq_use:Nnnn \l__spintenc_mc_args_seq
2117     {
2118         \MathMLintent { :silent } { , }
2119         \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" medio
2120     }
2121     {
2122         \MathMLintent { :silent } { , }
2123     }
2124     {
2125         \MathMLintent { :silent } { , }
2126         \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" final
2127     }
2128     }
2129     \MathMLintent { :silent } { ) }
2130 }

```

(End of definition for `\__spintenc_mc_process_args:nn`.)

`\__spintenc_spmc_process:nnnn`

The internal function `\__spintenc_spmc_process:nnnn` is the shared handler for both `\spmc` and `\spmc`. Takes the short identifier in `#1`, its full reading string in `#2`, the local `(keys)` in `#3`, and the optional arguments in `#4`.

The operator name is silenced with `silent=true` — `\__spintenc_mc_process_args:nn` silences the arguments too, in that case — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal `intent` instead — `#1` or `#2` per `read-cmd`, prefixed with `prefix` when given.

With `#4` given, classifies it through `\__spintenc_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure, and, when `result` is `true`, printing the computed value. `\__spintenc_mc_process_args:n` prints the argument list itself either way.

- We should use `\mathrm` instead of `\operatorname`, which generates unwanted spacing when `\c__spintenc_invisible_sep_int` is applied.

```

2131 \cs_new_protected:Nn \__spintenc_spmc_process:nnnn
2132 {
2133     \keys_set:nn { spintenc / sp #1 } { #3 }
2134     \bool_if:cTF { l__spintenc_sp #1 _silent_bool }
2135     {
2136         \MathMLintent { :silent }
2137         {
2138             \mathrm
2139             {
2140                 \bool_if:cTF { l__spintenc_sp #1 _print_upper_bool }
2141                 { \text_uppercase:n {#1} } { #1 }
2142             }
2143         }
2144     }
2145     {
2146         \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2147         {
2148             \MathMLintent
2149             {
2150                 \tl_if_empty:cF { l__spintenc_sp #1 _prefix_tl }
2151                 {
2152                     \tl_use:c { l__spintenc_sp #1 _prefix_tl } -
2153                 }
2154                 \bool_if:cTF { l__spintenc_sp #1 _read_terse_bool }
2155                 { #1 } { #2 }
2156             }
2157             {
2158                 \mathrm
2159                 {
2160                     \bool_if:cTF { l__spintenc_sp #1 _print_upper_bool }
2161                     { \text_uppercase:n {#1} } { #1 }
2162                 }
2163             }
2164         }
2165     }
2166 }

```

```

2165         {
2166             \MathMLintent { :silent }
2167             {
2168                 \mathrm
2169                 {
2170                     \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2171                     { \text_uppercase:n {#1} } { #1 }
2172                 }
2173             }
2174         }
2175     }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2176     \tl_if_novalue:nF { #4 }
2177     {
2178         \seq_set_from_clist:Nn \__spintent_mc_raw_args_seq {#4}
2179         \int_compare:nNnTF { \seq_count:N \__spintent_mc_raw_args_seq } < { 2 }
2180         {
2181             \msg_error:nne { spintent } { mc-too-few-arguments } { \exp_not:c { sp #1 } }
2182         }
2183         {
2184             \__spintent_luafun_mc_classify_args:n {#4}
2185             \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2186             {
2187                 \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2188                 \str_if_eq:VnT \__spintent_spmcm_spmcd_luaset_error_str { true }
2189                 {
2190                     \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2191                 }
2192             }
2193             \__spintent_mc_process_args:nn {#1} {#4}
2194             \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2195             {
2196                 \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2197                 {
2198                     \MathMLintent { _es-igual-a } { = }
2199                     \tl_use:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2200                 }
2201             }
2202         }
2203     }
2204 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmc` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2205 \NewDocumentCommand \spmc { 0{} d() }
2206 {
2207     \mode_if_math:TF
2208     {
2209         \group_begin:
2210         \__spintent_spmc_process:nnnn { mcd } { _máximo-común-divisor } {#1} {#2}
2211         \group_end:
2212     }
2213     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2214 }
2215 \NewDocumentCommand \spmc { 0{} d() }
2216 {
2217     \mode_if_math:TF
2218     {
2219         \group_begin:
2220         \__spintent_spmc_process:nnnn { mcm } { _mínimo-común-múltiplo } {#1} {#2}
2221         \group_end:
2222     }

```

```

2223     { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2224   }

```

(End of definition for `\spmcd` and `\spmcm`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Definition of variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spmmd_luaset_is_number_str
\__spintent_spm_spnD_luaset_error_str
\__spintent_spm_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spm_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spm_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spm_print_result_num_tl
\__spintent_spnD_print_result_num_tl
2225 \seq_new:N \__spintent_md_out_seq
2226 \str_new:N \__spintent_spm_spnD_luaset_error_str
2227 \str_new:N \__spintent_spmmd_luaset_is_number_str
2228 \clist_map_inline:nn { nM , nD }
2229 {
2230   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2231   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2232   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2233 }

```

(End of definition for `\__spintent_md_out_seq` and others.)

11.27.2 Keys for `\spnM` and `\spnD`

We define the shared `\keys` result, silent, result-num, prefix and concept.

```

result
silent
result-num
prefix
concept
unknown
2234 \clist_map_inline:nn { nM , nD }
2235 {
2236   \keys_define:nn { spintent / sp #1 }
2237   {
2238     result .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2239     result .initial:n = false,
2240     result .value_required:n = true,
2241     silent .bool_set:c = { \__spintent_sp #1 _silent_bool },
2242     silent .initial:n = false,
2243     silent .value_required:n = true,
2244     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2245     result-num .initial:n = ,
2246     prefix .code:n =
2247       { \__spintent_sanitizé_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2248     prefix .initial:n = ,
2249     prefix .value_required:n = true,
2250     concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
2251     concept .initial:n = true,
2252     concept .value_required:n = true,
2253     unknown .code:n =
2254       {
2255         \str_set:Nn \__spintent_command_name_str { sp #1 }
2256         \__spintent_unknown_keys_cmd:n {##1}
2257       },
2258   }
2259 }

```

(End of definition for `result` and others.)

11.27.3 Definition of error messages for `\spnM` and `\spnD`

Exception triggered when the argument for multiples or divisors is invalid.

```

md-invalid-argument
mc-too-few-arguments
2260 \msg_new:nnnn { spintent } { md-invalid-argument }
2261 {
2262   Invalid~argument~provided~to~#1~\msg_line_context:.
2263 }
2264 {
2265   The~argument~must~be~a~single~positive~integer. \l
2266 }

```

Exception triggered when fewer than two arguments are provided.

```

2267 \msg_new:nnnn { spintent } { mc-too-few-arguments }
2268 {
2269   Too~few~arguments~provided~to~#1~\msg_line_context:.
2270 }
2271 {
2272   At~least~two~arguments~are~required. \l
2273 }

```

(End of definition for `md-invalid-argument` and `mc-too-few-arguments`.)

11.27.4 Internal functions for `\spnM` and `\spnD`

`\__spintent_spnM_spnD_process:nnnn`

The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spn_M_and_D_exec:nnnn`.

```
2274 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnn
2275 {
2276   \keys_set:nn { spintent / sp #1 } { #4 }
2277   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2278 }
```

(End of definition for `\__spintent_spnM_spnD_process:nnnn`.)

`\__spintent_spnM_spnD_intent:nn`

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends ‘-de’.

```
2279 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2280 {
2281   \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2282   { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
2283   \str_case:nnF { #2 } { { múltiplos } { _múltiplos } }
2284   { #2 }
2285   -de
2286 }
```

(End of definition for `\__spintent_spnM_spnD_intent:nn`.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spn_M_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules.

```
2287 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2288 {
2289   \MathMLintent { :silent } { \operatorname { #1 } }
2290   \MathMLintent { :silent } { ( }
2291   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2292   \MathMLintent { :silent } { #2 }
2293   \MathMLintent { :silent } { ) }
2294 }
```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:nn`

The internal function `\__spintent_spn_M_and_D_result:nn` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’ and the classified numeric argument ‘#2’. It invokes the corresponding Lua backend calculation, reporting `md-invalid-argument` if that fails — a safety net beyond `\__spintent_luafun_spnmd_classify:n`’s own check, since the Lua calculation never trusted that check blindly. On success, it maps the resulting clist into a sequence, and formats the final printed output utilizing custom intents for both separators and conjunctions.

```
2295 \cs_new_protected:Nn \__spintent_spnM_spnD_result:nn
2296 {
2297   \use:c { __spintent_luafun_calcular_ #1 :nn } { #2 }
2298   {
2299     \tl_use:c { l__spintent_sp #1 _print_result_num_tl }
2300   }
2301   \str_if_eq:VnTF \__spintent_spnM_spnD_luaset_error_str { true }
2302   {
2303     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2304   }
2305   {
2306     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2307     \MathMLintent { _son } { = \{ }
2308     \seq_set_from_clist:Ne
2309       \l__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luaset_result_tl } }
2310     \seq_use:Nnnn \l__spintent_md_out_seq
2311       { \MathMLintent { _y- } { , } }
2312       { \MathMLintent { :silent } { , } }
2313   }
```

```

2314         \MathMLintent { :silent } { , }
2315         \str_if_eq:vnT { l__spinttent_sp #1 _luaset_is_truncated_str } { false }
2316         {
2317             \MathMLintent { _y- } { \c__spinttent_invisible_sep_int } % "y" final
2318         }
2319     }
2320     \str_if_eq:vnT { l__spinttent_sp #1 _luaset_is_truncated_str } { true }
2321     {
2322         \MathMLintent { :silent } { , } ~ \dots
2323     }
2324     \MathMLintent { :silent } { \} }
2325 }
2326 }

```

(End of definition for `\__spinttent_spnM_spnD_result:nn`.)

`\__spinttent_spnM_spnD_exec:nnnn`

The function `\__spinttent_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `\__spinttent_luafun_spnmd_classify` decides whether it is wrapped through `\spnum` — and, when it is, whether `\__spinttent_spnM_spnD_result:nn` also prints the computed set, per `result` — or left verbatim, with no result attempted.

Either way, `silent` silences the whole thing through `\__spinttent_spnM_spnD_silent:nn`; otherwise `concept=true` carries the literal intent from `\__spinttent_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2327 \cs_new_protected:Nn \__spinttent_spnM_spnD_exec:nnnn
2328 {
2329     \tl_if_novalue:nTF { #4 }
2330     {
2331         \bool_if:cTF { l__spinttent_sp #1 _silent_bool }
2332         { \__spinttent_spnM_spnD_silent:nn { #2 } { n } }
2333         {
2334             \bool_if:cTF { l__spinttent_sp #1 _concept_bool }
2335             {
2336                 \MathMLintent
2337                 { \__spinttent_spnM_spnD_intent:nn { #1 } { #3 } }
2338                 { \operatorname { #2 } }
2339             }
2340             { \MathMLintent { :silent } { \operatorname { #2 } } }
2341             \MathMLintent { :silent } { ( }
2342             \MathMLintent { :silent } { \c__spinttent_invisible_sep_int } % prevent doble "de"
2343             n
2344             \MathMLintent { :silent } { ) }
2345         }
2346     }
2347     {
2348         \__spinttent_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2349         \bool_if:cTF { l__spinttent_sp #1 _silent_bool }
2350         {
2351             \str_if_eq:VnTF \__spinttent_spnmd_luaset_is_number_str { true }
2352             { \__spinttent_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2353             { \__spinttent_spnM_spnD_silent:nn { #2 } { #4 } }
2354         }
2355         {
2356             \bool_if:cTF { l__spinttent_sp #1 _concept_bool }
2357             {
2358                 \MathMLintent
2359                 { \__spinttent_spnM_spnD_intent:nn { #1 } { #3 } }
2360                 { \operatorname { #2 } }
2361             }
2362             { \MathMLintent { :silent } { \operatorname { #2 } } }
2363             \MathMLintent { :silent } { ( }
2364             \str_if_eq:VnTF \__spinttent_spnmd_luaset_is_number_str { true }
2365             { \spnum { #4 } } { #4 }
2366             \MathMLintent { :silent } { ) }
2367         }
2368         \str_if_eq:VnT \__spinttent_spnmd_luaset_is_number_str { true }
2369         {
2370             \bool_if:cT { l__spinttent_sp #1 _print_result_bool }
2371             {
2372                 \__spinttent_spnM_spnD_result:nn { #1 } { #4 }

```

```

2373         }
2374     }
2375 }
2376 }

```

(End of definition for `\_spintent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\_spintent_spnM_spnD_process:nnnn`, passing along the optional *(keys)* ‘#1’ and the numeric *{(argument)}* ‘#2’.

```

2377 \NewDocumentCommand \spnM { O{} d() }
2378 {
2379     \mode_if_math:TF
2380     {
2381         \group_begin:
2382         \_spintent_spnM_spnD_process:nnnn { nM } { M } { multiplos } {#1} {#2}
2383         \group_end:
2384     }
2385     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2386 }
2387 \NewDocumentCommand \spnD { O{} d() }
2388 {
2389     \mode_if_math:TF
2390     {
2391         \group_begin:
2392         \_spintent_spnM_spnD_process:nnnn { nD } { D } { _divisores } {#1} {#2}
2393         \group_end:
2394     }
2395     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2396 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per *use-dfrac*), read natively — no Lua parsing involved.

11.28.1 Definition of keys for `\sprazon`

`silent` We define the *(keys)* `silent`, `use-dfrac` and `inline`.

`use-dfrac`

`inline`

`unknown`

```

2397 \keys_define:nn { spintent / sprazon }
2398 {
2399     silent .bool_set:N = \l__spintent_sprazon_silent_bool,
2400     silent .initial:n = false,
2401     silent .value_required:n = true,
2402     use-dfrac .bool_set:N = \l__spintent_sprazon_use_dfrac_bool,
2403     use-dfrac .initial:n = false,
2404     use-dfrac .value_required:n = true,
2405     inline .bool_set:N = \l__spintent_sprazon_inline_bool,
2406     inline .initial:n = false,
2407     inline .value_required:n = true,
2408     unknown .code:n =
2409     {
2410         \str_set:Nn \l__spintent_command_name_str { sprazon }
2411         \__spintent_unknown_keys_cmd:n {#1}
2412     },
2413 }

```

(End of definition for `silent` and others.)

`\_spintent_sprazon_exec:nnn` The function `\_spintent_sprazon_exec:nnn` receives the optional *(keys)* in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under `silent`, the whole thing is silenced as one unit — `\frac/\dfrac` (per *use-dfrac*) normally, or `#2 : #3` under `inline`.

Otherwise, with `inline=false` (the default), numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside `\frac/\dfrac`, and the outer `\MathMLintent` reads `_es-a:infix($num,$denom)`. With `inline=true`, the same two references are tagged side by side instead — no `\frac/\dfrac` involved — with the `:` between them carrying the `_es-a:infix($num,$denom)` intent on its own.

```

2414 \cs_new_protected:Nn \_spintent_sprazon_exec:nnn
2415 {
2416     \keys_set:nn { spintent / sprazon } { #1 }

```

```

2417 \__spintent_invisible_sep:
2418 \bool_if:NTF \l__spintent_sprazon_silent_bool
2419 {
2420   \MathMLintent { _- }
2421   {
2422     \__spintent_inner_sep:
2423     \bool_if:NTF \l__spintent_sprazon_inline_bool
2424     { #2 : #3 }
2425     {
2426       \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2427       { \dfrac { #2 } { #3 } }
2428       { \frac { #2 } { #3 } }
2429     }
2430   }
2431 }
2432 {
2433   \bool_if:NTF \l__spintent_sprazon_inline_bool
2434   {
2435     \MathMLintent { _es-a:infix($num,$denom) }
2436     {
2437       \MathMLarg { num } { #2 }
2438       :
2439       \MathMLarg { denom } { #3 }
2440     }
2441   }
2442   {
2443     \MathMLintent { _es-a:infix($num,$denom) }
2444     {
2445       \__spintent_inner_sep:
2446       \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2447       {
2448         \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2449       }
2450       { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2451     }
2452   }
2453 }
2454 }

```

(End of definition for __spintent_sprazon_exec:nnn.)

\sprazon The public command \sprazon enforces *math mode* and delegates execution to __spintent_sprazon_exec:nnn.

```

2455 \NewDocumentCommand \sprazon { 0{} m m }
2456 {
2457   \mode_if_math:TF
2458   {
2459     \group_begin:
2460     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2461     \group_end:
2462   }
2463   { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2464 }

```

(End of definition for \sprazon. This function is documented on page ??.)

11.29 The command \spProp

The user command \spProp typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing __spintent_sprazon_exec:nnn directly (not the public \sprazon command) for each ratio.

\spProp The public command \spProp enforces *math mode*, and forwards the same *<keys>* in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native \MathMLintent reading _como over the $=$ sign.

```

2465 \NewDocumentCommand \spProp { 0{} m m m m }
2466 {
2467   \mode_if_math:TF
2468   {
2469     \group_begin:
2470     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2471     \bool_if:NTF \l__spintent_sprazon_silent_bool
2472     {
2473       \MathMLintent { :silent } { = }
2474     }

```

```

2475         { \MathMLintent { _como } { = } }
2476     \__spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2477     \group_end:
2478 }
2479 { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2480 }

```

(End of definition for \spProp. This function is documented on page ??.)

11.30 The command \spang

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2481 \str_new:N \l__spintent_spang_luaset_grado_str
2482 \str_new:N \l__spintent_spang_luaset_minuto_str
2483 \str_new:N \l__spintent_spang_luaset_segundo_str
2484 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for \l__spintent_spang_luaset_grado_str and others.)

11.30.1 Definition of error messages for \spang

invalid-angle-format

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2485 \msg_new:nnnn { spintent } { invalid-angle-format }
2486 { Invalid~angle~format~in~#1. }
2487 {
2488     The~argument~'#2'~could~not~be~parsed. \\\
2489 }

```

(End of definition for `invalid-angle-format`.)

11.30.2 Internal functions for \spang

__spintent_spang_process:n

The internal function `\__spintent_spang_process:n` parses the angle input using `\__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2490 \cs_new_protected:Nn \__spintent_spang_process:n
2491 {
2492     \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2493     \__spintent_luafun_spang_parse:n { #1 }
2494     \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2495     {
2496         \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2497     }
2498     {
2499         \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2500         {
2501             \spunit{ \l__spintent_spang_luaset_grado_str °}
2502         }
2503         \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2504         {
2505             \MathMLintent { _minutos:postfix($m) }
2506             {
2507                 \MathMLLarg { m }
2508                 {
2509                     \spunit{ \l__spintent_spang_luaset_minuto_str ' }
2510                 }
2511             }
2512         }
2513         \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2514         {
2515             \MathMLintent { _segundos:postfix($s) }

```

```

2516         {
2517             \MathMLarg { s }
2518         {
2519             \spunit { \l__spintent_spang_luaset_segundo_str }
2520         }
2521     }
2522 }
2523 }
2524 }

```

(End of definition for `\__spintent_spang_process:n`.)

`\spang` The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2525 \NewDocumentCommand \spang { m }
2526 {
2527     \mode_if_math:TF
2528     {
2529         \group_begin:
2530         \__spintent_spang_process:n { #1 }
2531         \group_end:
2532     }
2533     { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2534 }

```

(End of definition for `\spang`. This function is documented on page 23.)

11.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.31.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

cifras-sep
read-sign
wrap-parens
read-parens
2535 \keys_define:nn { spintent / spnZ }
2536 {
2537     cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2538     read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2539     wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2540     wrap-parens .initial:n = false,
2541     wrap-parens .value_required:n = true,
2542     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2543     read-parens .initial:n = true,
2544     read-parens .value_required:n = true,
2545     unknown .code:n =
2546     {
2547         \str_set:Nn \l__spintent_command_name_str { spnZ }
2548         \__spintent_unknown_keys_cmd:n {#1}
2549     },
2550 }

```

(End of definition for `cifras-sep` and others.)

11.31.2 Definition of error messages

`spnz-not-integer` Raised when the $\langle argument \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2551 \msg_new:nnnn { spintent } { spnz-not-integer }
2552 { The~value~'#1'~is~not~a~valid~integer. }
2553 {
2554     Command~\token_to_str:N \spnZ~only~accepts~integers.\\
2555 }

```

(End of definition for `spnz-not-integer`.)

11.31.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2556 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2557 {
2558   \bool_if:NTF \__spintent_spnz_read_parens_bool
2559   {
2560     ( \__spintent_spnnum_print_num_start: )
2561   }
2562   {
2563     \MathMLintent { :silent } { ( }
2564     \__spintent_spnnum_print_num_start:
2565     \MathMLintent { :silent } { ) }
2566   }
2567 }

```

(End of definition for __spintent_spnz_render_parens:.)

__spintent_spnz_process:nn

The function `\__spintent_spnz_process:nn` first processes the `<keys>` passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnnum_print_num_start:`.

```

2568 \cs_new_protected:Nn \__spintent_spnz_process:nn
2569 {
2570   \keys_set:nn { spintent / spnZ } {#1}
2571   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2572   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2573   {
2574     \tl_if_empty:NTF \__spintent_luaset_part_int_tl
2575     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2576     {
2577       \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2578       { \__spintent_spnz_render_parens: }
2579       { \__spintent_spnnum_print_num_start: }
2580     }
2581   }
2582   { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2583 }

```

(End of definition for __spintent_spnz_process:nn.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2584 \NewDocumentCommand \spnZ { O{} m }
2585 {
2586   \mode_if_math:TF
2587   {
2588     \group_begin:
2589     \__spintent_spnz_process:nn { #1 } { #2 }
2590     \group_end:
2591   }
2592   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2593 }

```

(End of definition for \spnZ. This function is documented on page 16.)

11.32 The command \spmQ

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct `MathML` intent for `MathCAT`.

11.32.1 Definition of variables for \spmQ

The variable `\__spintent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\__spintent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in the integer part, used by the function `\__spintent_spmQ_print_scale_int:n`.

The variables `\__spintent_spmQ_wrap_parens_l_tl` and `\__spintent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\__spintent_saved_luamml_flag_int` temporarily stores the current value of `\__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```

2594 \str_new:N \__spintent_spmQ_join_word_str
2595 \str_new:N \__spintent_spmQ_intent_read_sign_str
2596 \tl_new:N \__spintent_spmQ_wrap_parens_l_tl
2597 \tl_new:N \__spintent_spmQ_wrap_parens_r_tl
2598 \int_new:N \__spintent_saved_luamml_flag_int

```

(End of definition for `\__spintent_spmQ_join_word_str` and others.)

11.32.2 Definition of error messages for \spmQ

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in $\{ \langle argument \rangle \}$ is not actually an integer.

```

2599 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2600 {
2601   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2602 }
2603 {
2604   The~denominator~of~a~fraction~cannot~be~zero.\\
2605 }
2606 \msg_new:nnnn { spintent } { spmQ-not-integer }
2607 {
2608   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2609 }
2610 {
2611   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2612 }

```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.32.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```

join-word      2613 \keys_define:nn { spintent / spmQ }
use-spnZ      2614 {
wrap-parens    2615   join-word      .choices:nn =
read-parens    2616   { entero , enteros , y }
print-dfrac    2617   {
unknown        2618     \int_case:nn { \l_keys_choice_int }
                2619     {
                2620       {1} { \str_set:Nn \__spintent_spmQ_join_word_str { entero } }
                2621       {2} { \str_set:Nn \__spintent_spmQ_join_word_str { enteros } }
                2622       {3} { \str_set:Nn \__spintent_spmQ_join_word_str { y } }
                2623     }
                2624   },
join-word      2625   join-word      .initial:n = y,
join-word      2626   join-word      .value_required:n = true,
join-word/unknown 2627   join-word/unknown .code:n =
                2628     \msg_error:nnee { spintent } { unknown-choice }
                2629     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
use-spnZ      2630   use-spnZ      .bool_set:N = \__spintent_spmQ_use_spnZ_bool,
use-spnZ      2631   use-spnZ      .initial:n = true,
use-spnZ      2632   use-spnZ      .value_required:n = true,
print-dfrac    2633   print-dfrac    .bool_set:N = \__spintent_spmQ_print_dfrac_bool,
print-dfrac    2634   print-dfrac    .initial:n = false,
print-dfrac    2635   print-dfrac    .value_required:n = true,
wrap-parens    2636   wrap-parens    .bool_set:N = \__spintent_spmQ_wrap_parens_bool,
wrap-parens    2637   wrap-parens    .initial:n = false,
wrap-parens    2638   wrap-parens    .value_required:n = true,

```

```

2639     read-parens      .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2640     read-parens      .initial:n = true,
2641     read-parens      .value_required:n = true,
2642     unknown          .code:n =
2643     {
2644         \str_set:Nn \l__spintent_command_name_str { spmQ }
2645         \__spintent_unknown_keys_cmd:n {#1}
2646     },
2647 }

```

(End of definition for join-word and others.)

11.32.4 Internal functions for \spmQ

__spintent_spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LaTeX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to unsized `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2648 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2649 {
2650     \int_case:nnF { \mathstyle }
2651     {
2652         { 0 }
2653         {
2654             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2655             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2656         }
2657         { 1 }
2658         {
2659             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2660             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2661         }
2662         { 2 }
2663         {
2664             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2665             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2666         }
2667         { 3 }
2668         {
2669             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2670             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2671         }
2672     }
2673     {
2674         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2675         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2676     }
2677 }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl\lparen`/`\Bigr\rparen` unconditionally.

```

2677     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2678     {
2679         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2680         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2681     }
2682 }

```

(End of definition for __spintent_spmQ_set_wrap_parens_size:.)

__spintent_spmQ_start_wrap_parens:

__spintent_spmQ_stop_wrap_parens:

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintend{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2683 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2684 {

```

```

2685 \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2686 {
2687   \__spinttent_spmQ_set_wrap_parens_size:
2688   \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2689     { \l__spinttent_spmQ_wrap_parens_l_tl }
2690     { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_l_tl } }
2691 }
2692 }
2693 \cs_new_protected:Nn \__spinttent_spmQ_stop_wrap_parens:
2694 {
2695   \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2696   {
2697     \__spinttent_spmQ_set_wrap_parens_size:
2698     \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2699       { \l__spinttent_spmQ_wrap_parens_r_tl }
2700       { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_r_tl } }
2701   }
2702 }

```

(End of definition for __spinttent_spmQ_start_wrap_parens: and __spinttent_spmQ_stop_wrap_parens:.)

__spinttent_spmQ_set_join_word:

The function `\__spinttent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spinttent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spinttent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2703 \cs_new_protected:Nn \__spinttent_spmQ_set_join_word:
2704 {
2705   \str_case:Nn \l__spinttent_spmQ_join_word_str
2706   {
2707     { entero } { \MathMLintent { _entero } { \c__spinttent_invisible_sep_int } }
2708     { enteros } { \MathMLintent { _enteros } { \c__spinttent_invisible_sep_int } }
2709     { y } { \MathMLintent { _y- } { \c__spinttent_invisible_sep_int } }
2710   }
2711 }

```

(End of definition for __spinttent_spmQ_set_join_word:.)

__spinttent_spmQ_print_scale_int:

The function `\__spinttent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spinttent_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spinttent_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spinttent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spinttent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2712 \cs_new_protected:Nn \__spinttent_spmQ_print_scale_int:
2713 {
2714   \int_set_eq:Nc \l__spinttent_saved_luamml_flag_int { \l__luamml_flag_int }
2715   \int_set:Nn \l__luamml_flag_int { 0 }
2716   \bool_if:NTF \l__spinttent_spmQ_print_dfrac_bool
2717     { \__spinttent_luafun_spmQ_scale_int:Nn \l__spinttent_luaset_part_int_tl { dfraction } }
2718     { \__spinttent_luafun_spmQ_scale_int:Nn \l__spinttent_luaset_part_int_tl { fraction } }
2719   \int_set_eq:cn { \l__luamml_flag_int } \l__spinttent_saved_luamml_flag_int
2720 }

```

(End of definition for __spinttent_spmQ_print_scale_int:.)

__spinttent_spmQ_print_scale_int:n

The function `\__spinttent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in `#1`. It first calls `\__spinttent_luafun_clean_split_and_set:n` on `#1` to parse and populate the shared Lua variables, then checks the status of `\l__spinttent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spinttent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spinttent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spinttent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An

`\__spintent_invisible_sep`: call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2721 \cs_new_protected:Nn \__spintent_spmQ_print_scale_int:n
2722 {
2723   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2724   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2725   {
2726     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
2727     {
2728       \__spintent_spmQ_print_scale_int:
2729     }
2730     {
2731       \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
2732       {
2733         \str_set:Nn
2734           \l__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2735       }
2736       {
2737         \str_set:Nn
2738           \l__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2739       }
2740       \MathMLintent { \l__spintent_spmQ_intent_read_sign_str }
2741       {
2742         \l__spintent_luaset_sign_tl \c__spintent_invisible_sep_int % \__spintent_invisible_s
2743         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2744       }
2745     }
2746   }
2747   { \msg_error:nnn { spintent } { spmQ-not-integer } { #1 } }
2748 }

```

(End of definition for `\__spintent_spmQ_print_scale_int:n`.)

`\__spintent_spmQ_print_spmZ_dfrac:nn`

The function `\__spintent_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintent_spmQ_use_spmZ_bool` and `\l__spintent_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spmZ` and `print-dfrac`. When `use-spmZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2749 \cs_new_protected:Nn \__spintent_spmQ_print_spmZ_dfrac:nn
2750 {
2751   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2752   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { false }
2753   {
2754     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2755   }
2756   \bool_if:NTF \l__spintent_spmQ_use_spmZ_bool
2757   {
2758     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2759     { \spnZ { #1 } } { \spnZ { #2 } }
2760   }
2761   {
2762     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2763     { #1 } { #2 }
2764   }
2765 }

```

(End of definition for `\__spintent_spmQ_print_spmZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:.` Then it calls `\__spintent_spmQ_print_spmZ_dfrac:nn` to

typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2766 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2767 {
2768   \__spintent_spmQ_start_wrap_parens:
2769   \__spintent_spmQ_print_scale_int:n { #1 }
2770   \__spintent_spmQ_set_join_word:
2771   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2772   \__spintent_spmQ_stop_wrap_parens:
2773 }

```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn`

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *⟨keys⟩* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *⟨keys⟩* from ‘#1’, then processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\l__spintent_luaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2774 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2775 {
2776   \keys_set:nn { spintent / spmQ } { #1 }
2777   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2778   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2779   {
2780     \int_compare:nNnTF { \l__spintent_luaset_part_int_tl } = { 0 }
2781     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2782     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2783   }
2784   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2785 }

```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *⟨keys⟩* and the *⟨arguments⟩*.

```

2786 \NewDocumentCommand \spmQ { O{} m m m }
2787 {
2788   \mode_if_math:TF
2789   {
2790     \group_begin:
2791     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2792     \group_end:
2793   }
2794   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2795 }

```

(End of definition for `\spmQ`. This function is documented on page 17.)

11.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l__spintent_geo_luaset_error_str`, `\l__spintent_geo_luaset_intent_str`, `\l__spintent_geo_luaset_print_tl`, `\l__spintent_geo_luaset_sub_str` and `\l__spintent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\__spintent_mathmlarg_point:nnn`. Every command also activates `\l__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spintent_geo_luaset_error_str
\l__spintent_geo_luaset_intent_str
\l__spintent_geo_luaset_print_tl
\l__spintent_geo_luaset_is_name_str
\l__spintent_geo_luaset_poly_sym_str
\l__spintent_geo_luaset_angulo_case_str
\l__spintent_geo_luaset_concept_str
\l__spintent_geo_luaset_letra_word_str
\l__spintent_geo_luaset_sub_spoken_str
\l__spintent_geo_luaset_nombre_es_num_str

```

```

2796 \str_new:N \l__spintent_geo_luaset_error_str
2797 \str_new:N \l__spintent_geo_luaset_intent_str
2798 \tl_new:N \l__spintent_geo_luaset_print_tl
2799 \clist_new:N \l__spintent_geo_luaset_points_clist
2800 \str_new:N \l__spintent_geo_luaset_is_name_str
2801 \str_new:N \l__spintent_geo_luaset_poly_sym_str
2802 \str_new:N \l__spintent_geo_luaset_angulo_case_str
2803 \str_new:N \l__spintent_geo_luaset_concept_str

```

```

2804 \str_new:N \l__spinttent_geo_luaset_letra_word_str
2805 \str_new:N \l__spinttent_geo_luaset_sub_spoken_str
2806 \str_new:N \l__spinttent_geo_luaset_nombre_es_num_str
2807 \str_new:N \l__spinttent_geo_luaset_letra_str
2808 \str_new:N \l__spinttent_geo_luaset_sub_str
2809 \str_new:N \l__spinttent_geo_luaset_tipo_str

```

(End of definition for `\l__spinttent_geo_luaset_error_str` and others.)

11.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x,y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as más/menos — never positivo/negativo, which is not how MathCAT reads a bare sign in this position.

11.34.1 Definition of variables for `\spcoord`

Variables set by the function `__spinttent_luafun_spcoord_parse_and_set:n`. The string variable `\l__spinttent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\l__spinttent_spcoord_luaset_#1_sign_str` is `+`, `-`, or empty; the variables `\l__spinttent_spcoord_luaset_#1_type_str` is `number` or `raw`; the variables `\l__spinttent_spcoord_luaset_#1_value_str` is the value with the sign already removed; and the variables `\l__spinttent_spcoord_luaset_#1_has_frac_str` is `none`, `frac`, or `dfrac`.

```

\l__spinttent_spcoord_luaset_error_str
\l__spinttent_spcoord_luaset_sep_str
\l__spinttent_spcoord_luaset_x_sign_str
\l__spinttent_spcoord_luaset_x_type_str
\l__spinttent_spcoord_luaset_x_value_str
\l__spinttent_spcoord_luaset_x_has_frac_str
\l__spinttent_spcoord_luaset_y_sign_str
\l__spinttent_spcoord_luaset_y_type_str
\l__spinttent_spcoord_luaset_y_value_str
\l__spinttent_spcoord_luaset_y_has_frac_str

2810 \str_new:N \l__spinttent_spcoord_luaset_error_str
2811 \str_new:N \l__spinttent_spcoord_luaset_sep_str
2812 \clist_map_inline:nn { x , y }
2813 {
2814   \str_new:c { \l__spinttent_spcoord_luaset_#1_sign_str }
2815   \str_new:c { \l__spinttent_spcoord_luaset_#1_type_str }
2816   \str_new:c { \l__spinttent_spcoord_luaset_#1_value_str }
2817   \str_new:c { \l__spinttent_spcoord_luaset_#1_has_frac_str }
2818 }

```

(End of definition for `\l__spinttent_spcoord_luaset_error_str` and others.)

```

\l__spinttent_spcoord_raw_arg_tl
\l__spinttent_spcoord_intent_str
\l__spinttent_spcoord_wrap_parens_l_tl
\l__spinttent_spcoord_wrap_parens_r_tl

```

The variable `\l__spinttent_spcoord_raw_arg_tl` holds the $\langle\textit{argument}\rangle$ exactly as given. The variable `\l__spinttent_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\l__spinttent_spcoord_wrap_parens_l_tl`/`\l__spinttent_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `\__spinttent_spcoord_set_wrap_parens_size:.`

```

2819 \tl_new:N \l__spinttent_spcoord_raw_arg_tl
2820 \str_new:N \l__spinttent_spcoord_intent_str
2821 \tl_new:N \l__spinttent_spcoord_wrap_parens_l_tl
2822 \tl_new:N \l__spinttent_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spinttent_spcoord_raw_arg_tl` and others.)

11.34.2 Definition of error messages for `\spcoord`

`spcoord-invalid-arg`

```

2823 \msg_new:nnn { spinttent } { spcoord-invalid-arg }
2824 {
2825   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2826 }

```

(End of definition for `spcoord-invalid-arg`.)

11.34.3 Definition of keys for `\spcoord`

`silent` We define the key `silent`.

```

2827 \keys_define:nn { spinttent / spcoord }
2828 {
2829   silent .bool_set:N = \l__spinttent_spcoord_silent_bool,
2830   silent .initial:n = false,
2831   silent .value_required:n = true,
2832   unknown .code:n =
2833   {
2834     \str_set:Nn \l__spinttent_command_name_str { spcoord }
2835     \__spinttent_unknown_keys_cmd:n {#1}
2836   },
2837 }

```

(End of definition for `silent`.)

11.34.4 Internal functions for \spcoord

With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```

2838 \cs_new_protected:Nn \__spintent_spcoord_set_wrap_parens_size:
2839 {
2840   \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \lparen }
2841   \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \rparen }
2842   \bool_if:nT
2843   {
2844     \bool_lazy_or_p:nn
2845     { \bool_not_p:n { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_x_has_frac_str { none } } }
2846     { \bool_not_p:n { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_y_has_frac_str { none } } }
2847   }
2848   {
2849     \int_case:nnF { \mathstyle }
2850     {
2851       { 0 }
2852       {
2853         \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2854         \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2855       }
2856       { 1 }
2857       {
2858         \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2859         \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2860       }
2861       { 2 }
2862       {
2863         \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2864         \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2865       }
2866       { 3 }
2867       {
2868         \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2869         \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2870       }
2871     }
2872   }
2873   \bool_if:nT
2874   {
2875     \bool_lazy_or_p:nn
2876     { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_x_has_frac_str { dfraction } }
2877     { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_y_has_frac_str { dfraction } }
2878   }
2879   {
2880     \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2881     \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2882   }
2883 }
2884

```

(End of definition for `\__spintent_spcoord_set_wrap_parens_size:`.)

With `\l__spintent_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\l__spintent_spcoord_luaset_ #1 _value_str`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in `#1`.

```

2885 \cs_new_protected:Nn \__spintent_spcoord_value:n
2886 {
2887   \str_if_eq:eeTF
2888   { \str_use:c { \l__spintent_spcoord_luaset_ #1 _type_str } } { number }
2889   {
2890     \exp_args:Ne \spnum { \str_use:c { \l__spintent_spcoord_luaset_ #1 _value_str } }
2891   }
2892   { \str_use:c { \l__spintent_spcoord_luaset_ #1 _value_str } }
2893 }

```

(End of definition for `\__spintent_spcoord_value:n`.)

With `\l__spintent_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\l__spintent_spcoord_luaset_ #1 _value_str` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read` as `más/menos`. With no sign, delegates directly to `\__spintent_spcoord_value:n`. Receives `x` or `y` in `#1`.

```

2894 \cs_new_protected:Nn \__spintent_spcoord_print:n

```

```

2895 {
2896   \str_case:enF { \str_use:c { l__spintenc_spcoord_luaset_ #1 _sign_str } }
2897   {
2898     { + }
2899     {
2900       \MathMLintent { _más:prefix ($vp) }
2901       {
2902         \str_use:c { l__spintenc_spcoord_luaset_ #1 _sign_str }
2903         \MathMLarg{ vp } { \__spintenc_spcoord_value:n { #1 } }
2904       }
2905     }
2906     { - }
2907     {
2908       \MathMLintent { _menos :prefix($vn) }
2909       {
2910         \str_use:c { l__spintenc_spcoord_luaset_ #1 _sign_str }
2911         \MathMLarg{ vn } { \__spintenc_spcoord_value:n { #1 } }
2912       }
2913     }
2914   }
2915   { \__spintenc_spcoord_value:n { #1 } }
2916 }

```

(End of definition for `\__spintenc_spcoord_print:n`.)

`\__spintenc_spcoord_intent_infix:` Sets `\l__spintenc_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\l__spintenc_spcoord_intent_infix:choice`. Then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintenc_spcoord_print:n`'s own output.

```

2917 \cs_new_protected:Nn \__spintenc_spcoord_intent_infix:
2918 {
2919   \str_if_eq:VnTF \l__spintenc_spcoord_luaset_sep_str { ; }
2920   { \str_set:Nn \l__spintenc_spcoord_intent_str { _punto-y-coma:infix } }
2921   { \str_set:Nn \l__spintenc_spcoord_intent_str { _coma:infix } }
2922   \MathMLintent{ \l__spintenc_spcoord_intent_str ($x, $y)}
2923   {
2924     \MathMLarg{x}{ \__spintenc_spcoord_print:n { x } }
2925     \MathMLintent { :silent } { \l__spintenc_spcoord_luaset_sep_str }
2926     \MathMLarg{y}{ \__spintenc_spcoord_print:n { y } }
2927   }
2928 }

```

(End of definition for `\__spintenc_spcoord_intent_infix:`.)

`\__spintenc_spcoord_mathml:` With `silent=true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\l__spintenc_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintenc_spcoord_print:n/\__spintenc_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintenc_spcoord_intent_infix:` provides the reading in between.

```

2929 \cs_new_protected:Nn \__spintenc_spcoord_mathml:
2930 {
2931   \bool_if:NTF \l__spintenc_spcoord_silent_bool
2932   {
2933     \MathMLintent { _- }
2934     {
2935       \l__spintenc_spcoord_wrap_parens_l_tl
2936       \l__spintenc_spcoord_raw_arg_tl
2937       \l__spintenc_spcoord_wrap_parens_r_tl
2938     }
2939   }
2940   {
2941     \MathMLintent { :silent } { \l__spintenc_spcoord_wrap_parens_l_tl }
2942     \__spintenc_spcoord_intent_infix:
2943     \MathMLintent { :silent } { \l__spintenc_spcoord_wrap_parens_r_tl }
2944   }
2945 }

```

(End of definition for `\__spintenc_spcoord_mathml:`.)

`\__spintenc_spcoord_exec:nn` Sets the *(keys)* in `#1` and `\l__spintenc_spcoord_raw_arg_tl` to `#2`. Calls `\__spintenc_luafun_spcoord_parse` either way — even with `silent=true`, since `\__spintenc_spcoord_set_wrap_parens_size:` still needs `\l__spintenc_spcoord_luaset_ #1 _has_frac_str` — but with `silent=true`, forces `\l__spintenc_spcoord_silent_bool` back to `false` rather than reporting it: the raw *{(argument)}* is typeset regardless of whether it parsed.

Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\__spintent_spcoord_mathml:` unless there was an error.

```

2946 \cs_new_protected:Nn \__spintent_spcoord_exec:nn
2947 {
2948   \keys_set:nn { spintent / spcoord } {#1}
2949   \tl_set:Nn \__spintent_spcoord_raw_arg_tl {#2}
2950   \bool_if:NTF \__spintent_spcoord_silent_bool
2951   {
2952     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2953     \str_set:Nn \__spintent_spcoord_luastr_error_str { false }
2954   }
2955   {
2956     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2957     \str_if_eq:VnT \__spintent_spcoord_luastr_error_str { true }
2958     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#2} } }
2959   }
2960   \__spintent_spcoord_set_wrap_parens_size:
2961   \str_if_eq:VnF \__spintent_spcoord_luastr_error_str { true }
2962   { \__spintent_spcoord_mathml: }
2963 }

```

(End of definition for `\__spintent_spcoord_exec:nn`.)

`\spcoord` The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spcoord_exec:nn`.

```

2964 \NewDocumentCommand \spcoord { 0{ } m }
2965 {
2966   \mode_if_math:TF
2967   {
2968     \group_begin:
2969     \attribute \__spintent_scope_attr_int = 1
2970     \__spintent_spcoord_exec:nn {#1} {#2}
2971     \group_end:
2972   }
2973   { \msg_error:nnn { spintent } { need-math-mode } { \spcoord } }
2974 }

```

(End of definition for `\spcoord`. This function is documented on page 17.)

11.35 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command's own `read-cmd/silent` state, looked up by name (#2).

```

\l__spintent_mathmlarg_point_ref_tl
\l__spintent_mathmlarg_point_mode_str
\l__spintent_mathmlarg_point_kv_tl
\l__spintent_mathmlarg_point_silent_bool
2975 \tl_new:N \__spintent_mathmlarg_point_ref_tl
2976 \str_new:N \__spintent_mathmlarg_point_mode_str
2977 \tl_new:N \__spintent_mathmlarg_point_kv_tl
2978 \bool_new:N \__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\__spintent_mathmlarg_point_ref_tl` and others.)

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\__spintent_geo_luastr_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

2979 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
2980 {
2981   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
2982   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
2983   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_bool }
2984   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
2985   \__spintent_invisible_sep:
2986   \str_case:VnF \__spintent_geo_luastr_tipo_str
2987   {
2988     { sub }
2989     {
2990       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2991       {
2992         \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
2993         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua

```

```

2994         \__spintent_luafun_mrow_intent_arg:nn
2995         { } { \__spintent_mathmlarg_point_kv_tl }
2996     {
2997         \__spintent_geo_luaset_letra_str
2998         \c_math_subscript_token
2999         { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
3000     }
3001 }
3002 {
3003     {
3004         \__spintent_inner_sep:
3005         \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
3006         {
3007             {
3008                 \__spintent_geo_luaset_letra_str
3009                 \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3010             }
3011         }
3012     }
3013 }
3014 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

3015     { prima }
3016     {
3017         \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3018         {
3019             \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
3020             { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
3021             \__spintent_luafun_mrow_intent_arg:nn
3022             { } { \__spintent_mathmlarg_point_kv_tl }
3023             { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
3024         }
3025         {
3026             {
3027                 \__spintent_inner_sep:
3028                 \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
3029                 {
3030                     { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
3031                 }
3032             }
3033         }
3034     }
3035 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3036     {
3037         \bool_if:NTF \__spintent_mathmlarg_point_silent_bool
3038         {
3039             \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
3040             { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "-_" }
3041             \__spintent_luafun_mrow_intent_arg:nn
3042             { } % debe ir vacío
3043             { \__spintent_mathmlarg_point_kv_tl }
3044             { \__spintent_geo_luaset_print_tl }
3045         }
3046         {
3047             \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3048             {
3049                 \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
3050                 { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
3051                 \__spintent_luafun_mrow_intent_arg:nn
3052                 { } % debe ir vacío
3053                 { \__spintent_mathmlarg_point_kv_tl }
3054                 { \__spintent_geo_luaset_print_tl }
3055             }
3056         }

```

```

3057             \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
3058             { \l__spintent_geo_luaseta_print_tl }
3059         }
3060     }
3061 }
3062 }
3063 \cs_generate_variant:Nn \l__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\l__spintent_mathmlarg_point:nnn`.)

11.36 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.36.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
3064 \str_new:N \l__spintent_spPoint_mode_str
3065 \tl_new:N \l__spintent_spPoint_prefix_tl
3066 \tl_new:N \l__spintent_spPoint_intent_arg_tl

```

(End of definition for `\l__spintent_spPoint_mode_str`, `\l__spintent_spPoint_prefix_tl`, and `\l__spintent_spPoint_intent_arg_tl`.)

11.36.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

3067 \msg_new:nnnn { spintent } { spPoint-invalid-point }
3068 {
3069     Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
3070 }
3071 {
3072     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
3073     subscript~(\_{...})~or~prime~(').\
3074 }

```

(End of definition for `spPoint-invalid-point`.)

11.36.3 Definition of keys for `\spPoint`

`read-cmd`
`prefix`
`silent`
`unknown`

We define the *(keys)* `read-cmd`, `prefix`, `silent` and `concept`.

```

3075 \keys_define:nn { spintent / spPoint }
3076 {
3077     read-cmd .choices:nn = { school , mathcat }
3078     {
3079         \int_case:nn { \l_keys_choice_int }
3080         {
3081             {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
3082             {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
3083         }
3084     },
3085     read-cmd .initial:n = school,
3086     read-cmd .value_required:n = true,
3087     read-cmd / unknown .code:n =
3088         \msg_error:nneee { spintent } { unknown-choice }
3089         { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3090     prefix .code:n =
3091         { \l__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
3092     prefix .initial:n = ,
3093     prefix .value_required:n = true,
3094     silent .bool_set:N = \l__spintent_spPoint_silent_bool,
3095     silent .initial:n = false,
3096     silent .value_required:n = true,
3097     concept .bool_set:N = \l__spintent_spPoint_concept_bool,
3098     concept .initial:n = true,
3099     concept .value_required:n = true,
3100     unknown .code:n =
3101     {
3102         \str_set:Nn \l__spintent_command_name_str { spPoint }
3103         \l__spintent_unknown_keys_cmd:n {#1}
3104     },
3105 }

```

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for \spPoint

_spintenc_spPoint_print:n

The internal function `\_spintenc\_spPoint\_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\_spintenc\_luafun\_geo\_point\_parse\_and\_set:n`, reporting `spPoint-invalid-point` on error. If `silent` is `true` or `concept` is `false`, sets `\_spintenc\_spPoint\_intent\_arg\_tl` to `\_($pt)` — forced further to `\_` — when `silent` is `true` and `read-cmd` is not `school`, since `\_spintenc\_spPoint\_mathml:n` then tags through a native `\MathMLarg` that needs the concept silenced too. Otherwise sets it to `punto:prefix($pt)`, prefixed with `prefix` when given. Finally calls `\_spintenc\_spPoint\_mathml:n`.

```

3106 \cs_new_protected:Nn \_spintenc\_spPoint\_print:n
3107 {
3108   \_spintenc\_luafun\_geo\_point\_parse\_and\_set:n { \exp_not:n {#1} }
3109   \str_if_eq:VnT \_spintenc\_geo\_luaset\_error\_str { true }
3110   {
3111     \msg_error:nne { spintenc } { spPoint-invalid-point } { \exp_not:n {#1} }
3112   }
3113   \bool_if:nTF
3114   {
3115     \bool_lazy_or:p:nn
3116     { \_spintenc\_spPoint\_silent\_bool }
3117     { ! \_spintenc\_spPoint\_concept\_bool }
3118   }
3119   {
3120     \tl_set:Nn \_spintenc\_spPoint\_intent\_arg\_tl { \_($pt) }
3121     \bool_if:NT \_spintenc\_spPoint\_silent\_bool
3122     {
3123       \str_if_eq:VnF \_spintenc\_spPoint\_mode\_str { school }
3124       {
3125         \tl_set:Nn \_spintenc\_spPoint\_intent\_arg\_tl { \_ }
3126       }
3127     }
3128   }
3129   {
3130     \tl_set:Ne \_spintenc\_spPoint\_intent\_arg\_tl
3131     {
3132       \tl_if_empty:NF \_spintenc\_spPoint\_prefix\_tl
3133       {
3134         \tl_use:N \_spintenc\_spPoint\_prefix\_tl -
3135       }
3136       punto:prefix($pt)
3137     }
3138   }
3139   \_spintenc\_spPoint\_mathml:n { #1 }
3140 }

```

(End of definition for `\_spintenc\_spPoint\_print:n`.)

_spintenc_spPoint_mathml:n

The function `\_spintenc\_spPoint\_mathml:n` calls `\_spintenc\_invisible\_sep:` first, then wraps `\_spintenc\_mathmlarg\_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\_spintenc\_spPoint\_intent\_arg\_tl` as the concept `intent`, with `\_spintenc\_inner\_sep:` first inside that group.

```

3141 \cs_new_protected:Nn \_spintenc\_spPoint\_mathml:n
3142 {
3143   \_spintenc\_invisible\_sep:
3144   \MathMLintent { \_spintenc\_spPoint\_intent\_arg\_tl }
3145   {
3146     \_spintenc\_inner\_sep:
3147     \_spintenc\_mathmlarg\_point:nnn { pt } { spPoint } { #1 }
3148   }
3149 }

```

(End of definition for `\_spintenc\_spPoint\_mathml:n`.)

_spintenc_spPoint_exec:nnn

The function `\_spintenc\_spPoint\_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’ and the point $\langle argument \rangle$ in ‘#2’, delegating to `\_spintenc\_spPoint\_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\_spintenc\_spcoord\_silent\_bool` to `\_spintenc\_spPoint\_silent\_bool` and calls `\_spintenc\_luafun\_spcoord\_parse\_and\_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\_spintenc\_spcoord\_set\_wrap\_parens\_size:` and `\_spintenc\_spcoord\_mathml:`.

```

3150 \cs_new_protected:Nn \_spintenc\_spPoint\_exec:nnn
3151 {
3152   \keys_set:nn { spintenc / spPoint } { #1 }

```

```

3153     \__spintent_spPoint_print:n { #2 }
3154     \tl_if_novalue:nF {#3}
3155     {
3156         \MathMLint { :silent } { \c__spintent_invisible_sep_int } % silencia el "por"
3157         \bool_set_eq:NN \__spintent_spcoord_silent_bool \__spintent_spPoint_silent_bool
3158         \tl_set:Nn \__spintent_spcoord_raw_arg_tl {#3}
3159         \bool_if:NTF \__spintent_spcoord_silent_bool
3160         {
3161             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3162             \str_set:Nn \__spintent_spcoord_luaerror_str { false }
3163         }
3164         {
3165             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3166             \str_if_eq:VnT \__spintent_spcoord_luaerror_str { true }
3167             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3168         }
3169         \__spintent_spcoord_set_wrap_parens_size:
3170         \str_if_eq:VnF \__spintent_spcoord_luaerror_str { true }
3171         { \__spintent_spcoord_mathml: }
3172     }
3173 }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

3174 \NewDocumentCommand \spPoint { 0{} m d() }
3175 {
3176     \mode_if_math:TF
3177     {
3178         \group_begin:
3179         \attribute \__spintent_scope_attr_int = 1
3180         \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
3181         \group_end:
3182     }
3183     { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
3184 }

```

(End of definition for `\spPoint`. This function is documented on page 18.)

11.37 The command `\spvec`

The user command `\spvec` supports two types of notation for the input $\langle \textit{argument} \rangle$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\langle \textit{argument} \rangle$.

11.37.1 Definition of variables for `\spvec`

The variable `\__spintent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\__spintent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

3185 \str_new:N \__spintent_spvec_mode_str
3186 \tl_new:N \__spintent_spvec_prefix_tl
3187 \tl_new:N \__spintent_spvec_intent_arg_tl
3188 \tl_new:N \__spintent_spvec_kv_tl
3189 \seq_new:N \__spintent_spvec_pt_seq
3190 \tl_new:N \__spintent_spvec_refs_tl
3191 \int_new:N \__spintent_spvec_pt_idx_int

```

(End of definition for `\__spintent_spvec_mode_str` and others.)

11.37.2 Definition of error messages for `\spvec`

`spvec-invalid-arg` The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

3192 \msg_new:nnn { spintent } { spvec-invalid-arg }
3193 {
3194     Invalid~argument~'#1'~for~\token_to_str:N \spvec.
3195 }

```

(End of definition for `spvec-invalid-arg`.)

11.37.3 Definition of keys for \spvec

We define the *(keys)* `silent`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

3196 \keys_define:nn { spintent / spvec }
3197 {
3198   silent      .bool_set:N = \l__spintent_spvec_silent_bool,
3199   silent      .initial:n = false,
3200   silent      .value_required:n = true,
3201   concept     .bool_set:N = \l__spintent_spvec_concept_bool,
3202   concept     .initial:n = true,
3203   concept     .value_required:n = true,
3204   autofit     .bool_set:N = \l__spintent_spvec_autofit_bool,
3205   autofit     .initial:n = false,
3206   autofit     .value_required:n = true,
3207   read-cmd    .choices:nn = { school , mathcat }
3208   {
3209     \int_case:nn { \l_keys_choice_int }
3210     {
3211       {1} { \str_set:Nn \l__spintent_spvec_mode_str { school } }
3212       {2} { \str_set:Nn \l__spintent_spvec_mode_str { mathcat } }
3213     }
3214   },
3215   read-cmd    .initial:n = school,
3216   read-cmd    .value_required:n = true,
3217   read-cmd / unknown .code:n =
3218     \msg_error:nneee { spintent } { unknown-choice }
3219     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3220   prefix      .code:n =
3221     { \__spintent_sanitizize_affix:Nn \l__spintent_spvec_prefix_tl {#1} },
3222   prefix      .initial:n = { },
3223   prefix      .value_required:n = true,
3224   unknown     .code:n =
3225     {
3226       \str_set:Nn \l__spintent_command_name_str { spvec }
3227       \__spintent_unknown_keys_cmd:n {#1}
3228     },
3229 }

```

(End of definition for `silent` and others.)

11.37.4 Internal functions for \spvec

`\__spintent_spvec_print:n` The internal function `\__spintent_spvec_print:n` processes the *(argument)* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\l__spintent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintent_spvec_mathml:n`.

```

3230 \cs_new_protected:Nn \__spintent_spvec_print:n
3231 {
3232   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3233   \str_if_eq:VnT \l__spintent_geo_luaset_is_name_str { true }
3234   {
3235     \str_if_eq:VnT \l__spintent_geo_luaset_tipo_str { prima }
3236     { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
3237   }
3238   \str_if_eq:VnT \l__spintent_geo_luaset_is_name_str { false }
3239   {
3240     \int_compare:nT { \clist_count:N \l__spintent_geo_luaset_points_clist != 2 }
3241     { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
3242   }
3243   \str_if_eq:VnTF \l__spintent_geo_luaset_error_str { true }
3244   { \msg_error:nne { spintent } { spvec-invalid-arg } { \exp_not:n {#1} } }
3245   { \__spintent_spvec_mathml:n { #1 } }
3246 }

```

(End of definition for `\__spintent_spvec_print:n`.)

`\__spintent_spvec_mathml:n` The function `\__spintent_spvec_mathml:n` branches on `\l__spintent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_($pt)` when `silent` is `true` or `concept` is `false` — forced further to `_-` when `silent` is `true` and `read-cmd` is not `school`; otherwise vector:prefix(`$pt`). With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the

subscript silenced only under `silent`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same kv logic wraps `\vec{print\_tl}` as a whole.

```

3247 \cs_new_protected:Nn \__spintent_sptvec_mathml:n
3248 {
3249   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
3250   {
3251     \bool_if:nTF
3252     {
3253       \bool_lazy_or_p:nn
3254       { \l__spintent_sptvec_silent_bool }
3255       { ! \l__spintent_sptvec_concept_bool }
3256     }
3257     {
3258       \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { _($pt) }
3259       \bool_if:NT \l__spintent_sptvec_silent_bool
3260       {
3261         \str_if_eq:VnF \l__spintent_sptvec_mode_str { school }
3262         { \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { _- } }
3263       }
3264     }
3265     {
3266       \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { vector:prefix($pt) }
3267     }
3268   }
3269   \str_case:VnF \l__spintent_geo_luaset_tipo_str
3270   {
3271     { sub }
3272     {
3273       \str_if_eq:VnTF \l__spintent_sptvec_mode_str { school }
3274       {
3275         \bool_if:NTF \l__spintent_sptvec_silent_bool
3276         { \tl_set:Nn \l__spintent_sptvec_kv_tl { arg = "pt", intent = "_-" } }
3277         {
3278           \tl_set:Ne \l__spintent_sptvec_kv_tl
3279           { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3280         }
3281       }
3282       \__spintent_invisible_sep:
3283       \MathMLintent { \l__spintent_sptvec_intent_arg_tl }
3284       {
3285         \__spintent_inner_sep:
3286         \__spintent_luafun_mrow_intent_arg:nn
3287         { } { \l__spintent_sptvec_kv_tl }
3288         {
3289           \vec
3290           {
3291             \bool_if:NT \l__spintent_sptvec_autofit_bool
3292             { \vphantom{ b } }
3293             \l__spintent_geo_luaset_letra_str
3294           }
3295           \c_math_subscript_token
3296           { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3297         }
3298       }
3299     }
3300     {
3301       \__spintent_invisible_sep:
3302       \MathMLintent { \l__spintent_sptvec_intent_arg_tl }
3303       {
3304         \__spintent_inner_sep:
3305         \vec
3306         {
3307           \bool_if:NT \l__spintent_sptvec_autofit_bool
3308           { \vphantom{ b } }
3309           \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
3310         }
3311         \bool_if:NT \l__spintent_sptvec_silent_bool
3312         { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3313       }
3314     }
3315     \bool_if:NF \l__spintent_sptvec_silent_bool
3316     { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3317   }

```

```

3316     }
3317     {
3318         \bool_if:NTF \l__spintent_sspec_silent_bool
3319         { \tl_set:Nn \l__spintent_sspec_kv_tl { arg = "pt", intent = "_-" } }
3320         {
3321             \tl_set:Nn \l__spintent_sspec_kv_tl
3322             { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3323         }
3324         \__spintent_invisible_sep:
3325         \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3326         {
3327             \__spintent_inner_sep:
3328             \__spintent_luafun_mrow_intent_arg:nn
3329             { } { \l__spintent_sspec_kv_tl }
3330             {
3331                 \vec
3332                 {
3333                     \bool_if:NT \l__spintent_sspec_autofit_bool
3334                     { \vphantom{ b } }
3335                     \l__spintent_geo_luaset_print_tl
3336                 }
3337             }
3338         }
3339     }
3340 }

```

For a pair of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sspec_pt_seq` and walked to build `\l__spintent_sspec_refs_tl`, the `$pts1`, `$pts2` reference list. The concept `intent` is `_` (refs) or `vector:prefix` (refs), same conditions as above, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3341     {
3342         \seq_set_from_clist:NN \l__spintent_sspec_pt_seq \l__spintent_geo_luaset_points_clist
3343         \tl_clear:N \l__spintent_sspec_refs_tl
3344         \int_zero:N \l__spintent_sspec_pt_idx_int
3345         \seq_map_inline:Nn \l__spintent_sspec_pt_seq
3346         {
3347             \int_incr:N \l__spintent_sspec_pt_idx_int
3348             \tl_if_empty:NF \l__spintent_sspec_refs_tl
3349             { \tl_put_right:Nn \l__spintent_sspec_refs_tl { , } }
3350             \tl_put_right:Ne \l__spintent_sspec_refs_tl { $ pts \int_use:N \l__spintent_sspec_pt_
3351         }
3352         \bool_if:nTF
3353         {
3354             \bool_lazy_or:p:nn
3355             { \l__spintent_sspec_silent_bool }
3356             { ! \l__spintent_sspec_concept_bool }
3357         }
3358         {
3359             \tl_set:Nn \l__spintent_sspec_intent_arg_tl { _ ( \l__spintent_sspec_refs_tl ) }
3360         }
3361         {
3362             \tl_set:Nn \l__spintent_sspec_intent_arg_tl
3363             {
3364                 \tl_if_empty:NF \l__spintent_sspec_prefix_tl
3365                 { \tl_use:N \l__spintent_sspec_prefix_tl - }
3366                 vector:prefix( \l__spintent_sspec_refs_tl )
3367             }
3368         }
3369         \__spintent_invisible_sep:
3370         \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3371         {
3372             \__spintent_inner_sep:
3373             \overrightarrow
3374             {
3375                 \int_zero:N \l__spintent_sspec_pt_idx_int
3376                 \seq_map_inline:Nn \l__spintent_sspec_pt_seq
3377                 {
3378                     \int_incr:N \l__spintent_sspec_pt_idx_int
3379                     \__spintent_mathmlarg_point:nnn
3380                     { pts \int_use:N \l__spintent_sspec_pt_idx_int }

```

```

3381         { spvec }
3382         { ##1 }
3383     }
3384 }
3385 }
3386 }
3387 }

```

(End of definition for `\__spintent_spvec_mathml:n`.)

`\__spintent_spvec_exec:nnn` The function `\__spintent_spvec_exec:nnn` receives the optional $\langle\textit{keys}\rangle$ in ‘#1’ and the $\{\langle\textit{argument}\rangle\}$ in ‘#2’, delegating to `\__spintent_spvec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\l__spintent_spcoord_silent_bool` to `\l__spintent_spvec_silent_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_mathml:` — the same rendering functions `\spcoord` itself uses.

```

3388 \cs_new_protected:Nn \__spintent_spvec_exec:nnn
3389 {
3390     \keys_set:nn { spintent / spvec } { #1 }
3391     \__spintent_spvec_print:n { #2 }
3392     \tl_if_novalue:nF {#3}
3393     {
3394         \equal % notación de vector =( , )
3395         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
3396         \bool_set_eq:NN \l__spintent_spcoord_silent_bool \l__spintent_spvec_silent_bool
3397         \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3398         \bool_if:NTF \l__spintent_spvec_silent_bool
3399         {
3400             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3401             \str_set:Nn \l__spintent_spcoord_luaerror_str { false }
3402         }
3403         {
3404             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3405             \str_if_eq:VnT \l__spintent_spcoord_luaerror_str { true }
3406             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3407         }
3408         \__spintent_spcoord_set_wrap_parens_size:
3409         \str_if_eq:VnF \l__spintent_spcoord_luaerror_str { true }
3410         { \__spintent_spcoord_mathml: }
3411     }
3412 }

```

(End of definition for `\__spintent_spvec_exec:nnn`.)

`\spvec` The public command `\spvec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spvec_exec:nnn`.

```

3413 \NewDocumentCommand \spvec { O{} m d() }
3414 {
3415     \mode_if_math:TF
3416     {
3417         \group_begin:
3418         \attribute \l__spintent_scope_attr_int = 1
3419         \__spintent_spvec_exec:nnn { #1 } { #2 } { #3 }
3420         \group_end:
3421     }
3422     { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
3423 }

```

(End of definition for `\spvec`. This function is documented on page 18.)

11.38 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\{\langle\textit{argument}\rangle\}$: when the argument contains two points (separated by an optional comma), `\overlefttrightarrow` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.38.1 Definition of variables for \sprecta

The variable `\__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

3424 \str_new:N \__spintent_sprecta_mode_str
3425 \tl_new:N \__spintent_sprecta_prefix_tl
3426 \seq_new:N \__spintent_sprecta_pt_seq
3427 \tl_new:N \__spintent_sprecta_refs_tl
3428 \tl_new:N \__spintent_sprecta_intent_arg_tl
3429 \tl_new:N \__spintent_sprecta_kv_tl
3430 \int_new:N \__spintent_sprecta_pt_idx_int

```

(End of definition for `\__spintent_sprecta_mode_str` and others.)

11.38.2 Definition of error messages for \sprecta

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3431 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3432 {
3433   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
3434 }
3435 {
3436   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3437 }

```

(End of definition for `sprecta-invalid-points`.)

11.38.3 Definition of keys for \sprecta

We define the *(keys)* `read-cmd`, `prefix` and `silent`.

```

read-cmd 3438 \keys_define:nn { spintent / sprecta }
prefix    3439 {
silent    3440   read-cmd           .choices:nn = { school , mathcat }
unknown   3441   {
3442     \int_case:nn { \_keys_choice_int }
3443     {
3444       {1} { \str_set:Nn \__spintent_sprecta_mode_str { school } }
3445       {2} { \str_set:Nn \__spintent_sprecta_mode_str { mathcat } }
3446     }
3447   },
3448   read-cmd           .initial:n = school,
3449   read-cmd           .value_required:n = true,
3450   read-cmd / unknown .code:n =
3451     \msg_error:nneee { spintent } { unknown-choice }
3452     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3453   prefix             .code:n =
3454     { \__spintent_sanitize_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
3455   prefix             .initial:n = ,
3456   prefix             .value_required:n = true,
3457   silent             .bool_set:N = \__spintent_sprecta_silent_bool,
3458   silent             .initial:n = false,
3459   silent             .value_required:n = true,
3460   concept            .bool_set:N = \__spintent_sprecta_concept_bool,
3461   concept            .initial:n = true,
3462   concept            .value_required:n = true,
3463   unknown            .code:n =
3464     {
3465       \str_set:Nn \__spintent_command_name_str { sprecta }
3466       \__spintent_unknown_keys_cmd:n {#1}
3467     },
3468 }

```

(End of definition for `read-cmd` and others.)

11.38.4 Internal functions for \sprecta

The internal function `\__spintent_sprecta_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `sprecta-invalid-points` on error, then `\__spintent_`

```

3469 \cs_new_protected:Nn \__spintent_sprecta_print:n
3470 {
3471   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3472   \str_if_eq:VnT \__spintent_geo_luaerror_str { true }
3473   {
3474     \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }

```

```

3475     }
3476     \__spintent_sprecta_mathml:n { #1 }
3477 }

```

(End of definition for __spintent_sprecta_print:n.)

__spintent_sprecta_mathml:n The function __spintent_sprecta_mathml:n branches on \l__spintent_geo_luaset_is_name_str. For a proper name (m, l, ...), the concept intent is :silent(\$pt) when silent is true or concept is false; otherwise recta:prefix(\$pt), prefixed with prefix when given.

With \l__spintent_geo_luaset_tipo_str equal to sub, read-cmd=school tags the base through __spintent_luafun_mrow_intent_arg:nn with a literal kv, the subscript appended and silenced; read-cmd=mathcat tags base and subscript together through a native \MathMLarg instead.

```

3478 \cs_new_protected:Nn \__spintent_sprecta_mathml:n
3479 {
3480     \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
3481     {
3482         \__spintent_invisible_sep:
3483         \bool_if:nTF
3484         {
3485             \bool_lazy_or_p:nn
3486             { \l__spintent_sprecta_silent_bool }
3487             { ! \l__spintent_sprecta_concept_bool }
3488         }
3489         {
3490             \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { :silent($pt) }
3491         }
3492         {
3493             \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
3494             {
3495                 \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3496                 { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3497                 recta:prefix($pt)
3498             }
3499         }
3500     \str_case:VnF \l__spintent_geo_luaset_tipo_str
3501     {
3502         { sub }
3503         {
3504             \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3505             {
3506                 \tl_set:Ne \l__spintent_sprecta_kv_tl
3507                 { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3508                 \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3509                 {
3510                     \__spintent_inner_sep:
3511                     \__spintent_luafun_mrow_intent_arg:nn
3512                     { } { \l__spintent_sprecta_kv_tl }
3513                     {
3514                         \l__spintent_geo_luaset_letra_str
3515                         \c_math_subscript_token
3516                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3517                     }
3518                 }
3519             }
3520         }
3521     }
3522 }

```

For read-cmd=mathcat, the same sub case tags base and subscript together through a native \MathMLarg instead — never mixed with the shared queue, since the queue does not recognize a native \MathMLarg claim as already taken.

```

3520 {
3521     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3522     {
3523         {
3524             \__spintent_inner_sep:
3525             \MathMLarg { pt }
3526             {
3527                 {
3528                     \l__spintent_geo_luaset_letra_str
3529                     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3530                 }
3531             }
3532         }
3533     }

```

```

3533         }
3534     }
3535 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

3536 { prima }
3537 {
3538     \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3539     {
3540         \tl_set:Ne \l__spintent_sprecta_kv_tl
3541         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3542         \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3543         {
3544             \__spintent_inner_sep:
3545             \__spintent_luafun_mrow_intent_arg:nn
3546             { } { \l__spintent_sprecta_kv_tl }
3547             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3548         }
3549     }
3550     {
3551         \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3552         {
3553             {
3554                 \__spintent_inner_sep:
3555                 \MathMLarg { pt }
3556                 {
3557                     { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3558                 }
3559             }
3560         }
3561     }
3562 }
3563 }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

3564 {
3565     \bool_if:NTF \l__spintent_sprecta_silent_bool
3566     {
3567         \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "--" }
3568     }
3569     {
3570         \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3571         {
3572             \tl_set:Ne \l__spintent_sprecta_kv_tl
3573             { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3574         }
3575         {
3576             \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt" }
3577         }
3578     }
3579     \__spintent_luafun_mrow_intent_arg:nn
3580     { \l__spintent_sprecta_intent_arg_tl }
3581     { \l__spintent_sprecta_kv_tl }
3582     { \l__spintent_geo_luaset_print_tl }
3583 }
3584 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sprecta_pt_seq` and walked to build `\l__spintent_sprecta_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept `intent` is `_(refs)` or `recta:prefix(refs)`, same conditions as above, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg` once per point.

```

3585 {
3586     \seq_set_from_clist:NN \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
3587     \tl_clear:N \l__spintent_sprecta_refs_tl
3588     \int_zero:N \l__spintent_sprecta_pt_idx_int
3589     \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3590     {
3591         \int_incr:N \l__spintent_sprecta_pt_idx_int

```

```

3592         \tl_if_empty:NF \l__spintent_sprecta_refs_tl
3593         { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
3594         \tl_put_right:Ne \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_
3595     }
3596     \bool_if:nTF
3597     {
3598         \bool_lazy_or_p:nn
3599         { \l__spintent_sprecta_silent_bool }
3600         { ! \l__spintent_sprecta_concept_bool }
3601     }
3602     {
3603         \tl_set:Ne \l__spintent_sprecta_intent_arg_tl { _( \l__spintent_sprecta_refs_tl ) }
3604     }
3605     {
3606         \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
3607         {
3608             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3609             { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3610             recta:prefix( \l__spintent_sprecta_refs_tl )
3611         }
3612     }
3613     \__spintent_invisible_sep:
3614     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3615     {
3616         \__spintent_inner_sep:
3617         \overleftrightharrow
3618         {
3619             \int_zero:N \l__spintent_sprecta_pt_idx_int
3620             \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3621             {
3622                 \int_incr:N \l__spintent_sprecta_pt_idx_int
3623                 \__spintent_mathmlarg_point:nnn
3624                 { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
3625                 { sprecta }
3626                 { ##1 }
3627             }
3628         }
3629     }
3630 }
3631 }

```

(End of definition for `\__spintent_sprecta_mathml:n`.)

`\__spintent_sprecta_exec:nn`

The function `\__spintent_sprecta_exec:nn` receives the optional *(keys)* passed in ‘#1’ and the *{(argument)}* passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

3632 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
3633 {
3634     \keys_set:nn { spintent / sprecta } { #1 }
3635     \__spintent_sprecta_print:n { #2 }
3636 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta`

The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

3637 \NewDocumentCommand \sprecta { 0{} m }
3638 {
3639     \mode_if_math:TF
3640     {
3641         \group_begin:
3642         \attribute \l__spintent_scope_attr_int = 1
3643         \__spintent_sprecta_exec:nn { #1 } { #2 }
3644         \group_end:
3645     }
3646     { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3647 }

```

(End of definition for `\sprecta`. This function is documented on page 19.)

11.39 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

11.39.1 Definition of variables for \sprayo

The variable `\l__spinttent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\l__spinttent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\l__spinttent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

- The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
3648 \str_new:N \l__spinttent_sprayo_mode_str
3649 \tl_new:N \l__spinttent_sprayo_prefix_tl
3650 \str_new:N \l__spinttent_sprayo_read_sty_str
```

(End of definition for `\l__spinttent_sprayo_mode_str`, `\l__spinttent_sprayo_prefix_tl`, and `\l__spinttent_sprayo_read_sty_str`.)

11.39.2 Definition of error messages for \sprayo

`sprayo-invalid-points` Raised when the argument is not a comma-separated pair of uppercase letters.

```
3651 \msg_new:nnnn { spinttent } { sprayo-invalid-points }
3652 {
3653   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3654 }
3655 {
3656   See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3657 }
```

(End of definition for `sprayo-invalid-points`.)

11.39.3 Definition of keys for \sprayo

We define the *(keys)* `read-cmd`, `prefix`, `silent`, `concept` and `read-sty`.

```
read-cmd 3658 \keys_define:nn { spinttent / sprayo }
prefix   3659 {
silent   3660   read-cmd          .choices:nn = { school , mathcat }
read-sty 3661   {
unknown  3662     \int_case:nn { \l_keys_choice_int }
          3663     {
          3664       {1} { \str_set:Nn \l__spinttent_sprayo_mode_str { school } }
          3665       {2} { \str_set:Nn \l__spinttent_sprayo_mode_str { mathcat } }
          3666     }
          3667   },
          3668   read-cmd          .initial:n = school,
          3669   read-cmd          .value_required:n = true,
          3670   read-cmd / unknown .code:n =
          3671     \msg_error:nneee { spinttent } { unknown-choice }
          3672     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
          3673   prefix          .code:n =
          3674     { \__spinttent_sanitiz_abbrev:Nn \l__spinttent_sprayo_prefix_tl {#1} },
          3675   prefix          .initial:n = ,
          3676   prefix          .value_required:n = true,
          3677   silent          .bool_set:N = \l__spinttent_sprayo_silent_bool,
          3678   silent          .initial:n = false,
          3679   silent          .value_required:n = true,
          3680   concept         .bool_set:N = \l__spinttent_sprayo_concept_bool,
          3681   concept         .initial:n = true,
          3682   concept         .value_required:n = true,
          3683   read-sty        .choices:nn = { rayo , semirrecta }
          3684   {
          3685     \int_case:nn { \l_keys_choice_int }
          3686     {
          3687       {1} { \str_set:Nn \l__spinttent_sprayo_read_sty_str { rayo } }
          3688       {2} { \str_set:Nn \l__spinttent_sprayo_read_sty_str { semirrecta } }
          3689     }
          3690   },
          3691   read-sty        .initial:n = rayo,
          3692   read-sty        .value_required:n = true,
          3693   read-sty / unknown .code:n =
          3694     \msg_error:nneee { spinttent } { unknown-choice }
          3695     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
          3696   unknown        .code:n =
          3697   {
          3698     \str_set:Nn \l__spinttent_command_name_str { sprayo }
          3699     \__spinttent_unknown_keys_cmd:n {#1}
          3700   },
          3701 }
```

(End of definition for `read-cmd` and others.)

11.39.4 Internal functions for \spray

Auxiliary variables used by `\__spintent_sprayo_mathml:` to split `\l__spintent_geo_luaset_points_clist` and build the `$pts1, $pts2, ...` reference list.

```

3702 \seq_new:N \l__spintent_sprayo_pt_seq
3703 \tl_new:N \l__spintent_sprayo_refs_tl
3704 \tl_new:N \l__spintent_sprayo_intent_arg_tl
3705 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for `\l__spintent_sprayo_pt_seq` and others.)

The internal function `\__spintent_sprayo_print:n` process the `{\langle argument \rangle}` passed in ‘#1’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process the `{\langle argument \rangle}`, returning an “error” if the input is invalid, then calls `\__spintent_sprayo_mathml:`. Unlike `\sprecta`, `\spray` has no proper-name argument: it always takes a list of points, so no `intent` is built here.

```

3706 \cs_new_protected:Nn \__spintent_sprayo_print:n
3707 {
3708   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3709   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3710   {
3711     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3712   }
3713   \__spintent_sprayo_mathml:
3714 }

```

(End of definition for `\__spintent_sprayo_print:n`.)

The function `\__spintent_sprayo_mathml:` splits `\l__spintent_geo_luaset_points_clist` (set by Lua, one raw point per element) into `\l__spintent_sprayo_pt_seq` and walks it once to build `\l__spintent_sprayo_refs_tl`, the comma-separated `$pts1, $pts2, ...` reference list, naming each point by position.

The concept `intent` is `_` (refs) when `silent` is `true` or `concept` is `false`; otherwise it is `\l__spintent_sprayo_read-sty` — the concept word itself (`rayo` or `semirrecta`) comes from the `read-sty` key, not a literal, since `\spray` verbalizes under two names — prefixed with `prefix` when given. It is applied with a native `\MathMLintent` wrapping `\overrightarrow`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sprayo:` it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3715 \cs_new_protected:Nn \__spintent_sprayo_mathml:
3716 {
3717   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3718   \tl_clear:N \l__spintent_sprayo_refs_tl
3719   \int_zero:N \l__spintent_sprayo_pt_idx_int
3720   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3721   {
3722     \int_incr:N \l__spintent_sprayo_pt_idx_int
3723     \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3724     { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
3725     \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3726   }
3727   \bool_if:nTF
3728   {
3729     \bool_lazy_or:p:nn
3730     { \l__spintent_sprayo_silent_bool }
3731     { ! \l__spintent_sprayo_concept_bool }
3732   }
3733   {
3734     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _ ( \l__spintent_sprayo_refs_tl ) }
3735   }
3736   {
3737     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
3738     {
3739       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3740       { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3741       \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3742     }
3743   }
3744   \__spintent_invisible_sep:
3745   \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
3746   {
3747     \__spintent_inner_sep:

```

```

3748     \overrightarrow
3749     {
3750         \int_zero:N \__spinttent_sprayo_pt_idx_int
3751         \seq_map_inline:Nn \__spinttent_sprayo_pt_seq
3752         {
3753             \int_incr:N \__spinttent_sprayo_pt_idx_int
3754             \__spinttent_mathmlarg_point:nnn
3755             { pts \int_use:N \__spinttent_sprayo_pt_idx_int }
3756             { sprayo }
3757             { ##1 }
3758         }
3759     }
3760 }
3761 }

```

(End of definition for `\__spinttent_sprayo_mathml:`.)

`\__spinttent_sprayo_exec:nn` The function `\__spinttent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{\langle argument \rangle\}$ passed in ‘#2’ delegating the rendering of the latter to `\__spinttent_sprayo_print:n`.

```

3762 \cs_new_protected:Nn \__spinttent_sprayo_exec:nn
3763 {
3764     \keys_set:nn { spinttent / sprayo } { #1 }
3765     \__spinttent_sprayo_print:n { #2 }
3766 }

```

(End of definition for `\__spinttent_sprayo_exec:nn`.)

`\sprayo` The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spinttent_sprayo_exec:nn`.

```

3767 \NewDocumentCommand \sprayo { 0{ } m }
3768 {
3769     \mode_if_math:TF
3770     {
3771         \group_begin:
3772         \attribute \__spinttent_scope_attr_int = 1
3773         \__spinttent_sprayo_exec:nn { #1 } { #2 }
3774         \group_end:
3775     }
3776     { \msg_error:nnn { spinttent } { need-math-mode } { \sprayo } }
3777 }

```

(End of definition for `\sprayo`. This function is documented on page 19.)

11.40 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.40.1 Definition of variables for `\spside` and `\spmside`

`\l__spinttent_spside_mode_str` Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\l__spinttent_spside_mode_str` and `\l__spinttent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\l__spinttent_spside_prefix_tl` and `\l__spinttent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\l__spinttent_spside_read_sty_str` and `\l__spinttent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\l__spinttent_spside_read_arg_tl` and `\l__spinttent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

• The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```

3778 \clist_map_inline:nn { side , mside }
3779 {
3780     \str_new:c { l__spinttent_sp #1 _mode_str }
3781     \tl_new:c { l__spinttent_sp #1 _prefix_tl }
3782     \str_new:c { l__spinttent_sp #1 _read_sty_str }
3783     \tl_new:c { l__spinttent_sp #1 _read_arg_tl }
3784 }

```

(End of definition for `\l__spinttent_spside_mode_str` and others.)

11.40.2 Definition of error messages for \spside and \spmside

spside-invalid-points
spmside-invalid-points

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “point” in the $\langle\textit{argument}\rangle$ passed in ‘#1’ to the command.

```

3785 \clist_map_inline:nn { side , mside }
3786 {
3787   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3788   {
3789     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3790   }
3791   {
3792     See~the~documentation~of~##1~for~the~accepted~point~formats.
3793   }
3794 }
```

(End of definition for *spside-invalid-points* and *spmside-invalid-points*.)

11.40.3 Definition of keys for \spside and \spmside

read-cmd
prefix
silent
concept
read-sty
read-arg
print-m
unknown

We define the shared $\langle\textit{keys}\rangle$ `read-cmd`, `prefix`, `silent`, `concept`, `read-sty`, `read-arg` and `print-m`.

```

3795 \clist_map_inline:nn { side , mside }
3796 {
3797   \keys_define:nn { spintent / sp #1 }
3798   {
3799     read-cmd      .choices:nn = { school , mathcat }
3800     {
3801       \int_case:nn { \l_keys_choice_int }
3802       {
3803         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3804         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
3805       }
3806     },
3807     read-cmd      .initial:n = school,
3808     read-cmd      .value_required:n = true,
3809     read-cmd / unknown .code:n =
3810       \msg_error:nneee { spintent } { unknown-choice }
3811       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3812     prefix        .code:n =
3813       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3814     prefix        .initial:n = ,
3815     prefix        .value_required:n = true,
3816     silent        .bool_set:c = { l__spintent_sp #1 _silent_bool },
3817     silent        .initial:n = false,
3818     silent        .value_required:n = true,
3819     concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
3820     concept       .initial:n = true,
3821     concept       .value_required:n = true,
3822     read-sty      .choices:nn = { lado , segmento , trazo }
3823     {
3824       \int_case:nn { \l_keys_choice_int }
3825       {
3826         {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado } }
3827         {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3828         {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo } }
3829       }
3830     },
3831     read-sty      .initial:n = lado,
3832     read-sty      .value_required:n = true,
3833     read-sty / unknown .code:n =
3834       \msg_error:nneee { spintent } { unknown-choice }
3835       { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3836     read-arg      .code:n =
3837     {
3838       \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
3839     },
3840     read-arg      .initial:n = ,
3841     read-arg      .value_required:n = true,
3842     print-m       .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3843     print-m       .initial:n = false,
3844     print-m       .value_required:n = true,
3845     unknown       .code:n =
3846     {
3847       \str_set:Nn \l__spintent_command_name_str { sp #1 }

```

```

3848         \__spintent_unknown_keys_cmd:n {##1}
3849     },
3850 }
3851 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

```

\__spintent_spside_intent_tl
\__spintent_spside_pt_seq
\__spintent_spside_refs_tl
\__spintent_spside_intent_arg_tl
\__spintent_spside_pt_idx_int

```

`\__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\__spintent_geo_luaset_points_clist` and build the `$pts1, $pts2, ...` reference list.

```

3852 \tl_new:N \__spintent_spside_intent_tl
3853 \seq_new:N \__spintent_spside_pt_seq
3854 \tl_new:N \__spintent_spside_refs_tl
3855 \tl_new:N \__spintent_spside_intent_arg_tl
3856 \int_new:N \__spintent_spside_pt_idx_int

```

(End of definition for `\__spintent_spside_intent_tl` and others.)

```

\__spintent_spside_process:nn

```

The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘`#1`’ and the `{⟨argument⟩}` in ‘`#2`’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘`#2`’, returning an “error” if invalid, then builds `\__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘`medida-del-`’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘`medida-del-`’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_mathml` builds after splitting the points.

```

3857 \cs_new_protected:Nn \__spintent_spside_process:nn
3858 {
3859     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3860     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3861     {
3862         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3863         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3864     }
3865     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
3866     {
3867         \tl_set:Ne \__spintent_spside_intent_tl { \str_use:c { \__spintent_sp #1 _read_sty_str } }
3868         \str_if_eq:eeT {#1} { mside }
3869         {
3870             \tl_put_left:Nn \__spintent_spside_intent_tl { medida-del- }
3871         }
3872     }
3873     {
3874         \tl_set:Ne \__spintent_spside_intent_tl { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
3875         \bool_if:cT { \__spintent_sp #1 _concept_bool }
3876         {
3877             \str_if_eq:eeT {#1} { mside }
3878             {
3879                 \tl_put_left:Nn \__spintent_spside_intent_tl { medida-del- }
3880             }
3881         }
3882     }
3883     \__spintent_spside_mathml:n { #1 }
3884 }

```

(End of definition for `\__spintent_spside_process:nn`.)

```

\__spintent_spside_mathml:n

```

The function `\__spintent_spside_mathml:n` receives the short command name in ‘`#1`’. It splits `\__spintent_geo_luaset_points_clist` into `\__spintent_spside_pt_seq` and walks it once to build `\__spintent_spside_refs_tl`, the comma-separated `$pts1, $pts2, ...` reference list, naming each point by position.

```

3885 \cs_new_protected:Nn \__spintent_spside_mathml:n
3886 {
3887     \seq_set_from_clist:NN \__spintent_spside_pt_seq \__spintent_geo_luaset_points_clist
3888     \tl_clear:N \__spintent_spside_refs_tl
3889     \int_zero:N \__spintent_spside_pt_idx_int
3890     \seq_map_inline:Nn \__spintent_spside_pt_seq
3891     {
3892         \int_incr:N \__spintent_spside_pt_idx_int

```

```

3893 \tl_if_empty:NF \l__spintent_spside_refs_tl
3894 { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3895 \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_id.
3896 }

```

The concept `intent` is `_(refs)` when `silent` is `true`; otherwise, if `read-arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3897 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
3898 {
3899   \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3900 }
3901 {
3902   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
3903   {
3904     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
3905     {
3906       \tl_set:Ne \l__spintent_spside_intent_arg_tl
3907       {
3908         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3909         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3910         \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3911       }
3912     }
3913     {
3914       \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3915     }
3916   }
3917   {
3918     \tl_set:Ne \l__spintent_spside_intent_arg_tl
3919     {
3920       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3921       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3922       \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3923     }
3924   }
3925 }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3926 \__spintent_invisible_sep: % (luamml issues #26)
3927 \MathMLintent { \l__spintent_spside_intent_arg_tl }
3928 {
3929   \__spintent_inner_sep:
3930   \str_if_eq:eeTF {#1} { side }
3931   {
3932     \overline { \__spintent_spside_points_loop:n {#1} }
3933   }
3934   {
3935     \bool_if:cTF { \l__spintent_sp #1 _print_m_bool }
3936     {
3937       \__spintent_luafun_mrow_intent_arg:nn
3938       { } { intent = ":silent" }
3939       { \operatorname { m } } }
3940     \overline { \__spintent_spside_points_loop:n {#1} }
3941   }
3942   {
3943     \__spintent_spside_points_loop:n {#1}
3944   }
3945 }
3946 }
3947 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n` Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in ‘#1’ again since a separate macro does not inherit its caller’s own #1.

```

3948 \cs_new_protected:Nn \__spintent_spside_points_loop:n
3949 {
3950   \int_zero:N \__spintent_spside_pt_idx_int
3951   \seq_map_inline:Nn \__spintent_spside_pt_seq
3952   {
3953     \int_incr:N \__spintent_spside_pt_idx_int
3954     \__spintent_mathmlarg_point:nnn
3955     { pts \int_use:N \__spintent_spside_pt_idx_int }
3956     { sp #1 }
3957     { ##1 }
3958   }
3959 }
```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn` The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spside_process:nn`.

```

3960 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3961 {
3962   \keys_set:nn { spintent / sp #1 } { #2 }
3963   \__spintent_spside_process:nn { #1 } { #3 }
3964 }
```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside` Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

3965 \NewDocumentCommand \spside { O{} m }
3966 {
3967   \mode_if_math:TF
3968   {
3969     \group_begin:
3970     \attribute \__spintent_scope_attr_int = 1
3971     \__spintent_spside_exec:nnn { side } { #1 } { #2 }
3972     \group_end:
3973   }
3974   { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3975 }
3976 \NewDocumentCommand \spmside { O{} m }
3977 {
3978   \mode_if_math:TF
3979   {
3980     \group_begin:
3981     \attribute \__spintent_scope_attr_int = 1
3982     \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
3983     \group_end:
3984   }
3985   { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3986 }
```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 20.)

11.41 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.41.1 Definition of variables for `\spangle` and `\spmangle`

`\__spintent_spangle_mode_str` Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\__spintent_spangle_mode_str` and `\__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spangle_prefix_tl` and `\__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
3987 \clist_map_inline:nn { angle , mangle }
3988 {
3989   \str_new:c { l__spintent_sp #1 _mode_str }
3990   \tl_new:c { l__spintent_sp #1 _prefix_tl }
3991 }
```

(End of definition for `l__spintent_spangle_mode_str` and others.)

11.41.2 Definition of error messages for `\spangle` and `\spmangle`

`spangle-invalid-arg` We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```
3992 \clist_map_inline:nn { angle , mangle }
3993 {
3994   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3995   {
3996     Invalid~format~in~##1:~'##2'~\msg_line_context:.
3997   }
3998   {
3999     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4000   }
4001 }
```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.41.3 Definition of keys for `\spangle` and `\spmangle`

`read-cmd` We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `recto`, `print-m`, `silent` and `concept`,

```
prefix 4002 \clist_map_inline:nn { angle , mangle }
recto   4003 {
print-m 4004   \keys_define:nn { spintent / sp #1 }
silent  4005   {
concept 4006     read-cmd      .choices:nn = { school , mathcat }
unknown 4007     {
4008       \int_case:nn { \l_keys_choice_int }
4009       {
4010         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4011         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4012       }
4013     },
4014     read-cmd      .initial:n = school,
4015     read-cmd      .value_required:n = true,
4016     read-cmd / unknown .code:n =
4017       \msg_error:nneee { spintent } { unknown-choice }
4018       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4019     prefix        .code:n =
4020       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4021     prefix        .initial:n = ,
4022     prefix        .value_required:n = true,
4023     recto         .bool_set:c = { l__spintent_sp #1 _recto_bool },
4024     recto         .initial:n = false,
4025     recto         .value_required:n = true,
4026     print-m       .bool_set:c = { l__spintent_sp #1 _print_m_bool },
4027     print-m       .initial:n = false,
4028     print-m       .value_required:n = true,
4029     silent        .bool_set:c = { l__spintent_sp #1 _silent_bool },
4030     silent        .initial:n = false,
4031     silent        .value_required:n = true,
4032     concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
4033     concept       .initial:n = true,
4034     concept       .value_required:n = true,
4035     unknown       .code:n =
4036     {
4037       \str_set:Nn \l__spintent_command_name_str { sp #1 }
4038       \__spintent_unknown_keys_cmd:n {##1}
4039     },
4040   }
4041 }
```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.41.4 Internal functions for \spangle and \spmangle

__spintent_spangle_sym:n

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

4042 \cs_new_protected:Nn \__spintent_spangle_sym:n
4043 {
4044   \str_if_eq:eeTF {#1} { mangle }
4045   {
4046     \bool_if:cTF { \__spintent_sp #1 _print_m_bool }
4047     {
4048       \bool_if:cTF { \__spintent_sp #1 _recto_bool }
4049       { \__spintent_right_angle_sqr: } { \angle }
4050     }
4051     {
4052       \bool_if:cTF { \__spintent_sp #1 _recto_bool }
4053       { \measuredrightangle } { \measuredangle }
4054     }
4055   }
4056   {
4057     \bool_if:cTF { \__spintent_sp #1 _recto_bool }
4058     { \__spintent_right_angle_sqr: } { \angle }
4059   }
4060 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

\l__spintent_spangle_intent_tl

`\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_lua` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:n`).

\l__spintent_spangle_pt_seq

4061 \tl_new:N \l__spintent_spangle_intent_tl

4062 \seq_new:N \l__spintent_spangle_pt_seq

4063 \tl_new:N \l__spintent_spangle_refs_tl

4064 \tl_new:N \l__spintent_spangle_intent_arg_tl

4065 \int_new:N \l__spintent_spangle_pt_idx_int

4066 \str_new:N \l__spintent_spangle_kv_tl

4067 \bool_new:N \l__spintent_spangle_print_m_now_bool

(End of definition for `\l__spintent_spangle_intent_tl` and others.)

__spintent_spangle_process:nn

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘`#1`’ and the `{\argument}` in ‘`#2`’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\l__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is true, and calls `\__spintent_spangle_mathml:n`.

```

4068 \cs_new_protected:Nn \__spintent_spangle_process:nn
4069 {
4070   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
4071   \str_if_eq:VnT \l__spintent_geo_luaerror_str { true }
4072   {
4073     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4074     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4075   }
4076   \str_if_eq:VnT \l__spintent_geo_luaerror_str { nombre }
4077   {
4078     \str_if_eq:VnF \l__spintent_geo_luaerror_nombre_es_num_str { true }
4079     {
4080       \int_compare:nT { \str_count:N \l__spintent_geo_luaerror_letra_str = 1 }
4081       {
4082         \str_if_eq:eeF { \l__spintent_geo_luaerror_letra_str } { \str_uppercase:n { \l__spintent_geo_luaerror_letra_str } }
4083         {
4084           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4085           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4086         }
4087       }
4088     }
4089   }
4090   \str_if_eq:eeTF {#1} { mangle }
4091   {

```

```

4092         \tl_set:Nn \l__spintent_spangle_intent_tl { medida-del-ángulo }
4093     }
4094     {
4095         \tl_set:Nn \l__spintent_spangle_intent_tl { ángulo }
4096     }
4097     \bool_if:cT { \l__spintent_sp #1 _recto_bool }
4098     {
4099         \tl_put_right:Nn \l__spintent_spangle_intent_tl { -recto }
4100     }
4101     \__spintent_spangle_mathml:n {#1}
4102 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n`

The function `\__spintent_spangle_mathml:n` branches on `\l__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol `(\__spintent_spangle_sym:n)` is tagged through `\__spintent_luafun_mrow_intent_arg` on its own, with `\l__spintent_spangle_intent_tl:prefix` (no $\$$ -reference: `nombre` never refers to more than one thing) — `-` when `silent` is `true` or `concept` is `false`. With `silent true`, the name that follows is silenced as a whole.

```

4103 \cs_new_protected:Nn \__spintent_spangle_mathml:n
4104 {
4105     \__spintent_invisible_sep: % (luamml issues #26)
4106     \str_if_eq:VnTF \l__spintent_geo_luaset_angulo_case_str { nombre }
4107     {
4108         \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4109         {
4110             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4111         }
4112         {
4113             \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4114             {
4115                 \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4116                 {
4117                     \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4118                     { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4119                     \l__spintent_spangle_intent_tl :prefix
4120                 }
4121             }
4122             {
4123                 \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4124             }
4125         }
4126         \tl_set:Ne \l__spintent_spangle_kv_tl
4127         { intent = "\l__spintent_spangle_intent_arg_tl" }
4128         \__spintent_luafun_mrow_intent_arg:nn
4129         { } { \l__spintent_spangle_kv_tl }
4130         { \__spintent_spangle_sym:n {#1} }
4131         \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4132         {
4133             \__spintent_luafun_mrow_intent_arg:nn { } { intent = "_-" }
4134             { \l__spintent_geo_luaset_print_tl }
4135         }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

4136     {
4137         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
4138         {
4139             \tl_set:Ne \l__spintent_spangle_kv_tl
4140             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4141             \__spintent_luafun_mrow_intent_arg:nn
4142             { } { \l__spintent_spangle_kv_tl }
4143             { \l__spintent_geo_luaset_print_tl }
4144         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no $\$$ -reference to resolve).

```

4145     {
4146         \str_case:VnF \l__spintent_geo_luaset_tipo_str
4147         {

```

```

4148         { sub }
4149         {
4150             \str_if_eq:eeTF { \use:c { l__spintent_sp #1 _mode_str } } { school }
4151             {
4152                 \tl_set:Ne \l__spintent_spangle_kv_tl
4153                 { intent = "_" \l__spintent_geo_luaset_intent_str -" }
4154                 \__spintent_luafun_mrow_intent_arg:nn
4155                 { } { \l__spintent_spangle_kv_tl }
4156                 {
4157                     \l__spintent_geo_luaset_letra_str
4158                     \c_math_subscript_token
4159                     { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4160                 }
4161             }
4162             {
4163                 \l__spintent_geo_luaset_letra_str
4164                 \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4165             }
4166         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

4167         { prima }
4168         {
4169             \str_if_eq:eeTF { \use:c { l__spintent_sp #1 _mode_str } } { school }
4170             {
4171                 \tl_set:Ne \l__spintent_spangle_kv_tl
4172                 { intent = "_" \l__spintent_geo_luaset_intent_str -" }
4173                 \__spintent_luafun_mrow_intent_arg:nn
4174                 { } { \l__spintent_spangle_kv_tl }
4175                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4176             }
4177             {
4178                 \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
4179             }
4180         }
4181     }
4182     { \l__spintent_geo_luaset_print_tl }
4183 }
4184 }
4185 }

```

For `tres-puntos/un-punto`, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt_seq` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(refs)` — `_` — when `silent` is `true`, `_(refs)` when `concept` is `false` alone. For `\spmangle` with `print-m` `true`, a silent `\operatorname{m}` precedes `\__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

4186     {
4187         \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
4188         \tl_clear:N \l__spintent_spangle_refs_tl
4189         \int_zero:N \l__spintent_spangle_pt_idx_int
4190         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4191         {
4192             \int_incr:N \l__spintent_spangle_pt_idx_int
4193             \tl_if_empty:NF \l__spintent_spangle_refs_tl
4194             { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
4195             \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_pt_idx_int }
4196         }
4197         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4198         {
4199             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4200         }
4201         {
4202             \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4203             {
4204                 \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4205                 {
4206                     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4207                     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }

```

```

4208         \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
4209     }
4210 }
4211 {
4212     \tl_set:Nx \l__spintent_spangle_intent_arg_tl { _( \l__spintent_spangle_refs_tl )
4213 }
4214 }
4215 \bool_set_false:N \l__spintent_spangle_print_m_now_bool
4216 \str_if_eq:eeT {#1} { mangle }
4217 {
4218     \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
4219     { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
4220 }
4221 \MathMLintent { \l__spintent_spangle_intent_arg_tl }
4222 {
4223     \__spintent_inner_sep:
4224     \bool_if:NT \l__spintent_spangle_print_m_now_bool
4225     {
4226         \__spintent_luafun_mrow_intent_arg:nn
4227         { } { intent = ":silent" }
4228         { \operatorname { m } } }
4229     }
4230     \__spintent_luafun_mrow_intent_arg:nn
4231     { } { intent = ":silent" }
4232     { \__spintent_spangle_sym:n {#1} }
4233     \int_zero:N \l__spintent_spangle_pt_idx_int
4234     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4235     {
4236         \int_incr:N \l__spintent_spangle_pt_idx_int
4237         \__spintent_mathmlarg_point:nnn
4238         { pt \int_use:N \l__spintent_spangle_pt_idx_int }
4239         { sp#1 }
4240         { ##1 }
4241     }
4242 }
4243 }
4244 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn` The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *⟨keys⟩* passed in ‘#2’ and the *⟨argument⟩* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

4245 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
4246 {
4247     \keys_set:nn { spintent / sp #1 } { #2 }
4248     \__spintent_spangle_process:nn { #1 } { #3 }
4249 }

```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle` Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```

4250 \NewDocumentCommand \spangle { 0{} m }
4251 {
4252     \mode_if_math:TF
4253     {
4254         \group_begin:
4255         \attribute \l__spintent_scope_attr_int = 1
4256         \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4257         \group_end:
4258     }
4259     { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
4260 }
4261 \NewDocumentCommand \spmangle { 0{} m }
4262 {
4263     \mode_if_math:TF
4264     {
4265         \group_begin:
4266         \attribute \l__spintent_scope_attr_int = 1
4267         \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }

```

```

4268     \group_end:
4269   }
4270   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4271 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 22.)

11.42 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.42.1 Definition of variables for `\sparc` and `\spmarc`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```

4272 \clist_map_inline:nn { arc , marc }
4273 {
4274   \str_new:c { \__spintent_sp #1 _mode_str }
4275   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4276 }

```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.42.2 Definition of error messages for `\sparc` and `\spmarc`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```

4277 \clist_map_inline:nn { arc , marc }
4278 {
4279   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4280   {
4281     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4282   }
4283   {
4284     See~the~documentation~of~##1~for~the~accepted~point~formats.
4285   }
4286 }

```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.42.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `silent` and `concept`.

```

read-cmd 4287 \clist_map_inline:nn { arc , marc }
prefix   4288 {
silent   4289   \keys_define:nn { spintent / sp #1 }
concept  4290   {
unknown  4291     read-cmd .choices:nn = { school , mathcat }
          4292     {
          4293       \int_case:nn { \l_keys_choice_int }
          4294       {
          4295         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
          4296         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
          4297       }
          4298     },
          4299     read-cmd .initial:n = school,
          4300     read-cmd .value_required:n = true,
          4301     read-cmd / unknown .code:n =
          4302       \msg_error:nnee { spintent } { unknown-choice }
          4303       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          4304     prefix .code:n =
          4305       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
          4306     prefix .initial:n = ,
          4307     prefix .value_required:n = true,
          4308     silent .bool_set:c = { \__spintent_sp #1 _silent_bool },
          4309     silent .initial:n = false,
          4310     silent .value_required:n = true,
          4311     concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
          4312     concept .initial:n = true,
          4313     concept .value_required:n = true,
          4314     unknown .code:n =

```

```

4315         {
4316             \str_set:Nn \l__spintent_command_name_str { sp #1 }
4317             \__spintent_unknown_keys_cmd:n {##1}
4318         },
4319     }
4320 }

```

(End of definition for read-cmd and others.)

11.42.4 Internal functions for \sparc and \spmarc

__spintent_sparc_wide_arc:n

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```

4321 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4322 {
4323     \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4324 }

```

(End of definition for __spintent_sparc_wide_arc:n)

__spintent_sparc_print_arc:n

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4325 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4326 {
4327     \cs_if_exist:NTF \widearc
4328     { \widearc {#1} }
4329     { \__spintent_sparc_wide_arc:n {#1} }
4330 }

```

(End of definition for __spintent_sparc_print_arc:n)

\l__spintent_sparc_intent_tl

`\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, \dots` reference list used in the concept `intent`.

```

4331 \tl_new:N \l__spintent_sparc_intent_tl
4332 \seq_new:N \l__spintent_sparc_pt_seq
4333 \tl_new:N \l__spintent_sparc_refs_tl
4334 \tl_new:N \l__spintent_sparc_intent_arg_tl
4335 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for \l__spintent_sparc_intent_tl and others.)

__spintent_sparc_process:nn

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the `{⟨argument⟩}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No `intent` is built here: it needs the reference list, which `\__spintent_sparc_mathml` builds after splitting the points.

```

4336 \cs_new_protected:Nn \__spintent_sparc_process:nn
4337 {
4338     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4339     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4340     {
4341         \msg_error:nnee { spintent } { sp #1 -invalid-points }
4342         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4343     }
4344     \str_if_eq:eeTF {#1} { marc }
4345     { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
4346     { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
4347     \__spintent_sparc_mathml:n {#1}
4348 }

```

(End of definition for __spintent_sparc_process:nn.)

__spintent_sparc_mathml:n

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated `$pt1, $pt2, \dots` reference list, naming each point by position.

```

4349 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4350 {
4351     \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaset_points_clist
4352 }

```

```

4353 \tl_clear:N \l__spintent_sparc_refs_tl
4354 \int_zero:N \l__spintent_sparc_pt_idx_int
4355 \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4356 {
4357   \int_incr:N \l__spintent_sparc_pt_idx_int
4358   \tl_if_empty:NF \l__spintent_sparc_refs_tl
4359   { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
4360   \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
4361 }

```

The concept `intent` is `_` when `silent` is `true` or `concept` is `false`; otherwise `\l__spintent_sparc_intent_tl`:p prefixed with `prefix` when given.

```

4362 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4363 {
4364   \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4365 }
4366 {
4367   \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4368   {
4369     \tl_set:Ne \l__spintent_sparc_intent_arg_tl
4370     {
4371       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4372       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4373       \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
4374     }
4375   }
4376   {
4377     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4378   }
4379 }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, luamml issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4380 \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4381 \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4382 {
4383   \__spintent_inner_sep:
4384   \str_if_eq:eeT {#1} { marc }
4385   {
4386     \__spintent_luafun_mrow_intent_arg:nn
4387     { } { intent = ":silent" }
4388     { \operatorname { m } } }
4389   }
4390   \__spintent_sparc_print_arc:n
4391   {
4392     \int_zero:N \l__spintent_sparc_pt_idx_int
4393     \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4394     {
4395       \int_incr:N \l__spintent_sparc_pt_idx_int
4396       \__spintent_mathmlarg_point:nnn
4397       { pt \int_use:N \l__spintent_sparc_pt_idx_int }
4398       { sp #1 }
4399       { ##1 }
4400     }
4401   }
4402 }
4403 }

```

(End of definition for `\__spintent_sparc_mathml:n`.)

`\__spintent_sparc_exec:nnn` The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

4404 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4405 {
4406   \keys_set:nn { spintent / sp #1 } { #2 }
4407   \__spintent_sparc_process:nn { #1 } { #3 }
4408 }

```

(End of definition for `\__spintent_sparc_exec:nnn`.)

`\sparc` Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

4409 \NewDocumentCommand \sparc { 0{ } m }
4410 {
4411   \mode_if_math:TF
4412   {
4413     \group_begin:
4414       \attribute \__spintent_scope_attr_int = 1
4415       \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4416     \group_end:
4417   }
4418   { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4419 }
4420 \NewDocumentCommand \spmarc { 0{ } m }
4421 {
4422   \mode_if_math:TF
4423   {
4424     \group_begin:
4425       \attribute \__spintent_scope_attr_int = 1
4426       \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4427     \group_end:
4428   }
4429   { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4430 }

```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 23.)

11.43 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\__spintent_geo_luaset_intent_str` (the full *intent*, with the figure name).

11.43.1 Definition of variables for `\spPoly`

`\__spintent_spPoly_mode_str` The variable `\__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

4431 \str_new:N \__spintent_spPoly_mode_str
4432 \tl_new:N \__spintent_spPoly_prefix_tl

```

(End of definition for `\__spintent_spPoly_mode_str` and `\__spintent_spPoly_prefix_tl`.)

11.43.2 Definition of error messages for `\spPoly`

`spPoly-invalid-points` Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

4433 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4434 {
4435   Invalid~point~format~in~\token_to_str:N \spPoly:~'~#1'~\msg_line_context:.
4436 }
4437 {
4438   See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4439 }

```

(End of definition for `spPoly-invalid-points`.)

11.43.3 Definition of keys for `\spPoly`

`read-cmd` We define the *(keys)* `read-cmd`, `prefix`, `print-sym`, `silent` and `concept`.

```

4440 \keys_define:nn { spintent / spPoly }
4441 {
4442   read-cmd .choices:nn = { school , mathcat }
4443   {
4444     \int_case:nn { \l_keys_choice_int }
4445     {
4446       {1} { \str_set:Nn \__spintent_spPoly_mode_str { school } }
4447       {2} { \str_set:Nn \__spintent_spPoly_mode_str { mathcat } }
4448     }
4449   },
4450   read-cmd .initial:n = school,
4451   read-cmd .value_required:n = true,
4452   read-cmd / unknown .code:n =
4453     \msg_error:nnee { spintent } { unknown-choice }

```

```

4454     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4455     prefix .code:n =
4456     { \__spintent_sanitizize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4457     prefix .initial:n = ,
4458     prefix .value_required:n = true,
4459     print-sym .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4460     print-sym .initial:n = true,
4461     print-sym .value_required:n = true,
4462     silent .bool_set:N = \l__spintent_spPoly_silent_bool,
4463     silent .initial:n = false,
4464     silent .value_required:n = true,
4465     concept .bool_set:N = \l__spintent_spPoly_concept_bool,
4466     concept .initial:n = true,
4467     concept .value_required:n = true,
4468     unknown .code:n =
4469     {
4470         \str_set:Nn \l__spintent_command_name_str { spPoly }
4471         \__spintent_unknown_keys_cmd:n {#1}
4472     },
4473 }

```

(End of definition for `read-cmd` and others.)

11.43.4 Internal functions for `\spPoly`

`\l__spintent_spPoly_pt_seq` Scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list, and to hold the final concept intent once built.

```

\l__spintent_spPoly_refs_tl
\l__spintent_spPoly_intent_arg_tl
\l__spintent_spPoly_pt_idx_int
4474 \seq_new:N \l__spintent_spPoly_pt_seq
4475 \tl_new:N \l__spintent_spPoly_refs_tl
4476 \tl_new:N \l__spintent_spPoly_intent_arg_tl
4477 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for `\l__spintent_spPoly_pt_seq` and others.)

`\__spintent_spPoly_sym:` Unchanged: reads `\l__spintent_geo_luaset_poly_sym_str` and typesets the corresponding symbol.

```

4478 \cs_new_protected:Nn \__spintent_spPoly_sym:
4479 {
4480     \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4481     {
4482         { triangle } { \triangle }
4483         { square } { \mdlgwhtsquare }
4484     }
4485 }

```

(End of definition for `\__spintent_spPoly_sym:`)

`\__spintent_spPoly_build_refs:` Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spPoly_pt_seq` and walks it once to build `\l__spintent_spPoly_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

4486 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4487 {
4488     \seq_set_from_clist:NN \l__spintent_spPoly_pt_seq \l__spintent_geo_luaset_points_clist
4489     \tl_clear:N \l__spintent_spPoly_refs_tl
4490     \int_zero:N \l__spintent_spPoly_pt_idx_int
4491     \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
4492     {
4493         \int_incr:N \l__spintent_spPoly_pt_idx_int
4494         \tl_if_empty:NF \l__spintent_spPoly_refs_tl
4495         { \tl_put_right:Nn \l__spintent_spPoly_refs_tl { , } }
4496         \tl_put_right:Ne \l__spintent_spPoly_refs_tl { $ pt \int_use:N \l__spintent_spPoly_pt_idx.
4497     }
4498 }

```

(End of definition for `\__spintent_spPoly_build_refs:`)

`\__spintent_spPoly_build_concept:` Builds `\l__spintent_spPoly_intent_arg_tl`, independently of `print-sym: _ (refs)` when `silent` is `true` or `concept` is `false`; otherwise `\l__spintent_geo_luaset_concept_str:prefix(refs)`, prefixed with `prefix` when given. Must be called after `\__spintent_spPoly_build_refs:`.

```

4499 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
4500 {
4501     \bool_if:NTF \l__spintent_spPoly_silent_bool
4502     {
4503         \tl_set:Ne \l__spintent_spPoly_intent_arg_tl { _ ( \l__spintent_spPoly_refs_tl ) }

```

```

4504     }
4505     {
4506         \bool_if:NTF \l__spintent_spPoly_concept_bool
4507         {
4508             \tl_set:Nx \l__spintent_spPoly_intent_arg_tl
4509             {
4510                 \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
4511                 { \tl_use:N \l__spintent_spPoly_prefix_tl - }
4512                 \l__spintent_geo_luaset_concept_str :prefix( \l__spintent_spPoly_refs_tl )
4513             }
4514         }
4515         {
4516             \tl_set:Nx \l__spintent_spPoly_intent_arg_tl { _( \l__spintent_spPoly_refs_tl ) }
4517         }
4518     }
4519 }

```

(End of definition for `\__spintent_spPoly_build_concept:`)

`\__spintent_spPoly_points_loop:`

Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg`/literal `intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute`/sub-sup limitation already found and worked around for `\spPoint`).

```

4520 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
4521 {
4522     \int_zero:N \l__spintent_spPoly_pt_idx_int
4523     \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
4524     {
4525         \int_incr:N \l__spintent_spPoly_pt_idx_int
4526         \__spintent_mathmlarg_point:nnn
4527         { pt \int_use:N \l__spintent_spPoly_pt_idx_int }
4528         { spPoly }
4529         { ##1 }
4530     }
4531 }

```

(End of definition for `\__spintent_spPoly_points_loop:`)

`\__spintent_spPoly_process:n`

Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4532 \cs_new_protected:Nn \__spintent_spPoly_process:n
4533 {
4534     \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {##1} }
4535     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4536     {
4537         \msg_error:nne { spintent } { spPoly-invalid-points }
4538         { \exp_not:n {##1} }
4539     }
4540     \__spintent_spPoly_mathml:
4541 }

```

(End of definition for `\__spintent_spPoly_process:n`)

`\__spintent_spPoly_mathml:`

Builds the reference list and the concept `intent` (`\__spintent_spPoly_build_refs:`, `\__spintent_spPoly_build_concept:`) then wraps everything in a single native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26). `print-sym` and `concept/silent` are independent: the former only decides whether the glyph (`\__spintent_spPoly_print_sym:` silenced with `\__spintent_luafun_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\l__spintent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4542 \cs_new_protected:Nn \__spintent_spPoly_mathml:
4543 {
4544     \__spintent_invisible_sep: % (luamml issues #26)
4545     \__spintent_spPoly_build_refs:
4546     \__spintent_spPoly_build_concept:
4547     \MathMLintent { \l__spintent_spPoly_intent_arg_tl }
4548     {
4549         \__spintent_inner_sep:
4550         \bool_if:NT \l__spintent_spPoly_print_sym_bool
4551         {

```

```

4552         \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
4553         {
4554             \__spintent_luafun_mrow_intent_arg:nn
4555             { } { intent = ":silent" }
4556             { \__spintent_spPoly_sym: }
4557         }
4558     }
4559     \__spintent_spPoly_points_loop:
4560 }
4561 }

```

(End of definition for `\__spintent_spPoly_mathml:`.)

`\__spintent_spPoly_exec:nn`

The function `\__spintent_spPoly_exec:nn` receives the optional keys in `#1` and the argument in `#2`, delegating the rendering to `\__spintent_spPoly_process:n`.

```

4562 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
4563 {
4564     \keys_set:nn { spintent / spPoly } { #1 }
4565     \__spintent_spPoly_process:n { #2 }
4566 }

```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly`

The public command `\spPoly` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoly_exec:nn`.

```

4567 \NewDocumentCommand \spPoly { O{} m }
4568 {
4569     \mode_if_math:TF
4570     {
4571         \group_begin:
4572         \attribute \l__spintent_scope_attr_int = 1
4573         \__spintent_spPoly_exec:nn { #1 } { #2 }
4574         \group_end:
4575     }
4576     { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
4577 }

```

(End of definition for `\spPoly`. This function is documented on page 24.)

11.44 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLint{<:silent>}`.

11.44.1 Definition of variables for `\spAfig` and `\spPfig`

`\l__spintent_geo_final_print_tl`

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```

4578 \tl_new:N \l__spintent_geo_final_print_tl

```

(End of definition for `\l__spintent_geo_final_print_tl`.)

`\l__spintent_spAfig_mode_str`

`\l__spintent_spPfig_mode_str`

`\l__spintent_spAfig_print_sym_str`

`\l__spintent_spPfig_print_sym_str`

`\l__spintent_spAfig_prefix_tl`

`\l__spintent_spPfig_prefix_tl`

`\l__spintent_spAfig_read_arg_tl`

`\l__spintent_spPfig_read_arg_tl`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```

4579 \clist_map_inline:nn { Afig , Pfig }
4580 {
4581     \str_new:c { \l__spintent_sp #1 _mode_str }
4582     \str_new:c { \l__spintent_sp #1 _print_sym_str }
4583     \tl_new:c { \l__spintent_sp #1 _prefix_tl }
4584     \tl_new:c { \l__spintent_sp #1 _read_arg_tl }
4585 }

```

(End of definition for `\l__spintent_spAfig_mode_str` and others.)

11.44.2 Definition of error messages for \spAfig and \spPfig

spAfig-invalid-points All eight messages are generated by the same map, using the same ##1/##2 doubling convention as \spside/\spmside. sp #1-invalid-points is currently unused by __spinttent_spfig_process:nn, which raises only sp #1-invalid-arg for any parse failure, regardless of whether a comma was present. sp #1-sym-letra-conflict and sp #1-sym-cardinality-conflict are raised when print-sym is forced to a value incompatible with the actual argument.

```

4586 \clist_map_inline:nn { Afig , Pfig }
4587 {
4588   \msg_new:nnnn { spinttent } { sp #1 -invalid-points }
4589   {
4590     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4591   }
4592   {
4593     See~the~documentation~of~##1~for~the~accepted~point~formats.
4594   }
4595   \msg_new:nnnn { spinttent } { sp #1 -invalid-arg }
4596   {
4597     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
4598   }
4599   {
4600     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4601   }
4602   \msg_new:nnnn { spinttent } { sp #1 -sym-letra-conflict }
4603   {
4604     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
4605   }
4606   {
4607     See~the~documentation~of~##1~for~details~on~print-sym.
4608   }
4609   \msg_new:nnnn { spinttent } { sp #1 -sym-cardinality-conflict }
4610   {
4611     print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
4612   }
4613   {
4614     See~the~documentation~of~##1~for~details~on~print-sym.
4615   }
4616   \msg_new:nnnn { spinttent } { sp #1 -read-arg-key }
4617   {
4618     read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
4619   }
4620   {
4621     See~the~documentation~of~##1~for~details~on~read-arg.
4622   }
4623 }

```

(End of definition for spAfig-invalid-points and others.)

11.44.3 Definition of keys for \spAfig and \spPfig

read-cmd The key print-sym chooses the visual symbol: auto (default) infers it from the vertex count (three points give \triangle, four give \mdlgwhtsquare, five or more give none); triangle/square force one, requiring the vertex count to match; none omits it, even with vertices. With a bare number/letter argument, auto and none behave the same (no symbol); triangle/square nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key read-arg overrides the spoken figure name for real vertices, independently of print-sym. The key read-sub controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

4624 \clist_map_inline:nn { Afig , Pfig }
4625 {
4626   \keys_define:nn { spinttent / sp #1 }
4627   {
4628     read-cmd .choices:nn = { school , mathcat }
4629     {
4630       \int_case:nn { \l_keys_choice_int }
4631       {
4632         {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
4633         {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
4634       }
4635     },
4636     read-cmd .initial:n = school,
4637     read-cmd .value_required:n = true,
4638     read-cmd / unknown .code:n =

```

```

4639     \msg_error:nneee { spintent } { unknown-choice }
4640     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4641 prefix      .code:n =
4642     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4643 prefix      .initial:n = ,
4644 prefix      .value_required:n = true,
4645 print-sym   .choices:nn = { auto , triangle , square , none }
4646     {
4647         \int_case:nn { \l_keys_choice_int }
4648         {
4649             {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto      } }
4650             {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
4651             {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square   } }
4652             {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none     } }
4653         }
4654     },
4655 print-sym   .initial:n = auto,
4656 print-sym   .value_required:n = true,
4657 print-sym / unknown .code:n =
4658     \msg_error:nneee { spintent } { unknown-choice }
4659     { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
4660 read-arg    .code:n =
4661     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4662 read-arg    .initial:n = ,
4663 read-arg    .value_required:n = true,
4664 read-sub    .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4665 read-sub    .initial:n = false,
4666 read-sub    .value_required:n = true,
4667 silent      .bool_set:c = { l__spintent_sp #1 _silent_bool },
4668 silent      .initial:n = false,
4669 silent      .value_required:n = true,
4670 concept     .bool_set:c = { l__spintent_sp #1 _concept_bool },
4671 concept     .initial:n = true,
4672 concept     .value_required:n = true,
4673 concept*    .meta:n =
4674     {
4675         concept = false,
4676         read-arg = { \str_if_eq:eeTF {#1} { Afig } { _área } { _perímetro } },
4677     },
4678 concept*    .value_forbidden:n = true,
4679 unknown     .code:n =
4680     {
4681         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4682         \__spintent_unknown_keys_cmd:n {##1}
4683     },
4684 }
4685 }

```

(End of definition for read-cmd and others.)

11.44.4 Internal functions for \spAfig and \spPfig

`\__spintent_spfig_sym:` The function `\__spintent_spfig_sym:` reads `\l__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

4686 \cs_new_protected:Nn \__spintent_spfig_sym:
4687 {
4688     \str_case:Vn \l__spintent_geo_print_sym_str
4689     {
4690         { triangle } { \triangle      }
4691         { square   } { \mdlgwhtsquare }
4692     }
4693 }

```

(End of definition for `\__spintent_spfig_sym:`)

`\l__spintent_spfig_op_tl` holds `área` or `perímetro`. `\l__spintent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\l__spintent_geo_luasets_points_clist` and build the `$pt1, $pt2, \dots` reference list.

```

4694 \tl_new:N \l__spintent_spfig_op_tl
4695 \tl_new:N \l__spintent_spfig_intent_tl
4696 \tl_new:N \l__spintent_spfig_desc_tl
4697 \tl_new:N \l__spintent_spfig_symbol_tl

```

```

4698 \seq_new:N \l__spinttent_spfig_pt_seq
4699 \tl_new:N \l__spinttent_spfig_refs_tl
4700 \tl_new:N \l__spinttent_spfig_intent_arg_tl
4701 \int_new:N \l__spinttent_spfig_pt_idx_int
4702 \str_new:N \l__spinttent_geo_print_sym_str

```

(End of definition for `\l__spinttent_spfig_op_tl` and others.)

`\__spinttent_spfig_process:nn`

Receives the short command name (Afig or Pfig) in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’. Calls `\__spinttent_luafun_geo` or `\__spinttent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\l__spinttent_spfig_op_tl` to `área/perímetro`, then delegates to `\__spinttent_spfig_mathml:n`.

```

4703 \cs_new_protected:Nn \__spinttent_spfig_process:nn
4704 {
4705   \str_if_eq:eeTF {#1} { Afig }
4706   { \__spinttent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4707   { \__spinttent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4708   \str_if_eq:VnT \l__spinttent_geo_luaset_error_str { true }
4709   {
4710     \msg_error:nnee { spinttent } { sp #1 -invalid-arg }
4711     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4712   }
4713   \clist_if_empty:NT \l__spinttent_geo_luaset_points_clist
4714   {
4715     \tl_if_empty:cF { \l__spinttent_sp #1 _read_arg_tl }
4716     {
4717       \msg_error:nne { spinttent } { sp #1 -read-arg-key }
4718       { \c_backslash_str sp #1 }
4719     }
4720   }
4721   \str_if_eq:eeTF {#1} { Afig }
4722   { \tl_set:Nn \l__spinttent_spfig_op_tl { área } }
4723   { \tl_set:Nn \l__spinttent_spfig_op_tl { perímetro } }
4724   \str_set:Ne \l__spinttent_geo_print_sym_str { \str_use:c { \l__spinttent_sp #1 _print_sym_str } }
4725   \__spinttent_spfig_mathml:n {#1}
4726 }

```

(End of definition for `\__spinttent_spfig_process:nn`.)

`\__spinttent_spfig_build_simple:n`

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\l__spinttent_geo_final_print` (the visual placed under `\mathcal`’s subscript) and `\l__spinttent_spfig_desc_tl` (the descriptive part, without `área/perímetro`). Receives the short command name in ‘#1’.

With an empty argument, both are cleared for `print-sym auto/ none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

4727 \cs_new_protected:Nn \__spinttent_spfig_build_simple:n
4728 {
4729   \tl_if_empty:nTF { \l__spinttent_geo_luaset_print_tl }
4730   {
4731     \str_case:VnF \l__spinttent_geo_print_sym_str
4732     {
4733       { auto }
4734       {
4735         \tl_clear:N \l__spinttent_geo_final_print_tl
4736         \tl_clear:N \l__spinttent_spfig_desc_tl
4737       }
4738       { none }
4739       {
4740         \tl_clear:N \l__spinttent_geo_final_print_tl
4741         \tl_clear:N \l__spinttent_spfig_desc_tl
4742       }
4743     }
4744     {
4745       \tl_set:Nn \l__spinttent_geo_final_print_tl { \__spinttent_spfig_sym: }
4746       \tl_set:Ne \l__spinttent_spfig_desc_tl
4747       {
4748         \str_if_eq:VnTF \l__spinttent_geo_print_sym_str { triángulo } { cuadrado }
4749       }
4750     }
4751   }
4752 }

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4753     {
4754       \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
4755       {
4756         \str_case:VnF \l__spintent_geo_print_sym_str
4757         {
4758           { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4759           { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4760         }
4761         {
4762           \tl_set:Nn \l__spintent_geo_final_print_tl
4763           {
4764             \__spintent_spfig_sym:
4765             \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4766           }
4767         }
4768         \str_case:VnF \l__spintent_geo_print_sym_str
4769         {
4770           { auto }
4771           {
4772             \tl_set:Ne \l__spintent_spfig_desc_tl
4773             {
4774               \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4775               \l__spintent_geo_luaset_sub_spoken_str
4776             }
4777           }
4778           { none }
4779           {
4780             \tl_set:Ne \l__spintent_spfig_desc_tl
4781             {
4782               \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4783               \l__spintent_geo_luaset_sub_spoken_str
4784             }
4785           }
4786         }
4787         {
4788           \tl_set:Ne \l__spintent_spfig_desc_tl
4789           {
4790             del-
4791             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4792             { triángulo } { cuadrado }
4793             -
4794             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4795             \l__spintent_geo_luaset_sub_spoken_str
4796           }
4797         }
4798       }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4799     {
4800       \str_case:VnF \l__spintent_geo_print_sym_str
4801       {
4802         { auto } { }
4803         { none } { }
4804       }
4805       {
4806         \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4807         { \c_backslash_str sp #1 }
4808       }
4809       \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
4810       \tl_set:Ne \l__spintent_spfig_desc_tl
4811       {
4812         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4813         { \l__spintent_geo_luaset_letra_conector_str - }
4814         \l__spintent_geo_luaset_letra_word_str

```

```

4815         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
4816         {
4817             -
4818             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
4819             \l__spintent_geo_luaset_sub_spoken_str
4820         }
4821     }
4822 }
4823 }

```

The variable `\l__spintent_spfig_intent_tl` is always the `_word-` shape: `\l__spintent_spfig_op_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent` overrides `\l__spintent_spfig_intent_arg_tl` with `_`; with `silent=false` and `concept false`, it becomes `_\l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro*); with `concept true` it is `\l__spintent_spfig_intent_tl`.

```

4824     \tl_set:Nx \l__spintent_spfig_intent_tl
4825     {
4826         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
4827         { _ \l__spintent_spfig_op_tl - }
4828         { _ \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
4829     }
4830     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4831     { \tl_set:Nx \l__spintent_spfig_intent_arg_tl { _- } }
4832     {
4833         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4834         { \tl_set:Nx \l__spintent_spfig_intent_arg_tl { \l__spintent_spfig_intent_tl } }
4835         { \tl_set:Nx \l__spintent_spfig_intent_arg_tl { _ \l__spintent_spfig_desc_tl - } }
4836     }
4837 }

```

(End of definition for `\__spintent_spfig_build_simple:n`.)

`\__spintent_spfig_build_refs:`

Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each vertex by position.

```

4838 \cs_new_protected:Nn \__spintent_spfig_build_refs:
4839 {
4840     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
4841     \tl_clear:N \l__spintent_spfig_refs_tl
4842     \int_zero:N \l__spintent_spfig_pt_idx_int
4843     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4844     {
4845         \int_incr:N \l__spintent_spfig_pt_idx_int
4846         \tl_if_empty:NF \l__spintent_spfig_refs_tl
4847         { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
4848         \tl_put_right:Ne \l__spintent_spfig_refs_tl { $ pt \int_use:N \l__spintent_spfig_pt_idx_int }
4849     }
4850 }

```

(End of definition for `\__spintent_spfig_build_refs:`.)

`\__spintent_spfig_build_concept_vertices:n`

Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\__spintent_spfig_build_refs:n`. `read-arg`, when given, overrides the auto-detected figure name in `\l__spintent_spfig_intent_tl`: with `concept true` it is still prefixed with *área-del-/perímetro-del-*, with `concept false` it is used verbatim. With `read-arg` empty, `\l__spintent_spfig_intent_tl` is simply `\l__spintent_geo_luaset_concept_str`.

```

4851 \cs_new_protected:Nn \__spintent_spfig_build_concept_vertices:n
4852 {
4853     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
4854     { \tl_set:Nx \l__spintent_spfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
4855     {
4856         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4857         {
4858             \tl_set:Nx \l__spintent_spfig_intent_tl
4859             {
4860                 \l__spintent_spfig_op_tl -del-
4861                 \tl_use:c { l__spintent_sp #1 _read_arg_tl }
4862             }
4863         }
4864         {
4865             \tl_set:Nx \l__spintent_spfig_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
4866         }
4867     }

```

Determines `\l__spintenz_spfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\l__spintenz_geo_luaset_poly_sym_str(\triangle/\mdlgwhtsquare)`, or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4868   \str_case:VnF \l__spintenz_geo_print_sym_str
4869   {
4870     { auto }
4871     {
4872       \str_case:Vn \l__spintenz_geo_luaset_poly_sym_str
4873       {
4874         { triangle } { \tl_set:Nn \l__spintenz_spfig_symbol_tl { \triangle } }
4875         { square } { \tl_set:Nn \l__spintenz_spfig_symbol_tl { \mdlgwhtsquare } }
4876       }
4877       \str_if_empty:NT \l__spintenz_geo_luaset_poly_sym_str
4878       { \tl_clear:N \l__spintenz_spfig_symbol_tl }
4879     }
4880     { none }
4881     { \tl_clear:N \l__spintenz_spfig_symbol_tl }
4882   }
4883   {
4884     \str_if_eq:VVTFF \l__spintenz_geo_print_sym_str \l__spintenz_geo_luaset_poly_sym_str
4885     { \tl_set:Nn \l__spintenz_spfig_symbol_tl { \l__spintenz_spfig_sym: } }
4886     {
4887       \msg_error:nne { spintenz } { sp #1 -sym-cardinality-conflict }
4888       { \c_backslash_str sp #1 }
4889     }
4890   }

```

`silent` sets `\l__spintenz_spfig_intent_arg_tl` to `_(refs)`. Otherwise, with `read-arg` empty, `concept` chooses between `\l__spintenz_spfig_intent_tl:prefix(refs)` (prefixed with `prefix` when given) and `_(refs)` alone; with `read-arg` given, it is always `\l__spintenz_spfig_intent_tl:prefix(refs)` — `read-arg` is never silenced by `concept` alone, only by `silent`.

```

4891   \bool_if:cTF { \l__spintenz_sp #1 _silent_bool }
4892   {
4893     \tl_set:Ne \l__spintenz_spfig_intent_arg_tl { _ ( \l__spintenz_spfig_refs_tl ) }
4894   }
4895   {
4896     \tl_if_empty:cTF { \l__spintenz_sp #1 _read_arg_tl }
4897     {
4898       \bool_if:cTF { \l__spintenz_sp #1 _concept_bool }
4899       {
4900         \tl_set:Ne \l__spintenz_spfig_intent_arg_tl
4901         {
4902           \tl_if_empty:cF { \l__spintenz_sp #1 _prefix_tl }
4903           { \tl_use:c { \l__spintenz_sp #1 _prefix_tl } - }
4904           \l__spintenz_spfig_intent_tl :prefix( \l__spintenz_spfig_refs_tl )
4905         }
4906       }
4907       {
4908         \tl_set:Ne \l__spintenz_spfig_intent_arg_tl { _ ( \l__spintenz_spfig_refs_tl ) }
4909       }
4910     }
4911     {
4912       \tl_set:Ne \l__spintenz_spfig_intent_arg_tl
4913       {
4914         \tl_if_empty:cF { \l__spintenz_sp #1 _prefix_tl }
4915         { \tl_use:c { \l__spintenz_sp #1 _prefix_tl } - }
4916         \l__spintenz_spfig_intent_tl :prefix( \l__spintenz_spfig_refs_tl )
4917       }
4918     }
4919   }
4920 }

```

(End of definition for `\l__spintenz_spfig_build_concept_vertices:n`)

`\l__spintenz_spfig_points_loop:n`

Walks `\l__spintenz_spfig_pt_seq` a second time, calling `\l__spintenz_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in `'#1'`.

```

4921   \cs_new_protected:Nn \l__spintenz_spfig_points_loop:n
4922   {
4923     \int_zero:N \l__spintenz_spfig_pt_idx_int
4924     \seq_map_inline:Nn \l__spintenz_spfig_pt_seq

```

```

4925     {
4926       \int_incr:N \l__spintent_sfig_pt_idx_int
4927       \l__spintent_mathmlarg_point:nnn
4928       { pt \int_use:N \l__spintent_sfig_pt_idx_int }
4929       { sp #1 }
4930       { ##1 }
4931     }
4932   }

```

(End of definition for `\l__spintent_sfig_points_loop:n`.)

`\l__spintent_sfig_mathml:n`

Receives the short command name in ‘#1’. Branches on whether `\l__spintent_geo_luaset_points_clist` is empty.

Without vertices, `\l__spintent_sfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` – with `concept=false` the content is typeset directly, with no `intent` at all.

With vertices, `\l__spintent_sfig_build_refs:` then `\l__spintent_sfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside, `\l__spintent_mathcal:n{⟨A | P⟩}` is silenced with `\l__spintent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`’s own symbol); `\l__spintent_sfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\l__spintent_sfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback’s recursive lookup through a noad’s sub field, not just its nucleus.

`\l__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

4933 \cs_new_protected:Nn \l__spintent_sfig_mathml:n
4934 {
4935   \l__spintent_invisible_sep: % (luamml issues #26)
4936   \clist_if_empty:NTF \l__spintent_geo_luaset_points_clist
4937   {
4938     \l__spintent_sfig_build_simple:n {#1}
4939     \MathMLintent { \l__spintent_sfig_intent_arg_tl }
4940     {
4941       \l__spintent_inner_sep:
4942       \str_if_eq:eeTF {#1} { Afig } { \l__spintent_mathcal:n { A } } { \l__spintent_mathcal:n
4943       \tl_if_empty:NF \l__spintent_geo_final_print_tl
4944       {
4945         \c_math_subscript_token { \l__spintent_geo_final_print_tl }
4946       }
4947     }
4948   }
4949   {
4950     \l__spintent_sfig_build_refs:
4951     \l__spintent_sfig_build_concept_vertices:n {#1}
4952     \MathMLintent { \l__spintent_sfig_intent_arg_tl }
4953     {
4954       \l__spintent_inner_sep:
4955       \l__spintent_luafun_mrow_intent_arg:nn
4956       { } { intent = ":silent" }
4957       {
4958         \str_if_eq:eeTF {#1} { Afig } { \l__spintent_mathcal:n { A } } { \l__spintent_mathcal:n
4959       }
4960       \c_math_subscript_token
4961       {
4962         \tl_if_empty:NF \l__spintent_sfig_symbol_tl
4963         {
4964           \l__spintent_luafun_mrow_intent_arg:nn
4965           { } { intent = ":silent" }
4966           { \tl_use:N \l__spintent_sfig_symbol_tl }
4967         }
4968         \l__spintent_sfig_points_loop:n {#1}
4969       }
4970     }
4971   }
4972 }

```

(End of definition for `\l__spintent_sfig_mathml:n`.)

`\l__spintent_sfig_exec:nnn`

The function `\l__spintent_sfig_exec:nnn` receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to `\l__spintent_sfig_process:nn`.

```

4973 \cs_new_protected:Nn \l__spintent_sfig_exec:nnn

```

```

4974 {
4975   \keys_set:nn { spintent / sp #1 } { #2 }
4976   \__spintent_spfig_process:nn { #1 } { #3 }
4977 }

```

(End of definition for __spintent_spfig_exec:nnn.)

\spAfig Both public commands \spAfig and \spPfig enforce math mode, open a local group, and delegate execution to __spintent_spfig_exec:nnn.

```

4978 \NewDocumentCommand \spAfig { 0{} d() }
4979 {
4980   \mode_if_math:TF
4981   {
4982     \group_begin:
4983     \attribute \__spintent_scope_attr_int = 1
4984     \tl_if_novalue:nTF {#2}
4985     { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
4986     { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
4987     \group_end:
4988   }
4989   { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
4990 }
4991 \NewDocumentCommand \spPfig { 0{} d() }
4992 {
4993   \mode_if_math:TF
4994   {
4995     \group_begin:
4996     \attribute \__spintent_scope_attr_int = 1
4997     \tl_if_novalue:nTF {#2}
4998     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
4999     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
5000     \group_end:
5001   }
5002   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
5003 }

```

(End of definition for \spAfig and \spPfig. These functions are documented on page 25.)

11.45 The commands \sprad and \spdiam

The user-level commands \sprad and \spdiam provide a semantic interface for the radius and diameter of a circle.

11.45.1 Definition of variables for \sprad and \spdiam

__spintent_sp_rad_diam_letra_tl and __spintent_sp_rad_diam_word_tl are scratch variables used by __spintent_sp_rad_diam_print:n. __spintent_sp_rad_diam_read_arg_full_tl is used by __spintent_sp_rad_diam_read_arg_print:n to hold the text to voice when read-arg is given.

```

5004 \tl_new:N \__spintent_sp_rad_diam_letra_tl
5005 \tl_new:N \__spintent_sp_rad_diam_word_tl
5006 \tl_new:N \__spintent_sp_rad_diam_read_arg_full_tl

```

(End of definition for __spintent_sp_rad_diam_letra_tl, __spintent_sp_rad_diam_word_tl, and __spintent_sp_rad_diam_read_arg_full_tl.)

11.45.2 Definition of keys for \sprad and \spdiam

We define the shared <keys> read-cmd, upper-case, sub-index, read-sub, read-arg, concept, silent and print-sym.

```

5007 \clist_map_inline:nn { rad , diam }
5008 {
5009   \keys_define:nn { spintent / sp #1 }
5010   {
5011     read-cmd .choices:nn = { school , mathcat }
5012     {
5013       \int_case:nn { \_keys_choice_int }
5014       {
5015         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
5016         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
5017       }
5018     },
5019     read-cmd .initial:n = school,
5020     read-cmd .value_required:n = true,

```

```

5021 read-cmd / unknown .code:n =
5022   \msg_error:nneee { spintent } { unknown-choice }
5023   { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5024 upper-case .bool_set:c = { l__spintent_sp #1 _upper_case_bool },
5025 upper-case .initial:n = false,
5026 upper-case .value_required:n = true,
5027 sub-index .code:n =
5028   { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
5029 sub-index .initial:n = ,
5030 sub-index .value_required:n = true,
5031 read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
5032 read-sub .initial:n = false,
5033 read-sub .value_required:n = true,
5034 read-arg .code:n =
5035   { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
5036 read-arg .initial:n = ,
5037 read-arg .value_required:n = true,
5038 concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
5039 concept .initial:n = true,
5040 concept .value_required:n = true,
5041 silent .bool_set:c = { l__spintent_sp #1 _silent_bool },
5042 silent .initial:n = false,
5043 silent .value_required:n = true,
5044 print-sym .bool_set:c = { l__spintent_sp #1 _print_sym_bool },
5045 print-sym .initial:n = false,
5046 print-sym .value_required:n = true,
5047 unknown .code:n =
5048   {
5049     \str_set:Nn \l__spintent_command_name_str { sp #1 }
5050     \__spintent_unknown_keys_cmd:n {##1}
5051   },
5052 }
5053 }

```

🔗 The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

11.45.3 Internal functions for `\sprad` and `\spdiam`

`\__spintent_sp_rad_diam_print:n`

The function `\__spintent_sp_rad_diam_print:n` sets `\l__spintent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per `upper-case`, and, for `\spdiam` with `print-sym true`, silently typesets `\diameter` first. Receives the short command name in ‘`#1`’ (`rad/diam`).

```

5054 \cs_new_protected:Nn \__spintent_sp_rad_diam_print:n
5055 {
5056   \bool_if:cTF { l__spintent_sp #1 _upper_case_bool }
5057   {
5058     \str_if_eq:eeTF {#1} { rad }
5059     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { R } }
5060     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { D } }
5061   }
5062   {
5063     \str_if_eq:eeTF {#1} { rad }
5064     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { r } }
5065     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { d } }
5066   }
5067   \__spintent_invisible_sep:
5068   \str_if_eq:eeT {#1} { diam }
5069   {
5070     \bool_if:NT \l__spintent_spdiam_print_sym_bool
5071     { \MathMLintent { :silent } { \diameter } }
5072   }

```

With `silent true`, the intent is `_` — the letter alone through the shared queue when there is no `sub-index`, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

5073   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5074   {
5075     \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
5076     {
5077       \__spintent_luafun_mrow_intent_arg:nn
5078       { } { intent = "_" }
5079       { \l__spintent_sp_rad_diam_letra_tl }
5080     }

```

```

5081     {
5082       \MathMLintent { _- }
5083       {
5084         \__spintenc_inner_sep:
5085         \l__spintenc_sp_rad_diam_letra_tl
5086         \c_math_subscript_token
5087         {
5088           \MathMLintent { :silent }
5089           { \str_use:c { l__spintenc_sp #1 _sub_index_str } }
5090         }
5091       }
5092     }
5093   }

```

Otherwise, with `concept true` (the default), it verbalizes radio/diámetro (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing sub- $\langle value \rangle$ or $\langle value \rangle$ alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

5094   {
5095     \tl_if_empty:cTF { l__spintenc_sp #1 _read_arg_tl }
5096     {
5097       \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
5098       {
5099         \str_if_eq:eeTF {#1} { rad }
5100         { \tl_set:Nn \l__spintenc_sp_rad_diam_word_tl { radio } }
5101         { \tl_set:Nn \l__spintenc_sp_rad_diam_word_tl { diámetro } }
5102         \str_if_empty:cTF { l__spintenc_sp #1 _sub_index_str }
5103         {
5104           \__spintenc_luafun_mrow_intent_arg:nn
5105           { } { intent = "_ \l__spintenc_sp_rad_diam_word_tl :nofix" }
5106           { \l__spintenc_sp_rad_diam_letra_tl }
5107         }
5108         {
5109           \str_if_eq:eeTF
5110           { \str_use:c { l__spintenc_sp #1 _mode_str } } { school }
5111           {
5112             \MathMLintent
5113             {
5114               _ \l__spintenc_sp_rad_diam_word_tl -
5115               \bool_if:cT { l__spintenc_sp #1 _read_sub_bool } { sub- }
5116               \str_use:c { l__spintenc_sp #1 _sub_index_str }
5117               :prefix
5118             }
5119             {
5120               \__spintenc_inner_sep:
5121               \l__spintenc_sp_rad_diam_letra_tl
5122               \c_math_subscript_token
5123               {
5124                 \MathMLintent { :silent }
5125                 { \str_use:c { l__spintenc_sp #1 _sub_index_str } }
5126               }
5127             }
5128           }
5129           {
5130             \__spintenc_luafun_mrow_intent_arg:nn
5131             { } { intent = "_ \l__spintenc_sp_rad_diam_word_tl :nofix" }
5132             { \l__spintenc_sp_rad_diam_letra_tl }
5133             \c_math_subscript_token
5134             { \str_use:c { l__spintenc_sp #1 _sub_index_str } }
5135           }
5136         }
5137       }

```

With `concept=false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd=mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by us.

```

5138   {
5139     \str_if_empty:cTF { l__spintenc_sp #1 _sub_index_str }

```

```

5140         {
5141             \str_if_eq:eeTF
5142             { \str_use:c { l__spintenz_sp #1 _mode_str } } { school }
5143             {
5144                 \__spintenz_luafun_mrow_intenz_arg:nn
5145                 { } { intenz = "_ \__spintenz_sp_rad_diam_letra_tl -" }
5146                 { \__spintenz_sp_rad_diam_letra_tl }
5147             }
5148             { \__spintenz_sp_rad_diam_letra_tl }
5149         }
5150     {
5151         \str_if_eq:eeTF
5152         { \str_use:c { l__spintenz_sp #1 _mode_str } } { school }
5153         {
5154             \MathMLintenz
5155             {
5156                 _ \__spintenz_sp_rad_diam_letra_tl -
5157                 \bool_if:cT { l__spintenz_sp #1 _read_sub_bool } { sub- }
5158                 \str_use:c { l__spintenz_sp #1 _sub_index_str }
5159                 :prefix
5160             }
5161             {
5162                 \__spintenz_inner_sep:
5163                 \__spintenz_sp_rad_diam_letra_tl
5164                 \c_math_subscript_token
5165                 {
5166                     \MathMLintenz { :silent }
5167                     { \str_use:c { l__spintenz_sp #1 _sub_index_str } }
5168                 }
5169             }
5170         }
5171         {
5172             \__spintenz_sp_rad_diam_letra_tl
5173             \c_math_subscript_token
5174             { \str_use:c { l__spintenz_sp #1 _sub_index_str } }
5175         }
5176     }
5177 }
5178 }
5179 { \__spintenz_sp_rad_diam_read_arg_print:n {#1} }
5180 }
5181 }

```

(End of definition for __spintenz_sp_rad_diam_print:n.)

__spintenz_sp_rad_diam_read_arg_print:n

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true` (the default), `\__spintenz_sp_rad_diam_read_arg_full_tl` becomes `radio/diámetro` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively, regardless of `read-cmd`: with no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintenz`.

```

5182 \cs_new_protected:Nn \__spintenz_sp_rad_diam_read_arg_print:n
5183 {
5184     \bool_if:cTF { l__spintenz_sp #1 _concept_bool }
5185     {
5186         \str_if_eq:eeTF {#1} { rad }
5187         {
5188             \tl_set:Nc \__spintenz_sp_rad_diam_read_arg_full_tl
5189             { radio- \tl_use:c { l__spintenz_sp #1 _read_arg_tl } }
5190         }
5191         {
5192             \tl_set:Nc \__spintenz_sp_rad_diam_read_arg_full_tl
5193             { diámetro- \tl_use:c { l__spintenz_sp #1 _read_arg_tl } }
5194         }
5195     }
5196     {
5197         \tl_set_eq:Nc \__spintenz_sp_rad_diam_read_arg_full_tl
5198         { l__spintenz_sp #1 _read_arg_tl }
5199     }
5200     \str_if_empty:cTF { l__spintenz_sp #1 _sub_index_str }
5201     {
5202         \__spintenz_luafun_mrow_intenz_arg:nn

```

```

5203     { } { intent = "_ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix" }
5204     { \l__spintent_sp_rad_diam_letra_tl }
5205   }
5206   {
5207     \MathMLintent
5208     { _ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix }
5209     {
5210       \__spintent_inner_sep:
5211       \l__spintent_sp_rad_diam_letra_tl
5212       \c_math_subscript_token
5213       {
5214         \MathMLintent { :silent }
5215         { \str_use:c { \l__spintent_sp #1 _sub_index_str } }
5216       }
5217     }
5218   }
5219 }

```

(End of definition for `\__spintent_sp_rad_diam_read_arg_print:n`.)

`\__spintent_sp_rad_diam_exec:nn` The function `\__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘#1’ and the optional *(keys)* passed in ‘#2’ then delegating the rendering to `\__spintent_sp_rad_diam_print:n`.

```

5220 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
5221 {
5222   \keys_set:nn { spintent / sp #1 } {#2}
5223   \__spintent_sp_rad_diam_print:n {#1}
5224 }

```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`.)

`\sprad` Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```

5225 \NewDocumentCommand \sprad { 0{} }
5226 {
5227   \mode_if_math:TF
5228   {
5229     \group_begin:
5230     \attribute \l__spintent_scope_attr_int = 1
5231     \__spintent_sp_rad_diam_exec:nn { rad } {#1}
5232     \group_end:
5233   }
5234   { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
5235 }
5236 \NewDocumentCommand \spdiam { 0{} }
5237 {
5238   \mode_if_math:TF
5239   {
5240     \group_begin:
5241     \attribute \l__spintent_scope_attr_int = 1
5242     \__spintent_sp_rad_diam_exec:nn { diam } {#1}
5243     \group_end:
5244   }
5245   { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
5246 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page 28.)

11.46 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside`/`\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.46.1 Definition of variables for `\spcirc`

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\l__spintent_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, *segment*, or an empty string when no radius was given. `\l__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

5247 \str_new:N \l__spintent_geo_luaset_circ_radio_type_str
5248 \str_new:N \l__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\l__spintent_geo_luaset_circ_radio_type_str` and `\l__spintent_geo_luaset_circ_radio_raw_str`.)

The variable `\l__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\l__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (`usual` or `odot`). The variable `\l__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\l__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\l__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\l__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article`: to `la` or `el`, matching the grammatical gender of the chosen word.

```
5249 \str_new:N \l__spintent_spcirc_mode_str
5250 \tl_new:N \l__spintent_spcirc_prefix_tl
5251 \str_new:N \l__spintent_spcirc_print_sym_str
5252 \str_new:N \l__spintent_spcirc_read_sty_str
5253 \str_new:N \l__spintent_spcirc_radio_sty_str
5254 \str_new:N \l__spintent_spcirc_sub_index_str
5255 \str_new:N \l__spintent_spcirc_articulo_str
5256 \tl_new:N \l__spintent_spcirc_read_arg_tl
```

(End of definition for `\l__spintent_spcirc_mode_str` and others.)

Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (`circunferencia/círculo` or the `read-arg` override) and the final literal `intent` string.

```
5257 \tl_new:N \l__spintent_spcirc_word_tl
```

(End of definition for `\l__spintent_spcirc_word_tl`.)

11.46.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg` Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```
5258 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
5259 {
5260   Invalid~argument~format~in~\token_to_str:N \spcirc:~'!#1'~\msg_line_context:.
5261 }
5262 {
5263   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
5264 }
```

(End of definition for `spcirc-invalid-arg`.)

11.46.3 Definition of keys for `\spcirc`

The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «*la circunferencia*», `circle` for «*el círculo*». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (`side`, via `\spside`) or its *measure* (`m-side`, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```
5265 \keys_define:nn { spintent / spcirc }
5266 {
5267   read-cmd      .choices:nn = { school , mathcat }
5268   {
5269     \int_case:nn { \l_keys_choice_int }
5270     {
5271       {1}
5272       {
5273         \str_set:Nn \l__spintent_spcirc_mode_str { school }
5274         \keys_set:nn { spintent / spside } { read-cmd = school }
5275         \keys_set:nn { spintent / spmside } { read-cmd = school }
5276       }
5277       {2}
5278       {
5279         \str_set:Nn \l__spintent_spcirc_mode_str { mathcat }
5280         \keys_set:nn { spintent / spside } { read-cmd = mathcat }
5281         \keys_set:nn { spintent / spmside } { read-cmd = mathcat }
5282       }
5283     }
5284   },
5285   read-cmd      .initial:n = school,
5286   read-cmd      .value_required:n = true,
```

```

5287     prefix      .code:n =
5288     { \__spintent_sanitiz_abbrev:Nn \__spintent_spcirc_prefix_tl {#1} },
5289     prefix      .initial:n = ,
5290     prefix      .value_required:n = true,
5291     print-sym   .choices:nn = { usual , odot }
5292     {
5293         \int_case:nn { \l_keys_choice_int }
5294         {
5295             {1} { \str_set:Nn \__spintent_spcirc_print_sym_str { usual } }
5296             {2} { \str_set:Nn \__spintent_spcirc_print_sym_str { odot } }
5297         }
5298     },
5299     print-sym   .initial:n = odot,
5300     print-sym   .value_required:n = true,
5301     read-sty    .choices:nn = { circun , circle }
5302     {
5303         \int_case:nn { \l_keys_choice_int }
5304         {
5305             {1} { \str_set:Nn \__spintent_spcirc_read_sty_str { circun } }
5306             {2} { \str_set:Nn \__spintent_spcirc_read_sty_str { circle } }
5307         }
5308     },
5309     read-sty    .initial:n = circun,
5310     read-sty    .value_required:n = true,
5311     diameter    .bool_set:N = \__spintent_spcirc_diameter_bool,
5312     diameter    .initial:n = false,
5313     diameter    .value_required:n = true,
5314     radio-sty   .choices:nn = { side , mside }
5315     {
5316         \int_case:nn { \l_keys_choice_int }
5317         {
5318             {1} { \str_set:Nn \__spintent_spcirc_radio_sty_str { side } }
5319             {2} { \str_set:Nn \__spintent_spcirc_radio_sty_str { mside } }
5320         }
5321     },
5322     radio-sty   .initial:n = mside,
5323     radio-sty   .value_required:n = true,
5324     sub-index   .code:n =
5325     { \str_set:Nn \__spintent_spcirc_sub_index_str {#1} },
5326     sub-index   .initial:n = ,
5327     sub-index   .value_required:n = true,
5328     read-sub    .bool_set:N = \__spintent_spcirc_read_sub_bool,
5329     read-sub    .initial:n = false,
5330     read-sub    .value_required:n = true,
5331     read-arg    .code:n =
5332     { \__spintent_sanitiz_abbrev:Nn \__spintent_spcirc_read_arg_tl {#1} },
5333     read-arg    .initial:n = ,
5334     read-arg    .value_required:n = true,
5335     silent      .bool_set:N = \__spintent_spcirc_silent_bool,
5336     silent      .initial:n = false,
5337     silent      .value_required:n = true,
5338     concept     .bool_set:N = \__spintent_spcirc_concept_bool,
5339     concept     .initial:n = true,
5340     concept     .value_required:n = true,
5341     spside      .meta:nn = { spintent / spside } { #1 },
5342     spside      .value_required:n = true,
5343     spmside     .meta:nn = { spintent / spmside } { #1 },
5344     spmside     .value_required:n = true,
5345     unknown     .code:n =
5346     {
5347         \str_set:Nn \__spintent_command_name_str { spcirc }
5348         \__spintent_unknown_keys_cmd:n {#1}
5349     },
5350 }

```

(End of definition for read-cmd and others.)

11.46.4 Internal functions for \spcirc

`\__spintent_spcirc_has_arg_bool` `\__spintent_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\__spintent_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional `sub-index` suffix) paired with the symbol.

`\__spintent_spcirc_intent_arg_tl`

`\__spintent_spcirc_connector_tl`

`\__spintent_spcirc_connector_arg_tl`

©2026 by Pablo González L

`\l__spintent_spcirc_connector_tl` holds de-centro-en, paired with the `(center)` group; the center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. O_{1} , as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word’s final wrapped form; a third, `\l__spintent_spcirc_radio_arg_tl`, is spoken directly on the `,` delimiter after the center, replacing its own reading rather than pairing with a group.

```
5351 \bool_new:N \l__spintent_spcirc_has_arg_bool
5352 \tl_new:N \l__spintent_spcirc_intent_tl
5353 \tl_new:N \l__spintent_spcirc_intent_arg_tl
5354 \tl_new:N \l__spintent_spcirc_connector_tl
5355 \tl_new:N \l__spintent_spcirc_connector_arg_tl
5356 \tl_new:N \l__spintent_spcirc_radio_arg_tl
```

(End of definition for `\l__spintent_spcirc_has_arg_bool` and others.)

`\__spintent_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```
5357 \cs_new_protected:Nn \__spintent_spcirc_sym:
5358 {
5359   \str_if_eq:NnTF \l__spintent_spcirc_print_sym_str { odot }
5360   {
5361     \group_begin:
5362     \textstyle\bigodot
5363     \group_end:
5364   }
5365   { C }
5366 }
```

(End of definition for `\__spintent_spcirc_sym:`.)

`\__spintent_spcirc_word_and_article:` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```
5367 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
5368 {
5369   \tl_if_empty:NnTF \l__spintent_spcirc_read_arg_tl
5370   {
5371     \str_if_eq:NnTF \l__spintent_spcirc_read_sty_str { circun }
5372     {
5373       \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia }
5374       \str_set:Nn \l__spintent_spcirc_articulo_str { la }
5375     }
5376     {
5377       \tl_set:Nn \l__spintent_spcirc_word_tl { círculo }
5378       \str_set:Nn \l__spintent_spcirc_articulo_str { el }
5379     }
5380   }
5381   {
5382     \tl_set:Nn \l__spintent_spcirc_word_tl { \l__spintent_spcirc_read_arg_tl }
5383     \str_if_eq:NnTF \l__spintent_spcirc_read_sty_str { circun }
5384     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5385     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5386   }
5387 }
```

(End of definition for `\__spintent_spcirc_word_and_article:`.)

`\__spintent_spcirc_radio_print:` Dispatches on `\l__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```
5388 \cs_new_protected:Nn \__spintent_spcirc_radio_print:
5389 {
5390   \str_case:Nn \l__spintent_geo_luaset_circ_radio_type_str
5391   {
5392     { number } { \exp_args:Ne \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
5393     { measure } { \exp_args:Ne \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
5394     { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
5395     { segment }
5396     {
5397       \str_if_eq:NnTF \l__spintent_spcirc_radio_sty_str { side }
5398       { \exp_args:Ne \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5399       { \exp_args:Ne \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5400     }
5401   }
5402 }
```

(End of definition for `\__spintent_spcirc_radio_print:`.)

`\__spintent_spcirc_build_concept:` Builds two words: `\l__spintent_spcirc_intent_tl` (via `\__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (de-centro-en), paired with the (center) group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center’s own text: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

```

5403 \cs_new_protected:Nn \__spintent_spcirc_build_concept:
5404 {
5405   \__spintent_spcirc_word_and_article:
5406   \tl_set:Nc \l__spintent_spcirc_intent_tl { \l__spintent_spcirc_word_tl }
5407   \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5408   {
5409     \tl_put_right:Nc \l__spintent_spcirc_intent_tl
5410     {
5411       -
5412       \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
5413       \l__spintent_spcirc_sub_index_str
5414     }
5415   }
5416   \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }

```

`silent true` overrides `\l__spintent_spcirc_intent_arg_tl/\l__spintent_spcirc_radio_arg_tl` with `_-` and `\l__spintent_spcirc_connector_arg_tl` with `_($pt)` (the center is always read), regardless of `read-arg`.

```

5417   \bool_if:NTF \l__spintent_spcirc_silent_bool
5418   {
5419     \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5420     \tl_set:Nc \l__spintent_spcirc_connector_arg_tl { _($ pt ) }
5421     \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5422   }

```

With `concept true`, `\l__spintent_spcirc_intent_arg_tl` is `\l__spintent_spcirc_intent_tl:prefix` (bare — the group it wraps has nothing to reference, only a silenced symbol), prefixed with `prefix` when given; `\l__spintent_spcirc_connector_arg_tl` is `\l__spintent_spcirc_connector_tl:prefix($pt)`; and `\l__spintent_spcirc_radio_arg_tl` — spoken directly on the , delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\__spintent_spcirc_radio_print:`’s independent reading follows as the next sibling.

```

5423   {
5424     \bool_if:NTF \l__spintent_spcirc_concept_bool
5425     {
5426       \tl_set:Nc \l__spintent_spcirc_intent_arg_tl
5427       {
5428         \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
5429         { \tl_use:N \l__spintent_spcirc_prefix_tl - }
5430         \l__spintent_spcirc_intent_tl :prefix
5431       }
5432       \tl_set:Nc \l__spintent_spcirc_connector_arg_tl
5433       { \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5434       \bool_if:NTF \l__spintent_spcirc_diameter_bool
5435       { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5436       { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5437     }

```

With `concept false` and `read-arg` empty, the same `_-/($pt)` silencing applies as `silent`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

5438   {
5439     \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5440     {
5441       \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5442       \tl_set:Nc \l__spintent_spcirc_connector_arg_tl { _($ pt ) }
5443       \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5444     }
5445     {
5446       \tl_set:Nc \l__spintent_spcirc_intent_arg_tl { \l__spintent_spcirc_intent_tl }
5447       \tl_set:Nc \l__spintent_spcirc_connector_arg_tl
5448       { \l__spintent_spcirc_connector_tl :prefix( $ pt ) }

```

```

5449         \bool_if:NTF \l__spintent_spcirc_diameter_bool
5450         { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5451         { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5452     }
5453 }
5454 }
5455 }

```

(End of definition for `\__spintent_spcirc_build_concept:`)

`\__spintent_spcirc_process:n` Receives the `d()` argument in ‘#1’, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

5456 \cs_new_protected:Nn \__spintent_spcirc_process:n
5457 {
5458     \tl_if_empty:NTF {#1}
5459     {
5460         \bool_set_false:N \l__spintent_spcirc_has_arg_bool
5461     }
5462     {
5463         \bool_set_true:N \l__spintent_spcirc_has_arg_bool
5464         \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5465         \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5466         {
5467             \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5468         }
5469     }
5470     \__spintent_spcirc_mathml:
5471 }

```

(End of definition for `\__spintent_spcirc_process:n`)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\l__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\l__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\l__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol’s own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`’s symbol and `\sparc`’s accent. When a radius is present, the `,` delimiter after the center carries `\l__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_mrow_in` separately — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` callback’s recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter’s own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

5472 \cs_new_protected:Nn \__spintent_spcirc_mathml:
5473 {
5474     \__spintent_invisible_sep: % (luamml issues #26)
5475     \__spintent_spcirc_build_concept:
5476     \MathMLintent { \l__spintent_spcirc_intent_arg_tl }
5477     {
5478         \__spintent_inner_sep:
5479         \__spintent_luafun_mrow_intent_arg:nn
5480         { } { intent = "silent" }
5481         { \__spintent_spcirc_sym: }
5482         \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5483         {

```

```

5484         \c_math_subscript_token
5485         {
5486             \__spintent_luafun_mrow_intent_arg:nn
5487             { } { intent = ":silent" }
5488             { \__spintent_spcirc_sub_index_str }
5489         }
5490     }
5491 }
5492 \bool_if:NT \__spintent_spcirc_has_arg_bool
5493 {
5494     \str_if_empty:NTF \__spintent_geo_luaset_circ_radio_type_str
5495     {
5496         \MathMLLintent { \__spintent_spcirc_connector_arg_tl }
5497         {
5498             \__spintent_inner_sep:
5499             \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5500             \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
5501             \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
5502         }
5503     }
5504     {
5505         \MathMLLintent { \__spintent_spcirc_connector_arg_tl }
5506         {
5507             \__spintent_inner_sep:
5508             \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5509             \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
5510             )
5511         \MathMLLintent { \__spintent_spcirc_radio_arg_tl } { , }
5512         \__spintent_spcirc_radio_print:
5513         \MathMLLintent { :silent } { \rparen }
5514     }
5515 }
5516 }

```

(End of definition for `\__spintent_spcirc_mathml:`)

`\__spintent_spcirc_exec:nn` The function `\__spintent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spcirc_print:n`.

```

5517 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
5518 {
5519     \keys_set:nn { spintent / spcirc } { #1 }
5520     \__spintent_spcirc_process:n { #2 }
5521 }

```

(End of definition for `\__spintent_spcirc_exec:nn`.)

`\spcirc` The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcld/\spmcmm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

5522 \NewDocumentCommand \spcirc { 0{ } d() }
5523 {
5524     \mode_if_math:TF
5525     {
5526         \group_begin:
5527         \attribute \__spintent_scope_attr_int = 1
5528         \tl_if_novalue:nTF {#2}
5529         { \__spintent_spcirc_exec:nn { #1 } { } }
5530         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
5531     \group_end:
5532     }
5533     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
5534 }

```

(End of definition for `\spcirc`. This function is documented on page 29.)

11.47 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.47.1 Definition of variables for \spAcirc and \spPcirc

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```
\__spinttent_spAcirc_mode_str
\__spinttent_spPcirc_mode_str
\__spinttent_spAcirc_print_sym_str
\__spinttent_spPcirc_print_sym_str
\__spinttent_spAcirc_read_sty_str
\__spinttent_spPcirc_read_sty_str
\__spinttent_spAcirc_radio_sty_str
\__spinttent_spPcirc_radio_sty_str
\__spinttent_spAcirc_sub_index_str
\__spinttent_spPcirc_sub_index_str
\__spinttent_spAcirc_prefix_tl
\__spinttent_spPcirc_prefix_tl
\__spinttent_spAcirc_read_arg_tl
\__spinttent_spPcirc_read_arg_tl
\__spinttent_geo_circfig_mode_str
```

```
5535 \clist_map_inline:nn { Acirc , Pcirc }
5536 {
5537   \str_new:c { \__spinttent_sp #1 _mode_str }
5538   \str_new:c { \__spinttent_sp #1 _print_sym_str }
5539   \str_new:c { \__spinttent_sp #1 _read_sty_str }
5540   \str_new:c { \__spinttent_sp #1 _radio_sty_str }
5541   \str_new:c { \__spinttent_sp #1 _sub_index_str }
5542   \tl_new:c { \__spinttent_sp #1 _prefix_tl }
5543   \tl_new:c { \__spinttent_sp #1 _read_arg_tl }
5544 }
```

(End of definition for __spinttent_spAcirc_mode_str and others.)

The variable `\__spinttent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (`bare`).

```
5545 \str_new:N \__spinttent_geo_circfig_mode_str
```

(End of definition for __spinttent_geo_circfig_mode_str.)

11.47.2 Definition of error messages for \spAcirc and \spPcirc

```
spAcirc-invalid-arg
spPcirc-invalid-arg
```

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```
5546 \clist_map_inline:nn { Acirc , Pcirc }
5547 {
5548   \msg_new:nnnn { spinttent } { sp #1 -invalid-arg }
5549   {
5550     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
5551   }
5552   {
5553     See~the~documentation~of~##1~for~the~accepted~argument.
5554   }
}
```

(End of definition for `spAcirc-invalid-arg` and `spPcirc-invalid-arg`.)

11.47.3 Definition of keys for \spAcirc and \spPcirc

```
read-cmd
prefix
print-sym
read-sty
concept
sub-index
read-sub
read-arg
silent
unknown
```

We define the shared (*keys*) `read-cmd`, `prefix`, `print-sym`, `read-sty`, `concept`, `sub-index`, `read-sub`, `read-arg` and `silent`.

```
5555 \keys_define:nn { spinttent / sp #1 }
5556 {
5557   read-cmd .choices:nn = { school , mathcat }
5558   {
5559     \int_case:nn { \l_keys_choice_int }
5560     {
5561       {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
5562       {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
5563     }
5564   },
5565   read-cmd .initial:n = school,
5566   read-cmd .value_required:n = true,
5567   read-cmd / unknown .code:n =
5568     \msg_error:nneee { spinttent } { unknown-choice }
5569     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5570   prefix .code:n =
5571     { \__spinttent_sanitizе_affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
5572   prefix .initial:n = ,
5573   prefix .value_required:n = true,
5574   print-sym .choices:nn = { usual , odot }
5575   {
5576     \int_case:nn { \l_keys_choice_int }
5577     {
5578       {1} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { usual } }
5579       {2} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { odot } }
5580     }
5581   },
5582   print-sym .initial:n = odot,
5583   print-sym .value_required:n = true,
5584   print-sym / unknown .code:n =
5585     \msg_error:nneee { spinttent } { unknown-choice }
```

```

5586         { print-sym } { usual,~odot } { \exp_not:n {##1} },
5587 read-sty                .choices:nn = { circun , circle }
5588     {
5589         \int_case:nn { \l_keys_choice_int }
5590         {
5591             {1} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { circun } }
5592             {2} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { circle } }
5593         }
5594     },
5595 read-sty                .initial:n = circun,
5596 read-sty                .value_required:n = true,
5597 concept                .bool_set:c = { \l__spintent_sp #1 _concept_bool },
5598 concept                .initial:n = true,
5599 concept                .value_required:n = true,
5600 sub-index               .code:n =
5601     { \str_set:cn { \l__spintent_sp #1 _sub_index_str } {##1} },
5602 sub-index               .initial:n = ,
5603 sub-index               .value_required:n = true,
5604 read-sub                .bool_set:c = { \l__spintent_sp #1 _read_sub_bool },
5605 read-sub                .initial:n = false,
5606 read-sub                .value_required:n = true,
5607 read-arg                .code:n =
5608     { \l__spintent_sanitize_affix:cn { \l__spintent_sp #1 _read_arg_tl } {##1} },
5609 read-arg                .initial:n = ,
5610 read-arg                .value_required:n = true,
5611 silent                  .bool_set:c = { \l__spintent_sp #1 _silent_bool },
5612 silent                  .initial:n = false,
5613 silent                  .value_required:n = true,
5614 unknown                 .code:n =
5615     {
5616         \str_set:Nn \l__spintent_command_name_str { sp #1 }
5617         \l__spintent_unknown_keys_cmd:n {##1}
5618     },
5619 }
5620 }

```

(End of definition for read-cmd and others.)

`\l__spintent_spcircfig_short_bool` `\l__spintent_spcircfig_op_tl` `\l__spintent_spcircfig_intent_tl` `\l__spintent_spcircfig_intent_arg_tl`

`\l__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\l__spintent_luafun_geo_afig_parse_and_set:n/\l__spintent_luafun_geo_pfig_parse_and_set:n`) the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\l__spintent_spcircfig_op_tl` holds área or perímetro. `\l__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\l__spintent_spcircfig_intent_phrase` (via `\l__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. `O_{1}`), as in the Euler circle) is never an issue.

```

5621 \bool_new:N \l__spintent_spcircfig_short_bool
5622 \tl_new:N   \l__spintent_spcircfig_op_tl
5623 \tl_new:N   \l__spintent_spcircfig_del_word_tl
5624 \tl_new:N   \l__spintent_spcircfig_intent_tl
5625 \tl_new:N   \l__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\l__spintent_spcircfig_short_bool` and others.)

`\l__spintent_spcircfig_sym:n` Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘#1’.

```

5626 \cs_new_protected:Nn \l__spintent_spcircfig_sym:n
5627 {
5628     \str_if_eq:vnTF { \l__spintent_sp #1 _print_sym_str } { odot }
5629     { \bigodot }
5630     { C }
5631 }

```

(End of definition for `\l__spintent_spcircfig_sym:n`.)

`\l__spintent_spcircfig_word_and_article:n` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘#1’.

```

5632 \cs_new_protected:Nn \l__spintent_spcircfig_word_and_article:n
5633 {
5634     \str_if_eq:vnTF { \l__spintent_sp #1 _read_sty_str } { circun }
5635     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }

```

```

5636     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5637 \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5638 {
5639     \str_if_eq:vnTF { l__spintent_sp #1 _read_sty_str } { circun }
5640     { \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia } }
5641     { \tl_set:Nn \l__spintent_spcirc_word_tl { círculo } }
5642 }
5643 {
5644     \tl_set:Ne \l__spintent_spcirc_word_tl
5645     {
5646         \tl_use:c { l__spintent_sp #1 _read_arg_tl }
5647     }
5648 }
5649 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n` Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`’s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in ‘#1’.

```

5650 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n
5651 {
5652     \__spintent_spcircfig_word_and_article:n {#1}
5653     \str_if_eq:VnTF \l__spintent_spcirc_articulo_str { el }
5654     {
5655         \tl_set:Ne \l__spintent_spcircfig_del_word_tl { del- \l__spintent_spcirc_word_tl }
5656     }
5657     {
5658         \tl_set:Ne \l__spintent_spcircfig_del_word_tl { de-la- \l__spintent_spcirc_word_tl }
5659     }
5660 }

```

(End of definition for `\__spintent_spcircfig_del_word:n`.)

`\__spintent_spcircfig_build_short:n` Handles the short number/letter case: builds `\l__spintent_spcircfig_intent_arg_tl` as `\l__spintent_spcircfig_intent_arg_tl` (*ber word*), wrapped in bare `:prefix` (no reference — there is nothing to point to), or `-` when `silent` is `true` or `concept` is `false`. Receives the short command name in ‘#1’.

```

5661 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
5662 {
5663     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5664     {
5665         \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { - }
5666     }
5667     {
5668         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5669         {
5670             \__spintent_spcircfig_del_word:n {#1}
5671             \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5672             {
5673                 \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl -
5674                 \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5675                 \l__spintent_geo_luaset_sub_spoken_str :prefix
5676             }
5677         }
5678         {
5679             \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { - }
5680         }
5681     }
5682 }

```

(End of definition for `\__spintent_spcircfig_build_short:n`.)

`\__spintent_spcircfig_finish_intent:nn` Appends an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` to `\l__spintent_spcircfig_intent_arg_tl` — shared by both branches of `\__spintent_spcircfig_build_concept:nn`, after either one has set the word itself. Receives the short command name in ‘#1’ and the argument in ‘#2’.

```

5683 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
5684 {
5685     \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5686     {
5687         \tl_put_right:Ne \l__spintent_spcircfig_intent_tl
5688         {

```

```

5689         -
5690         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5691         \str_use:c { l__spintent_sp #1 _sub_index_str }
5692     }
5693 }
5694 \tl_if_empty:nF {#2}
5695 {
5696     \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
5697 }
5698 }

```

(End of definition for `\l__spintent_spcircfig_finish_intent:nn`.)

`\l__spintent_spcircfig_build_concept:nn`

Builds `\l__spintent_spcircfig_intent_arg_tl`. `_-` when `silent` is `true`, or when `false` and `concept` is `false` with `read-arg` empty (`_( $pt )` instead when `#2` is non-empty — the center is still read either way). Otherwise `\l__spintent_spcircfig_intent_tl:prefix` — with `( $pt )` appended when `#2` is non-empty — prefixed with `prefix` when given, where `\l__spintent_spcircfig_intent_tl` is: with `concept` `true`, `\l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl` (via `\l__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with `\l__spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user's to write, same convention as `\l__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5699 \cs_new_protected:Nn \l__spintent_spcircfig_build_concept:nn
5700 {
5701     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5702     {
5703         \tl_if_empty:nTF {#2}
5704         { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5705         { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5706     }
5707     {
5708         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5709         {
5710             \l__spintent_spcircfig_del_word:n {#1}
5711             \tl_set:Ne \l__spintent_spcircfig_intent_tl
5712             {
5713                 \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl
5714             }
5715             \l__spintent_spcircfig_finish_intent:nn {#1} {#2}
5716             \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5717             {
5718                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5719                 {
5720                     \tl_use:c { l__spintent_sp #1 _prefix_tl } -
5721                 }
5722                 \l__spintent_spcircfig_intent_tl :prefix
5723                 \tl_if_empty:nF {#2} { ( $ pt ) }
5724             }
5725         }
5726         {
5727             \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5728             {
5729                 \tl_if_empty:nTF {#2}
5730                 { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5731                 { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5732             }
5733             {
5734                 \tl_set:Ne \l__spintent_spcircfig_intent_tl
5735                 { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
5736                 \l__spintent_spcircfig_finish_intent:nn {#1} {#2}
5737                 \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5738                 {
5739                     \l__spintent_spcircfig_intent_tl :prefix
5740                     \tl_if_empty:nF {#2} { ( $ pt ) }
5741                 }
5742             }
5743         }
5744     }

```

```
5745 }
```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

```
\__spintent_spcircfig_process:nn
```

Receives the short command name in ‘#1’ and the argument in ‘#2’. Sets `\__spintent_spcircfig_op_tl` to `área/perímetro`. With #2 empty, `\__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in #2, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```
5746 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5747 {
5748   \str_if_eq:eeTF {#1} { Acirc }
5749   { \tl_set:Nn \__spintent_spcircfig_op_tl { área } }
5750   { \tl_set:Nn \__spintent_spcircfig_op_tl { perímetro } }
5751   \tl_if_empty:nTF {#2}
5752   {
5753     \bool_set_false:N \__spintent_spcircfig_short_bool
5754   }
5755   {
5756     \bool_set_false:N \__spintent_spcircfig_short_bool
5757     \tl_if_in:nnTF {#2} { , }
5758     {
5759       \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5760       { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5761     }
5762     {
5763       \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5764       \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5765       {
5766         \str_if_eq:eeTF {#1} { Acirc }
5767         { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5768         { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5769         \str_if_eq:VnF \__spintent_geo_luaset_error_str { true }
5770         {
5771           \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
5772           { \bool_set_true:N \__spintent_spcircfig_short_bool }
5773           { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
5774         }
5775       }
5776       \bool_if:NF \__spintent_spcircfig_short_bool
5777       {
5778         \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5779         {
5780           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5781           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5782         }
5783       }
5784     }
5785   }
5786   \bool_if:NTF \__spintent_spcircfig_short_bool
5787   { \__spintent_spcircfig_build_short:n {#1} }
5788   { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
5789   \__spintent_spcircfig_mathml:nn { #1 } { #2 }
5790 }
```

(End of definition for `\__spintent_spcircfig_process:nn`.)

```
\__spintent_spcircfig_mathml:nn
```

Renders `\__spintent_spcircfig_intent_arg_tl`, already built by `\__spintent_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty #2 prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintent_mathcal:n{⟨A | P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings

inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5791 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn
5792 {
5793   \__spintent_invisible_sep: % (luamml issues #26)
5794   \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
5795   {
5796     \__spintent_inner_sep:
5797     \__spintent_luafun_mrow_intent_arg:nn
5798     { } { intent = ":silent" }
5799     {
5800       \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5801     }
5802     \bool_if:NTF \__spintent_spcircfig_short_bool
5803     {
5804       \c_math_subscript_token
5805       {
5806         \__spintent_spcircfig_sym:n {#1}
5807         \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
5808       }
5809     }
5810     {
5811       \c_math_subscript_token
5812       {
5813         \MathMLintent { :silent } { \__spintent_spcircfig_sym:n {#1} }
5814         \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
5815         {
5816           \c_math_subscript_token
5817           {
5818             \MathMLintent { :silent }
5819             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5820           }
5821         }
5822         \tl_if_empty:nF {#2}
5823         {
5824           \MathMLintent { :silent } { \lparen }
5825           \__spintent_mathmlarg_point:nnV { pt } { sp #1 } \__spintent_geo_luaset_poin
5826           \MathMLintent { :silent } { \rparen }
5827         }
5828       }
5829     }
5830   }
5831 }

```

(End of definition for `\__spintent_spcircfig_mathml:nn`.)

`\__spintent_spcircfig_exec:nnn`

The function `\__spintent_spcircfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spcircfig_process:nn`.

```

5832 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
5833 {
5834   \keys_set:nn { spintent / sp #1 } { #2 }
5835   \__spintent_spcircfig_process:nn { #1 } { #3 }
5836 }

```

(End of definition for `\__spintent_spcircfig_exec:nnn`.)

`\spAcirc`
`\spPcirc`

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

```

5837 \NewDocumentCommand \spAcirc { 0{} d() }
5838 {
5839   \mode_if_math:TF
5840   {
5841     \group_begin:
5842     \attribute \l__spintent_scope_attr_int = 1
5843     \tl_if_novalue:nTF {#2}
5844     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
5845     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5846     \group_end:
5847   }
5848   { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }

```

```

5849   }
5850   \NewDocumentCommand \spPcirc { 0{} d() }
5851   {
5852     \mode_if_math:TF
5853     {
5854       \group_begin:
5855         \attribute \l__spintent_scope_attr_int = 1
5856         \tl_if_novalue:nTF {#2}
5857         { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5858         { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5859       \group_end:
5860     }
5861     { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }
5862   }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page 30.)

11.48 The command `\spVol`

The user-level command `\spVol` provides a semantic interface for the volume of a solid, always typesetting `\mathcal{V}`.

11.48.1 Definition of variables for `\spVol`

`\l__spintent_spVol_mode_str` holds the `read-cmd` choice (`school` or `mathcat`); `\l__spintent_spVol_sub_index_str` holds the raw `sub-index` text; `\l__spintent_spVol_read_arg_tl` the sanitized `read-arg` text. `\l__spintent_spVol_read_arg_tl` and `\l__spintent_spVol_suffix_tl` are scratch variables holding the word to voice (`volumen`) and the text to attach after it; `\l__spintent_spVol_native_bool` records whether that text should be combined natively with the subscript (silenced) or left for MathCAT to read on its own. `\l__spintent_spVol_read_arg_full_tl` holds the text to voice when `read-arg` is given.

```

5863 \str_new:N \l__spintent_spVol_mode_str
5864 \str_new:N \l__spintent_spVol_sub_index_str
5865 \tl_new:N \l__spintent_spVol_read_arg_tl
5866 \tl_new:N \l__spintent_spVol_word_tl
5867 \tl_new:N \l__spintent_spVol_suffix_tl
5868 \bool_new:N \l__spintent_spVol_native_bool
5869 \tl_new:N \l__spintent_spVol_read_arg_full_tl

```

(End of definition for `\l__spintent_spVol_mode_str` and others.)

11.48.2 Definition of keys for `\spVol`

We define the *(keys)* `read-cmd`, `sub-index`, `read-sub`, `read-arg`, `concept` and `silent`.

```

read-cmd 5870 \keys_define:nn { spintent / spVol }
sub-index 5871 {
read-sub 5872   read-cmd   .choices:nn = { school , mathcat }
read-arg 5873   {
concept 5874     \int_case:nn { \l_keys_choice_int }
silent 5875     {
unknown 5876       {1} { \str_set:Nn \l__spintent_spVol_mode_str { school } }
5877       {2} { \str_set:Nn \l__spintent_spVol_mode_str { mathcat } }
5878     }
5879   },
5880   read-cmd   .initial:n = school,
5881   read-cmd   .value_required:n = true,
5882   read-cmd / unknown .code:n =
5883     \msg_error:nneee { spintent } { unknown-choice }
5884     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
5885   sub-index .code:n =
5886     { \str_set:Nn \l__spintent_spVol_sub_index_str {#1} },
5887   sub-index .initial:n = ,
5888   sub-index .value_required:n = true,
5889   read-sub .bool_set:N = \l__spintent_spVol_read_sub_bool,
5890   read-sub .initial:n = false,
5891   read-sub .value_required:n = true,
5892   read-arg .code:n =
5893     { \__spintent_sanitizize_affix:Nn \l__spintent_spVol_read_arg_tl {#1} },
5894   read-arg .initial:n = ,
5895   read-arg .value_required:n = true,
5896   concept .bool_set:N = \l__spintent_spVol_concept_bool,
5897   concept .initial:n = true,
5898   concept .value_required:n = true,
5899   silent .bool_set:N = \l__spintent_spVol_silent_bool,
5900   silent .initial:n = false,

```

```

5901     silent .value_required:n = true,
5902     unknown .code:n =
5903     {
5904         \str_set:Nn \l__spintent_command_name_str { spVol }
5905         \__spintent_unknown_keys_cmd:n {#1}
5906     },
5907 }

```

(End of definition for read-cmd and others.)

11.48.3 Internal functions for \spVol

`\__spintent_spVol_print:` The function `\__spintent_spVol_print:` dispatches to `\__spintent_spVol_silent:` when `silent` is `true`, or to `\__spintent_spVol_verbal:` otherwise.

```

5908 \cs_new_protected:Nn \__spintent_spVol_print:
5909 {
5910     \__spintent_invisible_sep:
5911     \bool_if:NTF \l__spintent_spVol_silent_bool
5912     { \__spintent_spVol_silent: }
5913     { \__spintent_spVol_verbal: }
5914 }

```

(End of definition for `\__spintent_spVol_print:`.)

`\__spintent_spVol_silent:` The function `\__spintent_spVol_silent:` typesets `\mathcal{V}` with the intent `_-` — the symbol alone through the shared queue when there is no `sub-index`, or symbol and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

5915 \cs_new_protected:Nn \__spintent_spVol_silent:
5916 {
5917     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
5918     {
5919         \__spintent_luafun_mrow_intent_arg:nn
5920         { } { intent = "_-" }
5921         { \mathcal{V} }
5922     }
5923     {
5924         \MathMLintent { _- }
5925         {
5926             \__spintent_inner_sep:
5927             \mathcal{V}
5928             \c_math_subscript_token
5929             {
5930                 \MathMLintent { :silent }
5931                 { \l__spintent_spVol_sub_index_str }
5932             }
5933         }
5934     }
5935 }

```

(End of definition for `\__spintent_spVol_silent:`.)

`\__spintent_spVol_verbal:` The function `\__spintent_spVol_verbal:` delegates to `\__spintent_spVol_read_arg_print:` when `read-arg` is given, and otherwise, with `concept false`, to `\__spintent_spVol_symbol_print:`. With `concept true` it sets `\l__spintent_spVol_word_tl` to `volumen` and `\l__spintent_spVol_suffix_tl` to `sub-(value)/\langle value \rangle` (per `read-sub`), combined natively only under `read-cmd=school` — under `read-cmd=mathcat` the suffix is left for MathCAT to read on its own. Delegates the actual typesetting to `\__spintent_spVol_attach_subscript:`

```

5936 \cs_new_protected:Nn \__spintent_spVol_verbal:
5937 {
5938     \tl_if_empty:NTF \l__spintent_spVol_read_arg_tl
5939     {
5940         \bool_if:NTF \l__spintent_spVol_concept_bool
5941         {
5942             \tl_set:Nn \l__spintent_spVol_word_tl { volumen }
5943             \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
5944             { \bool_set_true:N \l__spintent_spVol_native_bool }
5945             { \bool_set_false:N \l__spintent_spVol_native_bool }
5946             \tl_set:Nn \l__spintent_spVol_suffix_tl
5947             {
5948                 \bool_if:NT \l__spintent_spVol_read_sub_bool { sub- }
5949                 \l__spintent_spVol_sub_index_str
5950             }
5951             \__spintent_spVol_attach_subscript:

```

```

5952     }
5953     { \__spintent_spVol_symbol_print: }
5954   }
5955   { \__spintent_spVol_read_arg_print: }
5956 }

```

(End of definition for __spintent_spVol_verbal:.)

__spintent_spVol_symbol_print:

With `concept false`, the symbol itself is read instead — but only when `read-cmd` is `school`: with no `sub-index`, the shared queue tags the symbol with the intent `_V-`; with one, symbol and subscript are wrapped together in one native `\MathMLintent` (subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the symbol and its subscript, unmarked by us.

```

5957 \cs_new_protected:Nn \__spintent_spVol_symbol_print:
5958 {
5959   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
5960   {
5961     \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
5962     {
5963       \__spintent_luafun_mrow_intent_arg:nn
5964       { } { intent = "_V-" }
5965       { \mathcal{V} } }
5966     }
5967     { \mathcal{V} } }
5968   }
5969   {
5970     \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
5971     {
5972       \MathMLintent
5973       {
5974         _V-
5975         \bool_if:NT \l__spintent_spVol_read_sub_bool { sub- }
5976         \l__spintent_spVol_sub_index_str
5977         :prefix
5978       }
5979       {
5980         \__spintent_inner_sep:
5981         \mathcal{V}
5982         \c_math_subscript_token
5983         {
5984           \MathMLintent { :silent }
5985           { \l__spintent_spVol_sub_index_str }
5986         }
5987       }
5988     }
5989     {
5990       \mathcal{V}
5991       \c_math_subscript_token
5992       { \l__spintent_spVol_sub_index_str }
5993     }
5994   }
5995 }

```

(End of definition for __spintent_spVol_symbol_print:.)

__spintent_spVol_attach_subscript:

The function `\__spintent_spVol_attach_subscript`: typesets `\mathcal{V}`. An empty `sub-index` tags the symbol alone through the shared queue (`\l__spintent_spVol_suffix_tl` is discarded, since there's no subscript to attach it to). A given one combines symbol, subscript, and `\l__spintent_spVol_suffix_tl` in one native `\MathMLintent` (subscript silenced) when `\l__spintent_spVol_native_bool` is `true`, or tags the symbol alone through the shared queue with the subscript left bare when `false`.

```

5996 \cs_new_protected:Nn \__spintent_spVol_attach_subscript:
5997 {
5998   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
5999   {
6000     \__spintent_luafun_mrow_intent_arg:nn
6001     { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
6002     { \mathcal{V} } }
6003   }
6004   {
6005     \bool_if:NTF \l__spintent_spVol_native_bool
6006     {
6007       \MathMLintent

```

```

6008         { _ \l__spintent_spVol_word_tl - \l__spintent_spVol_suffix_tl :nofix }
6009     {
6010         \__spintent_inner_sep:
6011         \mathcal{V}
6012         \c_math_subscript_token
6013         {
6014             \MathMLintent { :silent }
6015             { \l__spintent_spVol_sub_index_str }
6016         }
6017     }
6018 }
6019 {
6020     \__spintent_luafun_mrow_intent_arg:nn
6021     { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
6022     { \mathcal{V} }
6023     \c_math_subscript_token
6024     { \l__spintent_spVol_sub_index_str }
6025 }
6026 }
6027 }

```

(End of definition for `\__spintent_spVol_attach_subscript:.`)

`\__spintent_spVol_read_arg_print:`

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true`, `\l__spintent_spVol_read_arg_full_tl` becomes `volumen` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively regardless of `read-cmd:` with no `sub-index`, the shared queue tags the symbol alone; with one, symbol and subscript are wrapped together in one native `\MathMLintent`.

```

6028 \cs_new_protected:Nn \__spintent_spVol_read_arg_print:
6029 {
6030     \bool_if:NTF \l__spintent_spVol_concept_bool
6031     {
6032         \tl_set:Nx \l__spintent_spVol_read_arg_full_tl
6033         { volumen- \tl_use:N \l__spintent_spVol_read_arg_tl }
6034     }
6035     { \tl_set_eq:NN \l__spintent_spVol_read_arg_full_tl \l__spintent_spVol_read_arg_tl }
6036     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6037     {
6038         \__spintent_luafun_mrow_intent_arg:nn
6039         { } { intent = "_ \l__spintent_spVol_read_arg_full_tl :nofix" }
6040         { \mathcal{V} }
6041     }
6042     {
6043         \MathMLintent
6044         { _ \l__spintent_spVol_read_arg_full_tl :nofix }
6045         {
6046             \__spintent_inner_sep:
6047             \mathcal{V}
6048             \c_math_subscript_token
6049             {
6050                 \MathMLintent { :silent }
6051                 { \l__spintent_spVol_sub_index_str }
6052             }
6053         }
6054     }
6055 }

```

(End of definition for `\__spintent_spVol_read_arg_print:.`)

`\spVol` The public command `\spVol` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spVol_print:.`

```

6056 \NewDocumentCommand \spVol { 0{} }
6057 {
6058     \mode_if_math:TF
6059     {
6060         \group_begin:
6061         \attribute \l__spintent_scope_attr_int = 1
6062         \keys_set:nn { spintent / spVol } {#1}
6063         \__spintent_spVol_print:
6064         \group_end:
6065     }

```

```
6066 { \msg_error:nnn { spintent } { need-math-mode } { \spVol } }
6067 }
```

(End of definition for \spVol. This function is documented on page 32.)

11.49 Finish package

Finish package implementation.

```
6068 \file_input_stop:
6069 \endpackage
```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A		silent	122
\angle	123	Keys for \sparc provide by spintent :	
autofit	3196	concept	127
		prefix	127
C		read-cmd	127
cifras-sep	284, 569, 842, 1874, 1945, 2535	silent	127
Document class:		Keys for \spcirc provide by spintent :	
l3doc	39	prefix	146
cmd-key-unknown	203	print-sym	146, 148
cmd-key-value-unknown	203	radio-sty	146
Commands provide by spintent :		read-arg	146
\spmoney	57	read-cmd	146
\spnum	47	read-sty	146
\spsiglo	66, 69	read-sub	146
\spunit	52	sub-index	146
concept 2036, 2234, 3196, 3795, 4002, 4287, 4440, 4624, 5007, 5265, 5555, 5870		Keys for \spcoord provide by spintent :	
concept*	4624	silent	99
currency	842	Keys for \spdiam provide by spintent :	
currency-no-subunits	811	concept	141
		read-arg	141
D		read-cmd	141
diameter	5265	read-sub	141
		silent	141
I		sub-index	141
inline	2397	upper-case	141
inline-numeric-denominator	547	Keys for \spdiv provide by spintent :	
invalid-angle-format	2485	cifras-sep	79
invalid-century-format	1237	notation-sym	79
invalid-currency	811	read-parens	79
invalid-date-format	1467	read-period-sep	79
invalid-time-format	1388	read-sign	79
		read-sym	79
J		set-sym	79
join-word	2613	wrap-parens	79
		Keys for \spdsym provide by spintent :	
K		prefix	76
Keys for \spAcirc provide by spintent :		read-sym	76
concept	152	set-sym	76
prefix	152	suffix	76
print-sym	152, 153	Keys for \spmangle provide by spintent :	
radio-sty	152	concept	122
read-arg	152	prefix	121, 122
read-cmd	152	print-m	121, 122
read-sty	152	read-cmd	121, 122
read-sub	152	recto	121, 122
silent	152	silent	122
sub-index	152	Keys for \spmarc provide by spintent :	
Keys for \spAfig provide by spintent :		concept	127
prefix	133	prefix	127
print-sym	133, 134	read-cmd	127
read-arg	133, 134, 136	silent	127
read-cmd	133	Keys for \spmcd provide by spintent :	
read-sub	134	concept	82
Keys for \spangle provide by spintent :		prefix	82
concept	122	read-cmd	82
prefix	121, 122	result	82
print-m	122	silent	82
read-cmd	121, 122	upper-case	82
recto	121, 122	Keys for \spmcm provide by spintent :	
		concept	82

prefix	82	print-sym	152, 153
read-cmd	82	radio-sty	152
result	82	read-arg	152
silent	82	read-cmd	152
upper-case	82	read-sty	152
Keys for \spmoney provide by spintent :		read-sub	152
cifras-sep	58	silent	152
currency	58	sub-index	152
money-math	58	Keys for \spPfig provide by spintent :	
sub-units	58	prefix	133
Keys for \spmQ provide by spintent :		print-sym	133, 134
join-word	94, 96	read-arg	133, 134, 136
print-dfrac	94-98	read-cmd	133
read-parens	94, 95	read-sub	134
use-spnZ	94, 97, 98	Keys for \spPoint provide by spintent :	
wrap-parens	94, 95	concept	104
Keys for \spmside provide by spintent :		prefix	104
concept	118	read-cmd	104
prefix	117, 118	silent	104
print-m	117, 118	Keys for \spPoly provide by spintent :	
print-sym	141	concept	130
read-arg	117, 118	prefix	130
read-cmd	117, 118, 120	print-sym	130
read-sty	117, 118	read-cmd	130
silent	118, 120	silent	130
Keys for \spmsym provide by spintent :		Keys for \sprad provide by spintent :	
not-print	78	concept	141
prefix	78	read-arg	141
read-sym	78	read-cmd	141
set-sym	78	read-sub	141
suffix	78	silent	141
Keys for \spmul provide by spintent :		sub-index	141
cifras-sep	80	upper-case	141
notation-sym	80	Keys for \sprayo provide by spintent :	
read-parens	80	concept	115
read-period-sep	80	prefix	115
read-sign	80	read-cmd	115, 116
read-sym	80	read-sty	114, 115
set-sym	80	silent	115, 116
wrap-parens	80	Keys for \sprazon provide by spintent :	
Keys for \spnD provide by spintent :		inline	89
concept	86	silent	89
prefix	86	use-dfrac	89
result-num	86	Keys for \spread provide by spintent :	
result	86	read-arg	75
silent	86	silent	75
Keys for \spnM provide by spintent :		Keys for \sprecta provide by spintent :	
concept	86	prefix	111
prefix	86	read-cmd	111
result-num	86	silent	111
result	86	Keys for \spshort provide by spintent :	
silent	86	read-arg	64
Keys for \spnum provide by spintent :		small-caps	64
cifras-sep	46-48	Keys for \spside provide by spintent :	
notation-sym	47	concept	118
read-period-sep	47, 49	prefix	117, 118
read-sign	47	read-arg	117, 118
Keys for \spnZ provide by spintent :		read-cmd	117, 118, 120
cifras-sep	92	read-sty	117, 118
read-parens	92, 93	silent	118, 120
read-sign	92	Keys for \spsiglo provide by spintent :	
wrap-parens	92, 93	print-era	67, 68
Keys for \spPcirc provide by spintent :		Keys for \spunit provide by spintent :	
concept	152	cifras-sep	53
prefix	152	notation-sym	53

print-cdot	53	result-num	2234
read-cdot	53	right-angle-sqr-fallback	80
read-period-sep	53	\rightangle	123
read-slash	53		
Keys for \spusep provide by spintent :		S	
silent	72	seq commands:	
size	72	\seq_set_from_clist:Nn	62
Keys for \spvec provide by spintent :		set-sym	1730, 1800, 1874, 1945
autofit	107	\setspintent	3, 43, 191
concept	107	silent 1515 , 1654 , 2036 , 2234 , 2397 , 2827 , 3075 , 3196 , 3438 , 3658 , 3795 , 4002 , 4287 , 4440 , 4624 , 5007 , 5265 , 5555 , 5870	
prefix	106, 107	size	1515
read-cmd	106, 107	small-caps	1115
silent	107	\spAcirc	30, 151, 5837
Keys for \spVol provide by spintent :		spAcirc-invalid-arg	5546
concept	158	\spAfig	25, 133, 4978
read-arg	158	spAfig-invalid-arg	4586
read-cmd	158	spAfig-invalid-points	4586
read-sub	158	spAfig-read-arg-key	4586
silent	158	spAfig-sym-cardinality-conflict	4586
sub-index	158	spAfig-sym-letra-conflict	4586
		\spang	23, 91, 2525
M		\spangle	22, 121, 4250
mathcal-fallback	80	spangle-invalid-arg	3992
\MathMLintent	125	\sparc	23, 127, 4409
mc-invalid-arguments	2029	sparc-invalid-points	4277
mc-too-few-arguments	2260	\spcirc	29, 145, 5522
md-invalid-argument	2260	spcirc-invalid-arg	5258
Lua module provide by spintent :		\spcoord	17, 99, 2964
spintent.lua 39 , 40 , 42 , 45-47 , 52 , 56-59 , 63 , 64 , 66 , 68 , 69 , 71 , 82 , 86 , 91 , 93 , 98 , 104		spcoord-invalid-arg	2823
money-math	842	\spdate	11, 71, 1491
		\spdiam	28, 141, 5225
N		\spdiv	13, 79, 1935
not-print	1800	spdiv-empty-argument	1894
notation-sym	284 , 569 , 1874 , 1945	\spdsym	12, 76, 1777
		spintent internal commands:	
P		_spintent_collapse_hyphens:N	128
Packages :		\l_spintent_command_name_str	203
expl3	45, 47	\l_spintent_geo_circfig_mode_str	5545
lua-unicode-math	39	\l_spintent_geo_final_print_tl	4578
luamml	42, 94, 96	\l_spintent_geo_luaaset_angulo_case_-str	2796
tagpdf	63, 64, 68	\l_spintent_geo_luaaset_circ_radio_raw_-str	5247
unicode-math	39, 73	\l_spintent_geo_luaaset_circ_radio_type_-str	5247
prefix 1730 , 1800 , 2036 , 2234 , 3075 , 3196 , 3438 , 3658 , 3795 , 4002 , 4287 , 4440 , 4624 , 5265 , 5555		\l_spintent_geo_luaaset_concept_str ...	2796
print-dfrac	2613	\l_spintent_geo_luaaset_error_str	2796
print-era	1245	\l_spintent_geo_luaaset_intent_str ...	2796
print-m	3795 , 4002	\l_spintent_geo_luaaset_letra_str	2796
print-sym	4440 , 4624 , 5007 , 5265 , 5555	\l_spintent_geo_luaaset_letra_word_str	2796
		\l_spintent_geo_luaaset_nombre_es_num_-str	2796
R		\l_spintent_geo_luaaset_points_-clist _{uuuu} \l_spintent_geo_luaaset_is_-name_str	2796
radio-sty	5265	\l_spintent_geo_luaaset_poly_sym_str ..	2796
read-arg ..	1115 , 1654 , 3795 , 4624 , 5007 , 5265 , 5555 , 5870	\l_spintent_geo_luaaset_print_tl	2796
read-cdot	569	\l_spintent_geo_luaaset_sub_spoken_str	2796
read-cmd ..	2036 , 3075 , 3196 , 3438 , 3658 , 3795 , 4002 , 4287 , 4440 , 4624 , 5007 , 5265 , 5555 , 5870	\l_spintent_geo_luaaset_sub_str	2796
read-parens	1874 , 1945 , 2535 , 2613	\l_spintent_geo_luaaset_tipo_str	2796
read-period-sep	284 , 569 , 1874 , 1945		
read-sign	284 , 1874 , 1945 , 2535		
read-slash	569		
read-sty	3658 , 3795 , 5265 , 5555		
read-sub	4624 , 5007 , 5265 , 5555 , 5870		
read-sym	1730 , 1800 , 1874 , 1945		
recto	4002		
result	2036 , 2234		

<code>\l__spintent_geo_print_sym_str</code>	4694	<code>__spintent_right_angle_sqr:</code>	107
<code>__spintent_inner_sep:</code>	120	<code>__spintent_sanitize_affix:Nn</code>	136
<code>__spintent_invisible_sep:</code>	116	<code>\l__spintent_saved_luaaml_flag_int</code> 96, 2594	
<code>\c__spintent_invisible_sep_int</code>	124	<code>\l__spintent_scope_attr_int</code>	74
<code>\c__spintent_invisible_sep_tl</code>	96	<code>\l__spintent_setspintent_input_arg_cmd_-</code> <code>tl</code>	154
<code>__spintent_luafun_clean_split_and_set:n</code> 76 , 241		<code>__spintent_setspintent_parse_args:nn</code> ..	174
<code>__spintent_luafun_mrow_intent_arg:nn</code> ..	116	<code>__spintent_setspintent_set_keys:nn</code> ...	163
<code>__spintent_luafun_set_debug_ghost:n</code> ...	71	<code>\l__spintent_smparc_prefix_tl</code>	4272
<code>__spintent_luafun_spang_parse:n</code>	91	<code>__spintent_sp_rad_diam_exec:nn</code>	5220
<code>__spintent_luafun_spdate_parse:n</code>	71	<code>\l__spintent_sp_rad_diam_letra_tl</code>	5004
<code>__spintent_luafun_spmoney_lookup_data:n</code> 76		<code>__spintent_sp_rad_diam_print:n</code>	5054
<code>__spintent_luafun_spmQ_scale_int:nn</code> ...	76	<code>\l__spintent_sp_rad_diam_read_arg_full_-</code> <code>tl</code>	5004
<code>__spintent_luafun_sptime_parse:n</code>	71	<code>__spintent_sp_rad_diam_read_arg_-</code> <code>print:n</code>	5182
<code>__spintent_luafun_spunit_lookup_alias:n</code> 76		<code>\l__spintent_sp_rad_diam_word_tl</code>	5004
<code>\l__spintent_luaset_arg_above_tl</code>	260	<code>\l__spintent_spAcirc_mode_str</code>	5535
<code>\l__spintent_luaset_arg_below_tl</code>	260	<code>\l__spintent_spAcirc_prefix_tl</code>	5535
<code>\l__spintent_luaset_cal_fam_ok_str</code>	67	<code>\l__spintent_spAcirc_print_sym_str</code> ...	5535
<code>\l__spintent_luaset_dec_is_all_zeros_-</code> <code>str</code>	253	<code>\l__spintent_spAcirc_radio_sty_str</code> ...	5535
<code>\l__spintent_luaset_dec_sep_tl</code>	241	<code>\l__spintent_spAcirc_read_arg_tl</code>	5535
<code>\l__spintent_luaset_decimal_period_str</code> 253		<code>\l__spintent_spAcirc_read_sty_str</code>	5535
<code>\l__spintent_luaset_denom_has_number_-</code> <code>str</code>	260	<code>\l__spintent_spAcirc_sub_index_str</code> ...	5535
<code>\l__spintent_luaset_exponent_tl</code>	253	<code>\l__spintent_spAfig_mode_str</code>	4579
<code>\l__spintent_luaset_format_part_dec_str</code> 248		<code>\l__spintent_spAfig_prefix_tl</code>	4579
<code>\l__spintent_luaset_format_part_int_str</code> 248		<code>\l__spintent_spAfig_print_sym_str</code>	4579
<code>\l__spintent_luaset_has_exponent_str</code> ..	253	<code>\l__spintent_spAfig_read_arg_tl</code>	4579
<code>\l__spintent_luaset_has_integer_str</code> .. 96–98, 241		<code>\l__spintent_spang_luaset_error_str</code> ...	2481
<code>\l__spintent_luaset_has_natural_str</code> ...	241	<code>\l__spintent_spang_luaset_grado_str</code> ...	2481
<code>\l__spintent_luaset_has_units_str</code>	260	<code>\l__spintent_spang_luaset_minuto_str</code> ..	2481
<code>\l__spintent_luaset_millions_str</code>	253	<code>\l__spintent_spang_luaset_segundo_str</code> .	2481
<code>\l__spintent_luaset_not_decimal_not_-</code> <code>period_str</code>	253	<code>__spintent_spang_process:n</code>	2490
<code>\l__spintent_luaset_not_decimal_period_-</code> <code>str</code>	253	<code>__spintent_spangle_exec:nnn</code>	4245
<code>\l__spintent_luaset_only_part_dec_str</code> .	248	<code>\l__spintent_spangle_intent_arg_tl</code> ...	4061
<code>\l__spintent_luaset_only_part_int_str</code> .	248	<code>\l__spintent_spangle_intent_tl</code>	4061
<code>\l__spintent_luaset_only_part_period_-</code> <code>str</code>	248	<code>__spintent_spangle_mathml:n</code>	4103
<code>\l__spintent_luaset_part_dec_tl</code>	241	<code>\l__spintent_spangle_mode_str</code>	3987
<code>\l__spintent_luaset_part_int_tl</code> 96, 241		<code>\l__spintent_spangle_prefix_tl</code>	3987
<code>\l__spintent_luaset_part_period_tl</code> ...	241	<code>\l__spintent_spangle_print_m_now_bool</code> .	4061
<code>\l__spintent_luaset_scope_attr_str</code>	67	<code>__spintent_spangle_process:nn</code>	4068
<code>\l__spintent_luaset_sign_tl</code>	96, 241	<code>\l__spintent_spangle_pt_idx_int</code>	4061
<code>\l__spintent_luaset_status_str</code>	260	<code>\l__spintent_spangle_pt_seq</code>	4061
<code>\l__spintent_luaset_sym_fam_ok_str</code>	67	<code>\l__spintent_spangle_refs_tl</code>	4061
<code>__spintent_mathcal:n</code>	94	<code>__spintent_spangle_sym:n</code>	4042
<code>__spintent_mathmlarg_point:nnn</code>	2979	<code>__spintent_sparc_exec:nnn</code>	4404
<code>\l__spintent_mathmlarg_point_kv_tl</code> ...	2975	<code>\l__spintent_sparc_intent_arg_tl</code>	4331
<code>\l__spintent_mathmlarg_point_mode_str</code> .	2975	<code>\l__spintent_sparc_intent_tl</code>	4331
<code>\l__spintent_mathmlarg_point_ref_tl</code> ...	2975	<code>__spintent_sparc_mathml:n</code>	4349
<code>\l__spintent_mathmlarg_point_silent_-</code> <code>bool</code>	2975	<code>\l__spintent_sparc_mode_str</code>	4272
<code>\l__spintent_mc_args_seq</code>	2016	<code>\l__spintent_sparc_prefix_tl</code>	4272
<code>\l__spintent_mc_luaset_all_numeric_str</code> 2016		<code>__spintent_sparc_print_arc:n</code>	4325
<code>\l__spintent_mc_luaset_types_str</code>	2016	<code>__spintent_sparc_process:nn</code>	4336
<code>__spintent_mc_process_args:nn</code>	2077	<code>\l__spintent_sparc_pt_idx_int</code>	4331
<code>\l__spintent_mc_raw_args_seq</code>	2016	<code>\l__spintent_sparc_pt_seq</code>	4331
<code>\l__spintent_mc_types_seq_{UUU}\l__spintent_-</code> <code>spmcd_print_result_bool</code>	2016	<code>\l__spintent_sparc_refs_tl</code>	4331
<code>\l__spintent_md_out_seq</code>	2225	<code>__spintent_sparc_wide_arc:n</code>	4321
<code>__spintent_print_spunit_hierarchy:</code> ...	670	<code>\l__spintent_spcirc_articulo_str</code>	5249
		<code>__spintent_spcirc_build_concept:</code>	5403
		<code>\l__spintent_spcirc_connector_arg_tl</code> ..	5351
		<code>\l__spintent_spcirc_connector_tl</code>	5351
		<code>__spintent_spcirc_exec:nn</code>	5517

\l__spintent_spcirc_has_arg_bool	5351	\l__spintent_spdsym_read_sym_tl	1726
\l__spintent_spcirc_intent_arg_tl	5351	\l__spintent_spdsym_suffix_tl	1726
\l__spintent_spcirc_intent_tl	5351	\l__spintent_spdsym_symbol_tl	1726
__spintent_spcirc_mathml:	5472	__spintent_sfig_build_concept_-	
\l__spintent_spcirc_mode_str	5249	vertices:n	4851
\l__spintent_spcirc_prefix_tl	5249	__spintent_sfig_build_refs:	4838
\l__spintent_spcirc_print_sym_str	5249	__spintent_sfig_build_simple:n	4727
__spintent_spcirc_process:n	5456	__spintent_sfig_exec:nnn	4973
\l__spintent_spcirc_radio_arg_tl	5351	\l__spintent_sfig_intent_arg_tl	4694
__spintent_spcirc_radio_print:	5388	\l__spintent_sfig_intent_tl	4694
\l__spintent_spcirc_radio_sty_str	5249	__spintent_sfig_mathml:n	4933
\l__spintent_spcirc_read_sty_str	5249	\l__spintent_sfig_op_tl	4694
\l__spintent_spcirc_sub_index_str	5249	__spintent_sfig_points_loop:n	4921
__spintent_spcirc_sym:	5357	__spintent_sfig_process:nn	4703
__spintent_spcirc_word_and_article:	5367	\l__spintent_sfig_pt_idx_int	4694
\l__spintent_spcirc_word_tl	5257	\l__spintent_sfig_pt_seq	4694
__spintent_spcircfig_build_concept:nn	5699	\l__spintent_sfig_refs_tl	4694
__spintent_spcircfig_build_short:n	5661	__spintent_sfig_sym:	4686
__spintent_spcircfig_del_word:n	5650	\l__spintent_sfig_symbol_tl	4694
__spintent_spcircfig_exec:nnn	5832	\l__spintent_spmangle_mode_str	3987
__spintent_spcircfig_finish_intent:nn	5683	\l__spintent_spmangle_prefix_tl	3987
\l__spintent_spcircfig_intent_arg_tl	5621	\l__spintent_spmarc_mode_str	4272
\l__spintent_spcircfig_intent_tl	5621	__spintent_spmc_process:nnnn	2131
__spintent_spcircfig_mathml:nn	5791	\l__spintent_spmcd_prefix_tl	2016
\l__spintent_spcircfig_op_tl	5621	\l__spintent_spmcd_print_upper_bool	2016
__spintent_spcircfig_process:nn	5746	\l__spintent_spmcd_read_terse_bool	2016
\l__spintent_spcircfig_short_bool	5621	\l__spintent_spmcd_suffix_tl	2016
__spintent_spcircfig_sym:n	5626	\l__spintent_spmcm_prefix_tl	2016
__spintent_spcircfig_word_and_-		\l__spintent_spmcm_print_result_bool	2016
article:n	5632	\l__spintent_spmcm_print_upper_bool	2016
__spintent_spcoord_exec:nn	2946	\l__spintent_spmcm_read_terse_bool	2016
__spintent_spcoord_intent_infix:	2917	\l__spintent_spmcm_spmcd_luaset_error_-	
\l__spintent_spcoord_intent_str	2819	str	2016
\l__spintent_spcoord_luaset_error_str	2810	\l__spintent_spmcm_suffix_tl	2016
\l__spintent_spcoord_luaset_sep_str	2810	__spintent_spmoney_build_tree:	1027
\l__spintent_spcoord_luaset_x_has_frac_-		__spintent_spmoney_check_sub_units:	978
str	2810	\l__spintent_spmoney_extracted_curr_str	803
\l__spintent_spcoord_luaset_x_sign_str	2810	\l__spintent_spmoney_extracted_num_tl	803
\l__spintent_spcoord_luaset_x_type_str	2810	__spintent_spmoney_intent_int:	943
\l__spintent_spcoord_luaset_x_value_-		\l__spintent_spmoney_intent_money_str	803
str	2810	\l__spintent_spmoney_intent_sign_str	803
\l__spintent_spcoord_luaset_y_has_frac_-		\l__spintent_spmoney_intent_sub_unit_-	
str	2810	str	803
\l__spintent_spcoord_luaset_y_sign_str	2810	\l__spintent_spmoney_luaset_conde_str	793
\l__spintent_spcoord_luaset_y_type_str	2810	\l__spintent_spmoney_luaset_currency_arg_-	
\l__spintent_spcoord_luaset_y_value_-		str	793
str	2810	\l__spintent_spmoney_luaset_plur_str	793
__spintent_spcoord_mathml:	2929	\l__spintent_spmoney_luaset_position_-	
__spintent_spcoord_print:n	2894	str	793
\l__spintent_spcoord_raw_arg_tl	2819	\l__spintent_spmoney_luaset_print_iso_-	
__spintent_spcoord_set_wrap_parens_-		str	793
size:	2838	\l__spintent_spmoney_luaset_sing_str	793
__spintent_spcoord_value:n	2885	\l__spintent_spmoney_luaset_status_str	793
\l__spintent_spcoord_wrap_parens_l_tl	2819	\l__spintent_spmoney_luaset_subplur_str	793
\l__spintent_spcoord_wrap_parens_r_tl	2819	\l__spintent_spmoney_luaset_subsing_str	793
\l__spintent_spdate_luaset_error_str	1465	\l__spintent_spmoney_luaset_symbol_str	793
\l__spintent_spdate_luaset_output_str	1465	__spintent_spmoney_normalize_key:n	828
__spintent_spdate_process:n	72, 1476	__spintent_spmoney_parse_and_route:n	1068
__spintent_spdiv_process:nnn	1920	__spintent_spmoney_print_dec_sub_-	
__spintent_spdiv_render_op:nn	1899	unit:	1027
__spintent_spdiv_render_parens:nn	1908	__spintent_spmoney_print_int:	914
\l__spintent_spdsym_prefix_tl	1726	__spintent_spmoney_print_sep_con:	1027

<code>__spintext_spmoney_print_symbol: ...</code>	876	<code>\l__spintext_spnun_intent_read_sign_str</code>	47,
<code>__spintext_spmoney_render_layout: ...</code>	893	279	
<code>__spintext_spmoney_render_number_node:</code>	905	<code>\l__spintext_spnun_notation_sym_tl ...</code>	279
<code>__spintext_spmoney_render_only_integer_node: ...</code>	905	<code>__spintext_spnun_print_integer: ...</code>	446
<code>\l__spintext_spmoney_res_main_voice_str</code>	803	<code>__spintext_spnun_print_num_start: ...</code>	446
<code>__spintext_spmoney_set_and_print_sub_units: ...</code>	978	<code>__spintext_spnun_print_number_core: ..</code>	446
<code>__spintext_spmoney_set_currency:n ...</code>	828	<code>__spintext_spnun_print_part_dec: ...</code>	425
<code>\l__spintext_spmoney_sub_units_bool ...</code>	803	<code>__spintext_spnun_print_part_period: ..</code>	425
<code>\l__spintext_spmoney_subnum_int ...</code>	803	<code>__spintext_spnun_print_period_bar: ...</code>	354
<code>__spintext_spmoney_validate_money: ...</code>	860	<code>\l__spintext_spnun_read_period_sep_str .</code>	47,
<code>__spintext_spmQ_exec:nnnn ...</code>	2774	279	
<code>\l__spintext_spmQ_join_word_str ...</code>	96, 2594	<code>\l__spintext_spnun_read_terse_bool .</code>	47, 279
<code>__spintext_spmQ_print:nnn ...</code>	2766	<code>__spintext_spnun_render_dec: ...</code>	337
<code>\l__spintext_spmQ_print_dfrac_bool ...</code>	97	<code>__spintext_spnun_render_decimal_sep: .</code>	391
<code>__spintext_spmQ_print_scale_int: ...</code>	2712	<code>__spintext_spnun_render_int: ...</code>	337
<code>__spintext_spmQ_print_scale_int:n ...</code>	2721	<code>__spintext_spnun_render_only_period: .</code>	354
<code>__spintext_spmQ_print_spnZ_dfrac:nn ..</code>	2749	<code>__spintext_spnun_render_part:n ...</code>	337
<code>__spintext_spmQ_set_join_word: ...</code>	2703	<code>__spintext_spnun_render_period_sep: ..</code>	354
<code>__spintext_spmQ_set_wrap_parens_size:</code>	2648	<code>__spintext_spnz_process:nn ...</code>	2568
<code>__spintext_spmQ_start_wrap_parens: ...</code>	2683	<code>__spintext_spnz_render_parens: ...</code>	2556
<code>__spintext_spmQ_stop_wrap_parens: ...</code>	2683	<code>\l__spintext_spPcirc_mode_str ...</code>	5535
<code>\l__spintext_spmQ_use_spnZ_bool ...</code>	97	<code>\l__spintext_spPcirc_prefix_tl ...</code>	5535
<code>\l__spintext_spm_side_mode_str ...</code>	3778	<code>\l__spintext_spPcirc_print_sym_str ...</code>	5535
<code>\l__spintext_spm_side_prefix_tl ...</code>	3778	<code>\l__spintext_spPcirc_radio_sty_str ...</code>	5535
<code>\l__spintext_spm_side_read_arg_tl ...</code>	3778	<code>\l__spintext_spPcirc_read_arg_tl ...</code>	5535
<code>\l__spintext_spm_side_read_sty_str ...</code>	3778	<code>\l__spintext_spPcirc_read_sty_str ...</code>	5535
<code>\l__spintext_spm_sym_prefix_tl ...</code>	1796	<code>\l__spintext_spPcirc_sub_index_str ...</code>	5535
<code>\l__spintext_spm_sym_read_sym_tl ...</code>	1796	<code>\l__spintext_spPfig_mode_str ...</code>	4579
<code>\l__spintext_spm_sym_suffix_tl ...</code>	1796	<code>\l__spintext_spPfig_prefix_tl ...</code>	4579
<code>\l__spintext_spm_sym_symbol_tl ...</code>	1796	<code>\l__spintext_spPfig_print_sym_str ...</code>	4579
<code>__spintext_spmul_process:nnn ...</code>	1991	<code>\l__spintext_spPfig_read_arg_tl ...</code>	4579
<code>__spintext_spmul_render_op:nn ...</code>	1970	<code>__spintext_spPoint_exec:nnn ...</code>	3150
<code>__spintext_spmul_render_parens:nn ...</code>	1979	<code>\l__spintext_spPoint_intent_arg_tl ...</code>	3064
<code>\l__spintext_spnD_luaset_is_truncated_str ...</code>	2225	<code>__spintext_spPoint_mathml:n ...</code>	3141
<code>\l__spintext_spnD_luaset_result_tl ...</code>	2225	<code>\l__spintext_spPoint_mode_str ...</code>	3064
<code>\l__spintext_spnD_prefix_tl ...</code>	2225	<code>\l__spintext_spPoint_prefix_tl ...</code>	3064
<code>\l__spintext_spnD_print_result_num_tl .</code>	2225	<code>__spintext_spPoint_print:n ...</code>	3106
<code>\l__spintext_spnM_luaset_is_truncated_str ...</code>	2225	<code>__spintext_spPoly_build_concept: ...</code>	4499
<code>\l__spintext_spnM_luaset_result_tl ...</code>	2225	<code>__spintext_spPoly_build_refs: ...</code>	4486
<code>\l__spintext_spnM_prefix_tl ...</code>	2225	<code>__spintext_spPoly_exec:nn ...</code>	4562
<code>\l__spintext_spnM_print_result_num_tl .</code>	2225	<code>\l__spintext_spPoly_intent_arg_tl ...</code>	4474
<code>__spintext_spnM_spnD_exec:nnnn ...</code>	2327	<code>__spintext_spPoly_mathml: ...</code>	4542
<code>__spintext_spnM_spnD_intent:nn ...</code>	2279	<code>\l__spintext_spPoly_mode_str ...</code>	4431
<code>\l__spintext_spnM_spnD_luaset_error_str ...</code>	2225	<code>__spintext_spPoly_points_loop: ...</code>	4520
<code>__spintext_spnM_spnD_process:nnnn ...</code>	2274	<code>\l__spintext_spPoly_prefix_tl ...</code>	4431
<code>__spintext_spnM_spnD_result:nn ...</code>	2295	<code>__spintext_spPoly_process:n ...</code>	4532
<code>__spintext_spnM_spnD_silent:nn ...</code>	2287	<code>\l__spintext_spPoly_pt_idx_int ...</code>	4474
<code>\l__spintext_spnmd_luaset_is_number_str ...</code>	2225	<code>\l__spintext_spPoly_pt_seq ...</code>	4474
<code>\l__spintext_spnQ_intent_read_sign_str</code>	2594	<code>\l__spintext_spPoly_refs_tl ...</code>	4474
<code>\l__spintext_spnQ_read_parens_bool ...</code>	95	<code>__spintext_spPoly_sym: ...</code>	4478
<code>\l__spintext_spnQ_wrap_parens_l_tl</code>	95, 2594	<code>__spintext_sprayo_exec:nn ...</code>	3762
<code>\l__spintext_spnQ_wrap_parens_r_tl</code>	95, 2594	<code>\l__spintext_sprayo_intent_arg_tl ...</code>	3702
<code>\l__spintext_spnun__notation_sym_tl ...</code>	47	<code>__spintext_sprayo_mathml: ...</code>	3715
<code>\l__spintext_spnun_intent_read_dec_sep_str ...</code>	47, 49, 279	<code>\l__spintext_sprayo_mode_str ...</code>	3648
		<code>\l__spintext_sprayo_prefix_tl ...</code>	3648
		<code>__spintext_sprayo_print:n ...</code>	3706
		<code>\l__spintext_sprayo_pt_idx_int ...</code>	3702
		<code>\l__spintext_sprayo_pt_seq ...</code>	3702
		<code>\l__spintext_sprayo_read_sty_str ...</code>	3648
		<code>\l__spintext_sprayo_refs_tl ...</code>	3702
		<code>__spintext_sprazon_exec:nnn ...</code>	2414

<code>__spintent_spread_exec:nn</code>	1669	<code>\l__spintent_sptime_luaset_seconds_str</code>	1383
<code>\l__spintent_spread_luaset_reading_str</code>	1651	<code>\l__spintent_spunit_exp_seq</code>	540
<code>\l__spintent_spread_luaset_status_str</code> .	1651	<code>\l__spintent_spunit_exponent_tl</code>	540
<code>\l__spintent_spread_read_arg_tl</code>	1651	<code>__spintent_spunit_format_canonical:</code> ..	603
<code>__spintent_sprecta_exec:nn</code>	3632	<code>\l__spintent_spunit_is_mult_bool</code>	540
<code>\l__spintent_sprecta_intent_arg_tl</code> ...	3424	<code>\l__spintent_spunit_luaset_canonical_-</code>	
<code>\l__spintent_sprecta_kv_tl</code>	3424	<code>str</code>	533
<code>__spintent_sprecta_mathml:n</code>	3478	<code>\l__spintent_spunit_luaset_compact_exp_-</code>	
<code>\l__spintent_sprecta_mode_str</code>	3424	<code>str</code>	533
<code>\l__spintent_sprecta_prefix_tl</code>	3424	<code>\l__spintent_spunit_luaset_is_compact_-</code>	
<code>__spintent_sprecta_print:n</code>	3469	<code>str</code>	533
<code>\l__spintent_sprecta_pt_idx_int</code>	3424	<code>\l__spintent_spunit_luaset_is_-</code>	
<code>\l__spintent_sprecta_pt_seq</code>	3424	<code>sexagesimal_str</code>	533
<code>\l__spintent_sprecta_refs_tl</code>	3424	<code>\l__spintent_spunit_luaset_lookup_status_-</code>	
<code>__spintent_spsshort_execute_layout:</code> ...	1176	<code>str</code>	533
<code>\l__spintent_spsshort_luaset_base_str</code> ..	1093	<code>\l__spintent_spunit_luaset_read_str</code> ...	533
<code>\l__spintent_spsshort_luaset_expand_str</code>	1093	<code>\l__spintent_spunit_luaset_status_str</code> .	533
<code>\l__spintent_spsshort_luaset_layout_str</code>	1093	<code>\l__spintent_spunit_mult_seq</code>	540
<code>\l__spintent_spsshort_luaset_output_str</code>	1093	<code>__spintent_spunit_process_base_exp:n</code> .	611
<code>\l__spintent_spsshort_luaset_status_str</code>	1093	<code>__spintent_spunit_process_mult:n</code>	663
<code>\l__spintent_spsshort_luaset_suffix_str</code>	1093	<code>\l__spintent_spunit_read_cdot_bool</code> ...	540
<code>__spintent_spsshort_print:nn</code>	1129	<code>\l__spintent_spunit_read_slash_str</code> ...	540
<code>__spintent_spsshort_process_and_-</code>		<code>__spintent_spunit_safe_exec:n</code>	682
<code>render:n</code>	1141	<code>\l__spintent_spunit_unit_name_tl</code>	540
<code>__spintent_spsshort_render_bypass:n</code> ...	1152	<code>\l__spintent_spusep_arg_tl</code>	72, 73, 1512
<code>__spintent_spsshort_render_error:n</code> ...	1152	<code>__spintent_spusep_convert:n</code>	1574
<code>__spintent_spsshort_render_fallback:</code> ..	1152	<code>\l__spintent_spusep_left_tl</code>	72, 73, 1512
<code>__spintent_spsshort_render_success:</code> ...	1152	<code>\l__spintent_spusep_right_tl</code>	72, 73, 1512
<code>__spintent_spsshort_tag_span:nn</code>	1102	<code>__spintent_spusep_set_parens:nn</code>	1563
<code>__spintent_spside_exec:nnn</code>	3960	<code>\l__spintent_spusep_silent_bool</code>	73
<code>\l__spintent_spside_intent_arg_tl</code>	3852	<code>__spintent_spvec_exec:nnn</code>	3388
<code>\l__spintent_spside_intent_tl</code>	3852	<code>\l__spintent_spvec_intent_arg_tl</code>	3185
<code>__spintent_spside_mathml:n</code>	3885	<code>\l__spintent_spvec_kv_tl</code>	3185
<code>\l__spintent_spside_mode_str</code>	3778	<code>__spintent_spvec_mathml:n</code>	3247
<code>__spintent_spside_points_loop:n</code>	3948	<code>\l__spintent_spvec_mode_str</code>	3185
<code>\l__spintent_spside_prefix_tl</code>	3778	<code>\l__spintent_spvec_prefix_tl</code>	3185
<code>__spintent_spside_process:nn</code>	3857	<code>__spintent_spvec_print:n</code>	3230
<code>\l__spintent_spside_pt_idx_int</code>	3852	<code>\l__spintent_spvec_pt_idx_int</code>	3185
<code>\l__spintent_spside_pt_seq</code>	3852	<code>\l__spintent_spvec_pt_seq</code>	3185
<code>\l__spintent_spside_read_arg_tl</code>	3778	<code>\l__spintent_spvec_refs_tl</code>	3185
<code>\l__spintent_spside_read_sty_str</code>	3778	<code>__spintent_spVol_attach_subscript:</code> ...	5996
<code>\l__spintent_spside_refs_tl</code>	3852	<code>\l__spintent_spVol_mode_str</code>	5863
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1233	<code>\l__spintent_spVol_native_bool</code>	5863
<code>\l__spintent_spsiglo_luaset_error_str</code> .	1233	<code>__spintent_spVol_print:</code>	5908
<code>\l__spintent_spsiglo_luaset_roman_min_-</code>		<code>\l__spintent_spVol_read_arg_full_tl</code> ...	5863
<code>str</code>	1233	<code>__spintent_spVol_read_arg_print:</code>	6028
<code>\l__spintent_spsiglo_print_era_str</code> ...	1233	<code>\l__spintent_spVol_read_arg_tl</code>	5863
<code>__spintent_spsiglo_print_math:n</code>	1317	<code>__spintent_spVol_silent:</code>	5915
<code>__spintent_spsiglo_print_text:n</code>	1317	<code>\l__spintent_spVol_sub_index_str</code>	5863
<code>__spintent_spsiglo_render_simple:nn</code> ..	1268	<code>\l__spintent_spVol_suffix_tl</code>	5863
<code>__spintent_spsiglo_render_with_-</code>		<code>__spintent_spVol_symbol_print:</code>	5957
<code>era:nnn</code>	1268	<code>__spintent_spVol_verbal:</code>	5936
<code>__spintent_spsiglo_tag_span:nn</code>	1304	<code>\l__spintent_spVol_word_tl</code>	5863
<code>__spintent_sptime_build_seconds:nn</code> ...	1397	<code>__spintent_unknown_keys:n</code>	230
<code>__spintent_sptime_build_tree:</code>	71, 1415	<code>__spintent_unknown_keys:nn</code>	230
<code>\l__spintent_sptime_luaset_base_str</code> ...	1383	<code>spintent.lua</code>	70
<code>\l__spintent_sptime_luaset_error_str</code> ..	1383	<code>\spmangle</code>	22, 121, 4250
<code>\l__spintent_sptime_luaset_has_seconds_-</code>		<code>spmangle-invalid-arg</code>	3992
<code>str</code>	1383	<code>\spmarc</code>	24, 127, 4409
<code>\l__spintent_sptime_luaset_is_fraction_-</code>		<code>spmarc-invalid-points</code>	4277
<code>str</code>	1383	<code>\spmcd</code>	15, 82, 2205

<code>\spmcn</code>	16, 82, 2205
<code>\spmoney</code>	6, 57, 1082
<code>\spmQ</code>	17, 93, 2786
<code>spmQ-not-integer</code>	2599
<code>spmQ-zero-denominator</code>	2599
<code>\spmside</code>	21, 117, 3965
<code>spmside</code>	5265
<code>spmside-invalid-points</code>	3785
<code>\spmsym</code>	13, 77, 1849
<code>\spmul</code>	13, 80, 2006
<code>spmul-empty-argument</code>	1965
<code>\spnD</code>	14, 86, 2377
<code>\spnewread</code>	11, 1708
<code>spnewread-duplicate</code>	1623
<code>spnewread-empty-args</code>	1623
<code>\spnewunit</code>	6, 56, 761
<code>spnewunit-duplicate</code>	740
<code>\spnM</code>	14, 86, 2377
<code>\spnum</code>	3, 47, 48, 515
<code>spnum-has-units</code>	501
<code>spnum-no-digits</code>	501
<code>\spnZ</code>	16, 92, 2584
<code>spnZ-not-integer</code>	2551
<code>\spPcirc</code>	31, 151, 5837
<code>spPcirc-invalid-arg</code>	5546
<code>\spPfig</code>	26, 133, 4978
<code>spPfig-invalid-arg</code>	4586
<code>spPfig-invalid-points</code>	4586
<code>spPfig-read-arg-key</code>	4586
<code>spPfig-sym-cardinality-conflict</code>	4586
<code>spPfig-sym-letra-conflict</code>	4586
<code>\spPoint</code>	18, 104, 3174
<code>spPoint-invalid-point</code>	3067
<code>\spPoly</code>	24, 130, 4567
<code>spPoly-invalid-points</code>	4433
<code>\spProp</code>	90, 2465
<code>\sprad</code>	28, 141, 5225
<code>\sprayo</code>	19, 114, 3767
<code>sprayo-invalid-points</code>	3651
<code>\sprazon</code>	89, 2455
<code>\spread</code>	11, 74, 1698
<code>spread-unknown-sym</code>	1623
<code>\sprecta</code>	19, 110, 3637
<code>sprecta-invalid-points</code>	3431
<code>\spshort</code>	8, 63, 1220
<code>\spside</code>	20, 117, 3965
<code>spside</code>	5265
<code>spside-invalid-points</code>	3785
<code>\spsiglo</code>	10, 66, 1370
<code>\sptime</code>	11, 69, 1447
<code>\spunit</code>	4, 52, 53, 55, 728
<code>spunit-invalid-unit</code>	547
<code>spunit-no-units</code>	547
<code>\spunitalias</code>	6, 56, 775
<code>spunitalias-duplicate</code>	740
<code>spunitalias-invalid-expr</code>	740
<code>\spusep</code>	12, 72, 1612
<code>spusep-empty-arg</code>	1501
<code>spusep-invalid-arg</code>	1501
<code>\spvec</code>	18, 106, 3413
<code>spvec-invalid-arg</code>	3192
<code>\spVol</code>	32, 158, 6056
str commands:	
<code>\str_if_eq:nnTF</code>	62
<code>\str_lowercase:n</code>	62
<code>\str_uppercase:n</code>	62
sub-index	5007 , 5265 , 5555 , 5870
sub-units	842
suffix	1730 , 1800
U	
unknown	1515 , 1654 , 1730 , 1800 , 2036 , 2234 , 2397 , 2613 , 3075 , 3196 , 3438 , 3658 , 3795 , 4002 , 4287 , 4440 , 4624 , 5007 , 5265 , 5555 , 5870
unknown-abbreviation	1099
unknown-choice	203
unknown-command-setup	154
upper-case	2036 , 5007
use-dfrac	2397
use-spnZ	2613
W	
wrap-parens	1874 , 1945 , 2535 , 2613